



MASTER THESIS

# Graph Neural Networks for Decentralized Multi-Agent Reinforcement Learning in Power-Grid Topology Control

---

Corentin Plumet

## Supervisors

Prof. Fink

Prof. Peng

Dr. Nejjar

August 2026

# Abstract

Power-grid topology control can relieve congestion by reconfiguring substation busbars, but fixed-width centralized actors scale poorly: their inputs and outputs grow with the grid and its combinatorial action space. Transmission systems are also organized into regions whose operators possess detailed local information and authority. Fully decentralized execution follows this structure, limits each actor to local observation and action spaces, and avoids a central coordinator. This thesis asks whether shared graph policies trained on the IEEE 14-bus system (**bus14**) can remain useful on the larger WCCI 2020 grid.

Using centralized-training, decentralized-execution MAPPO, we study sparse intervention, shared local graph encoders, and a grid-size-independent candidate-action scorer. We correct cross-grid input mismatch, use reduced WCCI action spaces, and compare zero-shot transfer, training from scratch, and fine-tuning. **To separate the source of transfer, we distinguish encoder-only transfer, where the grid-specific action heads are replaced and relearned on WCCI, from complete-policy transfer, where the candidate-action actor is reused without changing its output interface.**

On **bus14**, the strongest graph architectures exceed 98% mean survival, showing that the GNN actors can learn effective source-grid control. This does not imply that the same model family learns equally well when retrained on a larger grid. On the 24 difficult WCCI chronics, do nothing reaches 8.34% mean survival, while matched actors trained from scratch reach a median of only 9.6% at  $k = 64$ . In contrast, the architecture designed for weight transfer retains meaningful zero-shot performance: the best evaluated transferred graph actor, using the local safety gate, reaches 25.8%, about three times the do-nothing result. Transfer is therefore incomplete, but substantially more effective than retraining the same model family from scratch.

The main limitations are single-seed screens, 50 primary WCCI chronics, disabled WCCI maintenance, and one source–target grid pair. Graph encoders and candidate-action scoring remove the dimensional barrier to policy reuse, but structural compatibility alone does not ensure reliable control. Transfer also requires compatible inputs, compact action sets, policy-specific intervention control, and target-grid adaptation.

# Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                     | <b>i</b>  |
| <b>1 Research Questions and Contributions</b>                       | <b>1</b>  |
| 1.1 Thesis Title                                                    | 1         |
| 1.2 Central Thesis Question                                         | 1         |
| 1.3 Research Questions                                              | 1         |
| 1.4 Contributions                                                   | 2         |
| <b>2 Introduction</b>                                               | <b>3</b>  |
| 2.1 Topology control                                                | 3         |
| 2.2 Why topology control should be automated                        | 3         |
| 2.3 Why use decentralized agents                                    | 4         |
| 2.4 Reinforcement learning for topology control                     | 4         |
| 2.5 Starting point: MARL2Grid and MAPPO                             | 5         |
| 2.6 Outline                                                         | 5         |
| <b>3 Establishing a Baseline</b>                                    | <b>6</b>  |
| 3.1 Evaluation Protocol                                             | 6         |
| 3.2 IEEE 14-Bus Experiments                                         | 6         |
| 3.2.1 MLP Baseline                                                  | 6         |
| 3.2.2 Initial Do-Nothing Bias                                       | 8         |
| <b>4 Sparse Intervention Control</b>                                | <b>10</b> |
| 4.1 Evaluation-Time Rho Heuristics                                  | 10        |
| 4.2 Intervention-Gated Actor                                        | 11        |
| 4.3 Fixed Sparse-Action Penalty                                     | 11        |
| 4.4 Adaptive Intervention Budget                                    | 12        |
| 4.5 Behavioral Cloning from Heuristics                              | 12        |
| 4.6 Results                                                         | 12        |
| 4.7 Preliminary WCCI Stress Test                                    | 14        |
| <b>5 Methods</b>                                                    | <b>15</b> |
| 5.1 Graph Representations                                           | 15        |
| 5.1.1 Busbar Representation                                         | 16        |
| 5.1.2 Disaggregated Representation                                  | 16        |
| 5.1.3 Disaggregated Line-Node Representation                        | 17        |
| 5.1.4 Local actor information boundary and the global state graph   | 18        |
| 5.1.5 Graph actor and centralized critic                            | 19        |
| 5.2 Substation Representation: Direct Edges and Summary Nodes       | 20        |
| 5.2.1 Direct same-substation edges (e)                              | 20        |
| 5.2.2 Substation summary nodes (n)                                  | 21        |
| 5.3 Graph Readout                                                   | 21        |
| 5.3.1 Mean and maximum graph readout                                | 21        |
| 5.3.2 Virtual-node graph readout                                    | 22        |
| 5.4 Candidate-Action Scoring from the Disaggregated Line-Node Graph | 23        |
| 5.4.1 From action indices to candidate scoring                      | 23        |
| 5.4.2 What each action touches                                      | 24        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 5.4.3     | Candidate pooling of the touched nodes                 | 24        |
| 5.4.4     | Scoring and the idle action                            | 25        |
| 5.4.5     | Global-maximum and action-delta extension              | 25        |
| 5.5       | Cross-Grid Input Transformations                       | 26        |
| 5.5.1     | Physical scaling                                       | 26        |
| 5.5.2     | Angle representation                                   | 27        |
| 5.6       | Action-Space Reduction for Large Grids                 | 27        |
| <b>6</b>  | <b>Graph-Design Screening</b>                          | <b>29</b> |
| 6.1       | Shared Protocol                                        | 29        |
| 6.1.1     | Endpoint statistics                                    | 29        |
| 6.1.2     | Compute                                                | 30        |
| 6.2       | Screen A: Message-Passing Operator                     | 30        |
| 6.3       | Screen B: Encoder Depth and Width                      | 30        |
| 6.4       | Screen C: Graph Readout Scope and Reducer              | 32        |
| 6.5       | Screen D: Structural Augmentations of the Busbar Graph | 34        |
| 6.6       | Screen E: Generator and Load Message Direction         | 35        |
| 6.7       | Screen F: Candidate-Action Scoring                     | 36        |
| <b>7</b>  | <b>What the Shared Encoder Reads</b>                   | <b>39</b> |
| 7.1       | Measurement protocol                                   | 39        |
| 7.2       | What the encoder reads                                 | 40        |
| 7.3       | Results                                                | 40        |
| 7.4       | Two failure modes                                      | 41        |
| 7.5       | Correcting the inputs                                  | 42        |
| 7.5.1     | Running standardization makes the mismatch worse       | 42        |
| 7.5.2     | Physical scaling for the power channels                | 43        |
| 7.5.3     | The angle displacement is a gauge artifact             | 43        |
| 7.5.4     | The corrected distribution                             | 44        |
| <b>8</b>  | <b>WCCI36: Target Environment and Controls</b>         | <b>46</b> |
| 8.1       | Target environment                                     | 46        |
| 8.1.1     | Removing maintenance                                   | 46        |
| 8.1.2     | Reduced action spaces                                  | 47        |
| 8.2       | Evaluation protocol                                    | 47        |
| 8.3       | Inputs used for transfer                               | 47        |
| 8.4       | Controls trained on WCCI                               | 47        |
| <b>9</b>  | <b>Encoder-Only Transfer Across Grids</b>              | <b>50</b> |
| 9.1       | Question and hypotheses                                | 50        |
| 9.2       | Transferred components and controls                    | 50        |
| 9.3       | Results                                                | 52        |
| 9.3.1     | Learning dynamics                                      | 52        |
| 9.3.2     | Full-test comparison                                   | 52        |
| 9.4       | Interpretation rules                                   | 53        |
| <b>10</b> | <b>Complete-Policy Transfer and Target Adaptation</b>  | <b>54</b> |
| 10.1      | Comparison rules                                       | 54        |
| 10.2      | What controls zero-shot transfer?                      | 54        |
| 10.3      | Policy changes and the local heuristic                 | 56        |
| 10.4      | Does candidate-policy target training help?            | 58        |
| 10.5      | Limits of the study                                    | 58        |
| <b>A</b>  | <b>Additional Experimental Evidence</b>                | <b>60</b> |
| A.1       | Sparse Intervention Control                            | 60        |
| A.1.1     | Additional method details                              | 60        |
| A.1.2     | Exact bus14 results                                    | 61        |
| A.1.3     | Preliminary maintenance-enabled WCCI pilot             | 62        |
| A.2       | Graph-Design Screening                                 | 63        |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| A.2.1    | Screen B: Encoder Depth and Width . . . . .                      | 63        |
| A.2.2    | Screen C: Graph Readout Scope and Reducer . . . . .              | 64        |
| A.2.3    | Screen D: Structural Augmentations of the Busbar Graph . . . . . | 65        |
| A.2.4    | Screen E: Generator and Load Message Direction . . . . .         | 66        |
| A.2.5    | Screen F: Candidate-Action Scoring . . . . .                     | 68        |
| A.3      | <b>Cross-Grid Input Analysis</b> . . . . .                       | 69        |
| A.3.1    | Encoder Input Distributions . . . . .                            | 69        |
| A.3.2    | Deterministic Feature Scales . . . . .                           | 70        |
| A.4      | <b>WCCI Transfer and Target-Control Results</b> . . . . .        | 71        |
| A.4.1    | Zero-Shot and Intervention Matrices . . . . .                    | 71        |
| A.4.2    | Target-trained controls . . . . .                                | 73        |
| A.4.3    | Per-chronic diagnostic . . . . .                                 | 76        |
| <b>B</b> | <b>Graph-Policy Implementation Details</b> . . . . .             | <b>77</b> |
| B.1      | Shared input and encoder pipeline . . . . .                      | 77        |
| B.2      | GINE message passing . . . . .                                   | 77        |
| B.3      | <b>Graph readout and output</b> . . . . .                        | 78        |
| B.4      | Agent-specific action head . . . . .                             | 78        |
| B.5      | Candidate-action scorer implementation . . . . .                 | 79        |
| B.6      | Exact node inputs by representation . . . . .                    | 79        |
| B.7      | Exact edge inputs by representation . . . . .                    | 80        |
| B.8      | <b>Graph Construction and Local Boundaries</b> . . . . .         | 80        |
| B.8.1    | Candidate graph sizes . . . . .                                  | 81        |
| B.8.2    | Indexing and masking conventions . . . . .                       | 81        |
| B.8.3    | Substation augmentation details . . . . .                        | 81        |
| B.8.4    | Local actor graphs . . . . .                                     | 82        |
| B.8.5    | Boundary verification . . . . .                                  | 83        |
| <b>C</b> | <b>Reproducibility Summary</b> . . . . .                         | <b>84</b> |
| C.1      | Campaign protocols . . . . .                                     | 84        |
| C.2      | Shared MAPPO settings . . . . .                                  | 84        |
| C.3      | Exploratory GINE variants . . . . .                              | 85        |
| C.4      | Checkpoint and artifact provenance . . . . .                     | 87        |

# Chapter 1

## Research Questions and Contributions

### 1.1 Thesis Title

Graph Neural Networks for Decentralized Multi-Agent Reinforcement Learning in Power-Grid Topology Control

### 1.2 Central Thesis Question

Which graph architecture and action-selection design provide the most effective representation and control performance for decentralized policies, and to what extent can a policy trained on a small grid retain useful performance on a larger grid?

### 1.3 Research Questions

1. **Baseline learning:** Which training changes are needed for stable decentralized MAPPO topology control on `bus14`? Chapter 3 establishes the reference through the training, initialization, and initial-action-bias experiments.
2. **Sparse intervention:** Can the agents reduce unnecessary topology interventions without sacrificing source-grid survival? Chapter 4 compares evaluation-time heuristics, a learned intervention gate, action penalties, adaptive budgets, and behavioral cloning on both survival and intervention frequency.
3. **Graph representation:** Which graph representation, message-passing, graph-readout, and candidate-action design choices produce a useful local actor? Chapter 5 defines these choices and Chapter 6 evaluates them on `bus14`.

4. **Cross-grid transfer and adaptation:** To what extent do graph-actor weights learned on `bus14` retain useful performance on WCCI, and when zero-shot transfer is insufficient, does source initialization improve target-grid learning over training from scratch? Chapters 7–10 diagnose input compatibility, define the target protocol, test whether a frozen source encoder remains useful after replacing its action heads, and finally compare complete-policy zero-shot transfer with scratch training and fine-tuning.

## 1.4 Contributions

- **A stabilized decentralized MAPPO reference on `bus14`.** Chapter 3 isolates the training and initialization changes used by the later experiments.
- **A source-grid study of sparse intervention.** Chapter 4 evaluates several mechanisms under the joint criteria of survival and intervention frequency rather than survival alone.
- **A shared graph actor with size-independent candidate-action scoring.** Chapters 5 and 6 define the local graph, score actions from the nodes they affect, and screen representation, message-passing, graph-readout, and action-scoring choices.
- **A diagnosis and correction of cross-grid input mismatch.** Chapter 7 measures what the shared encoder receives on both grids and motivates physically compatible power and angle inputs.
- **A controlled WCCI transfer and adaptation study.** Chapters 8–10 separate encoder-only transfer from complete-policy transfer, compare original and corrected input preprocessing, and evaluate zero-shot deployment, WCCI training from scratch, and source-initialized adaptation.

# Chapter 2

## Introduction

### 2.1 Topology control

Although a transmission grid is often represented as a fixed graph, operators can change its topology. Inside each substation, transmission lines, generators, and loads are connected to one or more busbars. Moving an element from one busbar to another, or splitting a substation into two separate electrical nodes, changes the paths available to the power flow. The loading of the lines changes as a result. This type of reconfiguration is called topology control.

This flexibility is attractive because it can reduce congestion without adding new infrastructure. The usual alternatives are to redispatch generation, curtail renewable production, or build new transmission lines. Redispatch and curtailment cost money whenever they are used, while new lines are expensive and take years to build. Topology control instead uses switching equipment that is already installed, so the marginal cost of an action is close to zero [1]. This option becomes even more useful as renewable production increases, since variable generation makes power flows less predictable and pushes existing lines closer to their limits.

Its low operating cost does not make topology control easy to use. In practice, operators rely on a limited set of remedial actions prepared from experience and offline studies. Searching every possible configuration during operation is not realistic, which means that much of the available flexibility remains unused [2].

### 2.2 Why topology control should be automated

Two features of the problem make manual search difficult. The first is the number of possible actions. If a substation has  $d$  connected elements and two busbars, it already has  $2^d$  possible assignments. Considering several substations together makes this number grow very quickly. Even the IEEE 14-bus system used in this thesis has 178 distinct unitary topology actions, while the 36-substation WCCI grid in Chapter 8 has more than 66 000. Its action space must therefore be reduced before training (Section 5.6).

The second difficulty is that these decisions are linked over time. An action taken now changes the grid state and may affect which contingencies can be handled later. Cooldown rules can also prevent another action at the same substation for several time steps. The operator must therefore choose quickly without considering only the immediate effect of an action.

Together, these properties make reinforcement learning (RL) a reasonable method to investigate. An RL agent learns through repeated interaction with a simulator, where each decision changes the next state and produces feedback. There is no simple formula for the best action, but the objective is clear: keep the grid operating and avoid a blackout. RTE created the Learning to Run a Power Network (L2RPN) competitions to study this problem. Grid2Op supports these experiments by providing the simulator, the load and generation time series, and the operating rules [3], [4].

## 2.3 Why use decentralized agents

Most published methods use one agent to control the full grid, but this is not how large power systems are usually operated. The network is divided into regions, and each control centre has detailed information about its own area and a less complete view of the rest of the system [2]. A decentralized model follows this organization more closely: each actor receives local observations, controls the substations in its region, and can operate without a central controller at execution time.

The same organization also helps with scalability. A central actor must process the full grid and produce scores for all available actions, so both its input and output grow with the network. In a multi-agent system, each actor chooses only among the actions of its own region. Adding another region therefore does not require every actor to represent the full joint action space.

Decentralization does not remove the difficulty of the problem; it changes its nature. Because all agents update their policies during training, the environment appears non-stationary from the point of view of any one agent [5]. Local observations also hide part of the grid, so two actions that appear safe separately may be harmful when applied together.

Centralized training with decentralized execution helps address these issues. During training, a critic can use the global state to evaluate the joint behaviour of the agents, while each actor sees only the information that will be available in its region at deployment time [6], [7]. The critic is removed after training, leaving actors that operate independently. This is the setting used throughout the thesis.

## 2.4 Reinforcement learning for topology control

Previous RL work on topology control can be grouped into four main approaches. A broader review of graph-based RL for power systems is provided by Hassouna et al. [8].

**Single-agent policies.** The earliest approaches train one agent to control the whole grid. Subramanian et al. [9] showed that deep RL with a selected action set can outperform a do-nothing policy on L2RPN benchmarks. SMAAC, the winner of the L2RPN WCCI 2020 challenge, combines actor-critic learning over *afterstates* with a hierarchical decision about where and how to act [10]. These methods can perform well, but the action space must first be reduced, selected, or factorized.

**Policies combined with heuristics.** A second approach uses simple rules around a learned policy. A common choice is to query the agent only when the maximum line loading  $\rho_{\max}$  becomes high, and to do nothing while the grid remains safe. PowRL combined this rule with an overload manager and a reduced action set to win the L2RPN NeurIPS 2020 robustness track [11]. Lehna et al. [12] found similar survival for advanced rule-based and RL agents, although RL was cheaper at execution time.

**Hierarchical and centrally coordinated multi-agent methods.** A third family of methods divides the decision between several components while keeping a central coordinator. Manczak et al. [13] separate the decisions of when, where, and how to act. van der Sar et al. [14] use substation agents under a higher-level selector, while de Mol et al. [15] use a central agent to choose between regional proposals. These methods reduce the size of each decision, but a component still observes the full grid and coordinates the agents. They therefore do not show whether local policies can operate on their own.

**Decentralized multi-agent control.** Fully decentralized control has received less attention. Here, agents use local observations and act without communication or a central selector, which is the setting studied in this thesis. MARL2Grid-TR addresses the same gap because earlier research mainly considered single-agent control and did not represent the decentralized organization of grid operation [16].

**Graph representations.** Regardless of the control structure, the representation of the grid matters. A flat vector hides the network structure and has a size tied to one grid. A graph neural network (GNN) follows the grid connections and can use the same parameters on networks of different sizes [17], [18], [19].

However, a homogeneous graph may omit connections that a topology action could create. Heterogeneous graphs can reduce this *busbar information asymmetry* [20]. Chapters 5 and 6 compare these choices.

## 2.5 Starting point: MARL2Grid and MAPPO

This work builds on MARL2Grid [16], the multi-agent extension of RL2Grid [21]. MARL2Grid connects Grid2Op to a multi-agent training pipeline and defines the regions, observations, action spaces, rewards, and training loop used as the starting point of this thesis.

**Grid partition.** The grid is divided in advance into fixed, non-overlapping regions, with one agent assigned to each region. In the IEEE 14-bus environment (**bus14**), the three agents control substations  $\{0, 1, 2, 4\}$ ,  $\{3, 6, 7, 8\}$ , and  $\{5, 9, \dots, 13\}$ . These regions contain 52, 50, and 76 non-idle actions, which together form the 178 unitary actions of the full grid. Each agent observes its own region and chooses an action at one of its substations. The agents then act at the same time without seeing each other’s choices, and action 0 always means do nothing.

**Reward and evaluation metric.** The reward is mainly based on survival, with +1 given for each survived step and an additional term that penalizes overloads. Training therefore encourages the agents to keep the episode alive. For one chronic, survival is defined as the number of survived time steps divided by the maximum episode length, and mean survival is this value averaged over the evaluated chronics.

**MAPPO.** Multi-agent PPO (MAPPO) [7] extends PPO [22] to centralized training with decentralized execution. Each actor maps its local observation to a probability distribution over its local actions. During training, PPO limits the size of each policy update so that the actors do not change too abruptly. A centralized value function can use the global state to evaluate the agents and estimate the advantage of their actions. This global information is never given to the actors, which continue to use only their local observations. Once training is complete, the centralized value function is no longer needed and deployment remains decentralized.

**Initial baseline.** The initial model uses flat multilayer-perceptron actors and one critic trained on **bus14**. Without further tuning, this baseline is unstable and has low survival. It is also tied to one grid because the actor input size depends on the number of substations, so its weights cannot be used directly on a larger network. The next chapters address these limitations in sequence: they first stabilize training, then reduce unnecessary interventions, and finally replace the fixed-size actor with a graph-based model.

## 2.6 Outline

The thesis is organized in two stages. Chapters 3 and 4 first focus on learning a reliable decentralized controller on **bus14**. Chapter 3 establishes a stable flat MAPPO baseline, and Chapter 4 studies how to reduce unnecessary interventions. Its preliminary WCCI stress test also reveals the limitations of applying a fixed-size actor to a larger grid.

Chapters 5–10 then develop and evaluate cross-grid transfer. Chapter 5 introduces the graph actor and candidate-action scorer, and Chapter 6 evaluates their main design choices on **bus14**. Chapter 7 makes the physical inputs more comparable across grids, while Chapter 8 defines the WCCI environment and evaluation protocol. Chapter 9 tests whether the encoders selected by Screens A–E remain useful when their grid-specific action heads are replaced. Chapter 10 then studies the stricter case in which the complete candidate-action policy is transferred, with and without target adaptation.

## Chapter 3

# Establishing a Baseline

This chapter builds a solid MLP actor baseline for the IEEE 14-bus task.

### 3.1 Evaluation Protocol

All experiments use an 80/20 train/test chronic split. The 80% training split is used for policy updates, while the 20% test split measures how well the policy generalizes. During training, checkpoint evaluation uses 20 chronics from the training split. Each evaluation is a new rollout, so these 20 chronics change from one evaluation to the next rather than forming a fixed subset. The checkpoint-selection score is mean survival over the 20 chronics in the current rollout.

For one chronic, survival is the number of survived time steps divided by that chronic’s maximum episode length. Mean survival is the unweighted mean of these normalized durations over the evaluated chronics. Results are reported as mean curves across seeds, with shaded regions showing the spread. Final evaluations report the checkpoint with the highest training-split checkpoint-selection score.

### 3.2 IEEE 14-Bus Experiments

The initial MAPPO baseline from the MARL2Grid framework led to unstable policies and low mean survival on the IEEE 14-bus topology-control task. The experiments below test changes to the training and to the architecture, in order to build a strong baseline.

#### 3.2.1 MLP Baseline

**Question.** Which changes are needed to get a stable MLP MAPPO baseline on the 14-bus task?

**Baseline and changes.** We start from the MLP MAPPO baseline of the MARL2Grid framework and test several main changes meant to stabilize training:

- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent;
- tuning the optimization hyperparameters to stabilize training, as summarized in Table [3.1](#).

Table 3.1: New hyperparameters for MAPPO training.

| Hyperparameter                       | New Value  | Paper Baseline |
|--------------------------------------|------------|----------------|
| <b>ENVIRONMENT</b>                   |            |                |
| n_envs (number parallel environment) | 20         | 10             |
| <b>PPO OPTIMIZATION</b>              |            |                |
| Total time step                      | 15,000,000 | 25,000,000     |
| Actor learning rate                  | 1e-4       | 3e-5           |
| Critic learning rate                 | 1e-4       | 3e-5           |
| Gamma                                | 0.9        | 0.99           |
| Update epochs                        | 10         | 80             |
| Minibatches                          | 16         | 4              |
| Maximum gradient norm                | 1.0        | 10.0           |

**Evidence.** Table 3.3 reports final and peak mean survival for the core ablations. Figure 3.1 overlays all core MLP baseline curves.

**Interpretation.** The **Optimized MLP Baseline** is retained because it gives the best overall balance of high survival, fast convergence, and stable training. Some ablations reach comparable or slightly higher isolated endpoints, but their trajectories are slower or more variable. The selection therefore reflects stability and performance across training rather than the final point alone.

Table 3.2: MLP baseline run configurations. The reference run is **Optimized MLP Baseline**.

| Run family                      | Actor/critic hidden layers    | Reward norm. | Initial action-0 prob. | Optimized critic update | Difference from <b>Optimized MLP Baseline</b> |
|---------------------------------|-------------------------------|--------------|------------------------|-------------------------|-----------------------------------------------|
| <b>Optimized MLP Baseline</b>   | $3 \times 256 / 3 \times 256$ | true         | 0.0                    | true                    | Reference baseline                            |
| Disable optimized critic update | $3 \times 256 / 3 \times 256$ | true         | 0.0                    | false                   | Disables optimized critic updates             |
| Smaller MLP actor/critic        | $2 \times 256 / 2 \times 256$ | true         | 0.0                    | true                    | Uses a smaller MLP actor and critic           |
| Disable reward normalization    | $3 \times 256 / 3 \times 256$ | false        | 0.0                    | true                    | Disables reward normalization                 |
| Initial Bias 50%                | $3 \times 256 / 3 \times 256$ | true         | 0.5                    | true                    | Uses action-0 bias 0.5                        |
| Initial Bias 70%                | $3 \times 256 / 3 \times 256$ | true         | 0.7                    | true                    | Uses action-0 bias 0.7                        |

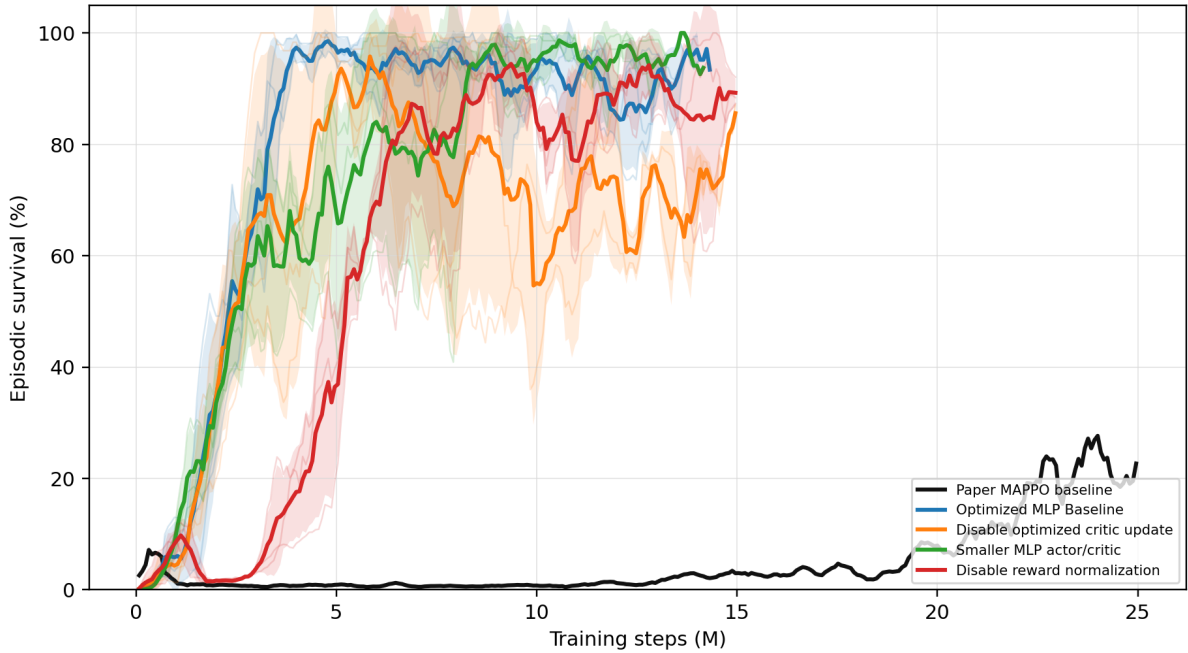


Figure 3.1: All MLP baseline core rerun survival curves shown on one axis.

Table 3.3: MLP baseline core rerun mean-survival summary.

| Run                             | Intended change             | Final mean survival (%) | Peak mean survival (%) |
|---------------------------------|-----------------------------|-------------------------|------------------------|
| Optimized MLP Baseline          | Optimized MLP baseline      | 93.4                    | 98.6                   |
| Disable optimized critic update | Non-optimized critic update | 93.1                    | 95.8                   |
| Smaller MLP actor/critic        | Smaller MLP architecture    | 93.7                    | 100.0                  |
| Disable reward normalization    | No reward normalization     | 81.8                    | 97.1                   |

### 3.2.2 Initial Do-Nothing Bias

**Question.** Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning? The idea makes sense: under normal conditions, human operators usually leave the grid alone and take no action.

**Baseline and changes.** Our baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

**Implementation detail.** The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. It is implemented by raising the initial pre-softmax logits of the do-nothing action, so that the policy picks this safe action more often before any learning update happens. This makes the ablation a test of exploration bias rather than a test of model capacity.

**Evidence.** Table 3.4 and Figure 3.2 compare the zero-bias baseline with Initial Bias 50% and Initial Bias 70%.

**Interpretation.** The zero-bias model is retained because it reaches high survival earlier and more consistently. Although the 70% bias finishes higher, it learns slowly, while the 50% bias remains unstable. Neither gives a better overall stability–performance trade-off.

Table 3.4: Initial do-nothing bias mean-survival summary.

| Run                    | Final mean survival (%) | Peak mean survival (%) |
|------------------------|-------------------------|------------------------|
| Optimized MLP Baseline | 93.4                    | 98.6                   |
| Initial Bias 50%       | 50.8                    | 70.2                   |
| Initial Bias 70%       | 99.3                    | 100.0                  |

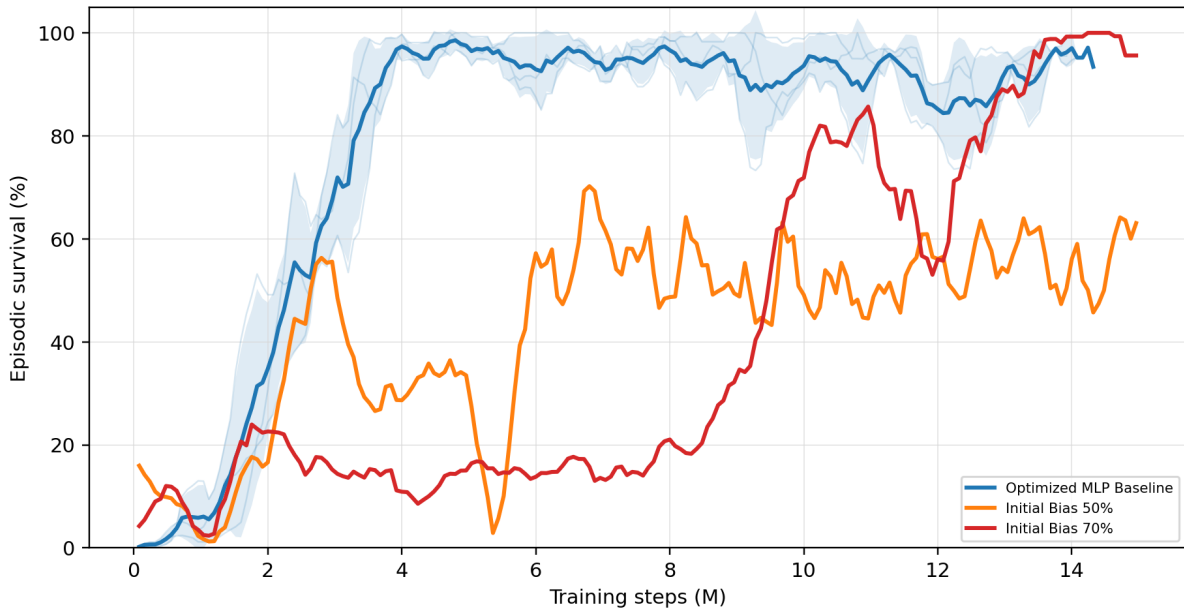


Figure 3.2: Optimized MLP baseline compared with initial do-nothing bias controls **Initial Bias 50%** and **Initial Bias 70%**.

### Chapter Summary

**Purpose:** Establish a stable, high-performing flat MAPPO reference on `bus14`.

**Content:** Compare the original training recipe with reward normalization, critic-update, model-size, and initial action-0-bias ablations.

#### Findings:

- **MLP optimization:** The selected reference enters the high-survival regime earlier and sustains it more consistently than the main training ablations. Its final value is about 93%, but it is not the largest isolated endpoint in the screen.
- **Do-nothing initialization:** A 50% bias is weak and unstable, whereas a 70% bias learns slowly before reaching a high final value. The effect therefore depends strongly on the bias level.

**Takeaway:** The tuned zero-bias actor is used as a stable reference because of its convergence trajectory, not because it wins every endpoint comparison. Chapter 4 next asks whether this actor can intervene less often without losing survival.

## Chapter 4

# Sparse Intervention Control

The previous chapter optimized survival on held-out chronics. This chapter adds a second objective: sparsity. A policy that keeps the grid alive by reconfiguring substations at every opportunity is impractical for a control room. Interventions carry operational costs, and an agent acting constantly is not trustworthy. The question is whether decentralized agents can keep high survival while intervening rarely, locally, and only when necessary.

In the literature, sparse activity is most often enforced by hard-coding a physical threshold that decides when the agent is allowed to act. Instead of relying on an external override, this chapter tries to make sparse intervention something the agent learns by itself.

Mechanisms are therefore measured on two competing axes: mean survival and intervention sparsity. A good mechanism must improve sparsity without hurting survival.

**Question.** Can the agents keep high survival while behaving more like operators: acting rarely and mainly when the grid state makes an intervention useful?

**Baseline and change.** This block starts from the flat decentralized MAPPO topology-control actor. For agent  $i$ , the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \quad a_i \in \{0, 1, \dots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{eval} = \arg \max_a \pi_i(a \mid o_{i,t}).$$

This chapter tests five ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, adaptive intervention budgets, and behavioral cloning from heuristics.

The intervention gate separates whether to act from which action to take, following the hierarchical splits used in topology control [13], [14]. The fixed and adaptive penalties instead treat non-idle interventions as a limited resource, following constrained and budgeted reinforcement learning [23], [24], [25].

### 4.1 Evaluation-Time Rho Heuristics

**Implementation detail.** The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action  $\bar{a}_{i,t}$ , then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise,} \end{cases} \quad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent  $i$  computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when  $\rho_{i,t}^{local} < \rho_{safe}$ . A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

**Interpretation.** The heuristic is useful to ask what happens if safe-state interventions are blocked, and if succesful, it can prove that the agents act too often.

## 4.2 Intervention-Gated Actor

**Implementation detail.** The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i | o_i), \quad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i | o_i), \quad b_i \in \{1, \dots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If  $g_{i,t} = 0$ , then  $a_{i,t} = 0$ . If  $g_{i,t} = 1$ , the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for  $a_i > 0$ ,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i | g_i = 1).$$

The exact evaluation rules used to convert the two heads into one action are documented in Appendix A.1.1.

**Interpretation.** The gate is learned and decentralized, but it can limit exploration more than a reward penalty does. If it collapses early to no-op, the non-idle head is rarely used and gets little learning signal.

## 4.3 Fixed Sparse-Action Penalty

**Implementation detail.** The fixed-penalty sweep adds a reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed} \mathbf{1}[a_{i,t} \neq 0],$$

with  $\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}$ . The complete 24-run actor, penalty, and seed grid is reported in Appendix A.1.1.

**Interpretation.** This mechanism changes only the training reward. It does not override actions during evaluation and it does not block exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost.

## 4.4 Adaptive Intervention Budget

**Implementation detail.** The adaptive intervention budget does not use a fixed intervention penalty. Instead, each agent has its own penalty coefficient  $\lambda_i$ , which is adjusted after every rollout. The local-safe version used in Figure 4.1 first converts the maximum line loading visible to agent  $i$  into a safety weight:

$$w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local})).$$

When the local grid is safely below  $\rho_{safe} = 0.90$ , this weight is high. It decreases as the local lines approach or exceed the threshold. The action cost is then

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]w_i^{local}(s_t).$$

Doing nothing always has zero cost. An intervention is expensive in a safe state but remains relatively cheap when the local grid is stressed and an action may be necessary. During the rollout, this cost is subtracted from the original reward:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

A large value of  $\lambda_i$  therefore makes interventions more expensive, whereas a small value leaves the policy freer to act.

At the end of the rollout, the average observed cost is

$$\hat{C}_i = \frac{1}{T} \sum_{t=1}^T c_i(s_t, a_{i,t}).$$

This value is compared with the target budget  $d$ , and the penalty coefficient is updated as

$$\lambda_i \leftarrow \text{clip}(\lambda_i + \eta(\hat{C}_i - d), 0, \lambda_{\max}).$$

If  $\hat{C}_i > d$ , the agent has exceeded its budget, so  $\lambda_i$  increases and future interventions become more expensive. If  $\hat{C}_i < d$ , the penalty decreases. The budget  $d$  is therefore a target used to adjust the penalty after each rollout; it is not a fixed amount subtracted from every reward. The value  $\rho_{safe} = 0.90$  is only the center of the smooth safety weight; it does not force the agent to do nothing. The multiplier hyperparameters and complete budget ablation are reported in Appendix A.1.1.

## 4.5 Behavioral Cloning from Heuristics

A teacher-student diagnostic tests whether the behavior of the local rho heuristic can be copied into the policy weights. The student is trained by behavioral cloning and evaluated without the heuristic. The training setup and exact results are reported in Appendix A.1.1.

## 4.6 Results

Figure 4.1 summarizes mean survival and the mean action-0 fraction across agents. Exact values and per-agent action-0 fractions are reported in Appendix A.1.

Table 4.1: Sparse-control mechanisms and where they affect the policy.

| Method             | Training effect                           | Evaluation effect                       | Main risk                                                               |
|--------------------|-------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------|
| Baseline           | Samples from the flat policy              | Greedy argmax by default                | No explicit mechanism to favor sparse control                           |
| Rho heuristic      | No training change                        | Hard no-op override in safe states      | Evaluation is manually changed                                          |
| Intervention gate  | Changes action factorization and sampling | Final-action MAP or hierarchical greedy | Gate collapse can starve non-idle exploration                           |
| Fixed penalty      | Fixed reward penalty on non-idle actions  | No action override                      | Penalty coefficient must be tuned                                       |
| AIB local-safe     | Adaptive state-dependent reward penalty   | No action override                      | Depends on the local rho                                                |
| Behavioral cloning | Distills heuristic into actor weights     | No action override                      | Safety-weight design<br>Struggles with extreme action-0 class imbalance |

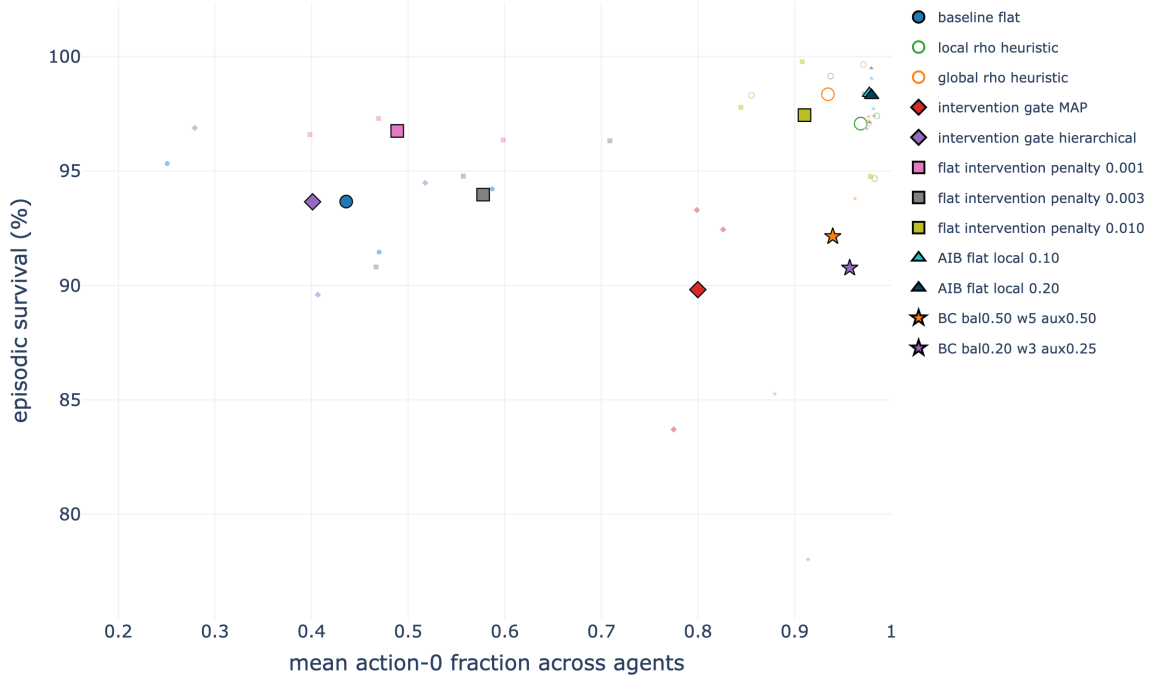


Figure 4.1: Joint action-0/mean-survival comparison of the selected sparse-control mechanisms on **bus14**. Large markers show condition means and faint markers the individual seeds. The plot uses checkpoint-aligned action-0 rates and full-test mean survival, so it shows the chapter’s two objectives at once.

**Interpretation.** The intervention gate is not the strongest method by itself. The fixed penalty with  $\lambda_{fixed} = 0.010$  already reduces unnecessary actions, while local-safe AIB with  $d = 0.20$  gives the best balance between survival and action-0 usage among the learned methods. It remains decentralized and allows the agents to act more freely when their local grid is stressed.

## 4.7 Preliminary WCCI Stress Test

As an early scale test, representative mechanisms were retrained from scratch with a fixed-width MLP actor on the maintenance-enabled WCCI grid. The mechanisms changed intervention frequency, but none reliably improved survival over the weak baseline, and the differences were small relative to seed variation. Because this pilot changes the grid, action space, and maintenance setting at the same time, it cannot isolate the cause. It nevertheless motivates the size-independent graph actor introduced next. The full protocol, table, and trade-off figure are retained in Appendix [A.1.3](#).

### Chapter Summary

**Purpose:** Keep survival while reducing unnecessary topology interventions.

**Content:** Compare evaluation-time rho overrides, a learned intervention gate, fixed action penalties, adaptive intervention budgets, and behavioral cloning from a sparse teacher.

**Findings:**

- **Evaluation heuristics** reach 97–98% mean survival with action 0 in 94–97% of decisions, but impose sparsity after the policy acts.
- **A fixed penalty** already gives a strong learned controller:  $\lambda_{fixed} = 0.010$  reaches 97.44% mean survival and a 0.905 action-0 fraction without an evaluation override.
- **Local-safe AIB** gives the best observed learned trade-off in this experiment: budgets  $d = 0.10$  and  $d = 0.20$  reach about 98.4% mean survival and 0.98 action-0 usage. The gated variants are less reliable and can collapse to almost permanent no-op behavior.

**Takeaway:** Sparse behavior can be learned on **bus14**, but the preliminary maintenance-enabled WCCI stress test does not isolate transfer and its fixed-width MLP policies remain weak. This motivates the graph representation introduced next.

# Chapter 5

## Methods

Chapter 4 made the flat policy behave more like an operator on `bus14`, but the first WCCI test was still weak. The sparse control mechanisms changed when the policy acted, but they did not solve a more basic problem: the MLP input and output sizes were tied to one grid. This chapter replaces that fixed representation with graph-based actors that can be used on grids of different sizes. It introduces the choices studied later in the thesis: graph construction, substation augmentations, graph readout, candidate-action scoring, cross-grid input transformations, and action-space reduction.

### 5.1 Graph Representations

I use three graph representations. They mainly differ in which physical objects receive their own node. The *busbar* graph places all measurements on busbar nodes. The *disaggregated* graph adds separate generator and load nodes, but still represents transmission lines as edges. The *disaggregated line-node* graph also turns each transmission line into a node.

Table 5.1 compares the three choices. The representation also changes what a local actor can see at the far end of a boundary line. The graphs therefore differ not only in size, but also in the information they contain. Section 5.1.4 describes this boundary in detail.

Table 5.1: The three graph schemas. Each row follows from how much equipment receives its own node. In the node counts,  $S$  is the number of substations,  $B$  the busbars per substation,  $N_g$  and  $N_d$  the included generators and loads, and  $L$  the included transmission lines.

| Aspect                                 | busbar                                                        | disaggregated                                                           | disaggregated line-node                                           |
|----------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>gnn_graph_type</b>                  | <b>bus</b>                                                    | <b>heterogeneous</b>                                                    | <b>heterogeneous_line</b>                                         |
| Node types                             | Busbar                                                        | Busbar, generator, load                                                 | Busbar, generator, load, transmission line                        |
| Generator and load state               | Summed and averaged onto the busbar                           | One node per asset                                                      | One node per asset                                                |
| Physical line                          | Direct busbar-to-busbar edge                                  | Typed direct busbar-to-busbar edge                                      | Typed node between its endpoint busbars                           |
| Line state                             | Continuous edge attributes                                    | Continuous edge attributes                                              | Features of the line node                                         |
| Shortest message path                  | One step between adjacent busbars                             | One step between adjacent busbars; two from an asset to a remote busbar | Two steps between adjacent busbars through a line node            |
| Hidden line state                      | None: the edge modifies a message                             | None: the edge modifies a message                                       | Yes: each line obtains and updates an embedding                   |
| Far-end context in a local actor graph | Live neighboring busbar, including aggregated power and angle | Live neighboring busbar, but no neighboring generator or load nodes     | None; the shared line node replaces the need for a far-end busbar |
| Node count                             | $SB$                                                          | $SB + N_g + N_d$                                                        | $SB + N_g + N_d + L$                                              |

The message-path row describes distances in each graph schema. It does not set the encoder depth  $K$ , which is selected experimentally in Chapter 6.

### 5.1.1 Busbar Representation

The busbar graph is the simplest and most compact representation. Busbars are the only nodes and active transmission lines are the edges. Generator and load measurements are added to the busbar to which each asset is currently connected.

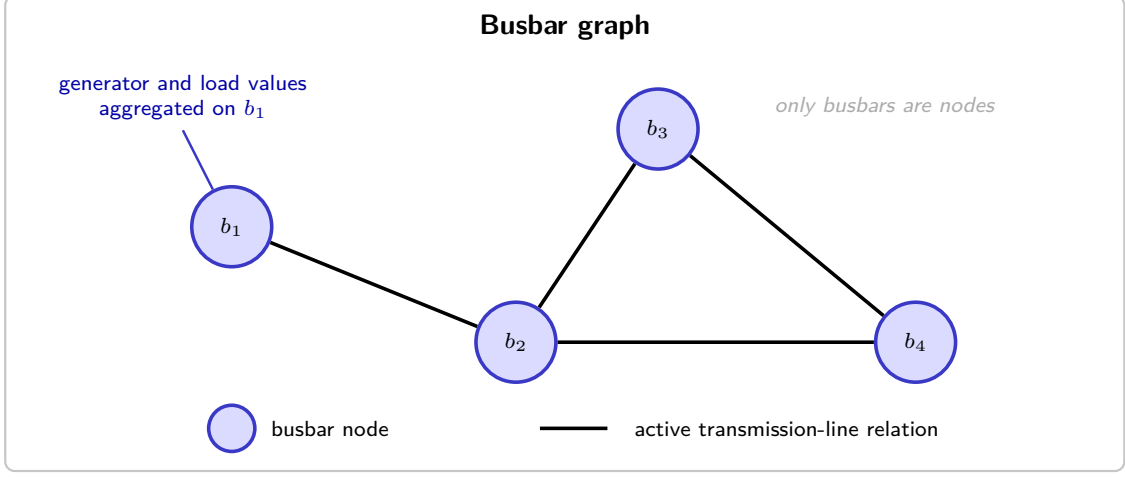


Figure 5.1: Busbar graph representation. Blue nodes are busbars and black connections are active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

#### Nodes and features

For  $S$  substations and at most  $B$  busbars per substation, the graph contains one node  $i(s, b)$  for every substation–busbar pair. Empty busbars are kept because a topology action may connect equipment to them later. Generator and load powers are summed on their current busbar, their angles are averaged, and the substation cooldown is copied to both busbars. The complete feature inventory is reported in Appendix B.6.

#### Relations and edge features

The only physical relation in this graph is a transmission line between two busbars. Each active line edge carries its status, loading, overload duration, and cooldown, together with maintenance timing when that information is available. Appendix B.7 gives the exact edge vector.

### 5.1.2 Disaggregated Representation

The disaggregated graph keeps the busbars as its backbone, but gives each generator and load a separate node. Typed directed edges connect these assets to their current busbar. Transmission lines remain direct edges between busbars. This avoids mixing the states of several assets into one busbar feature vector.

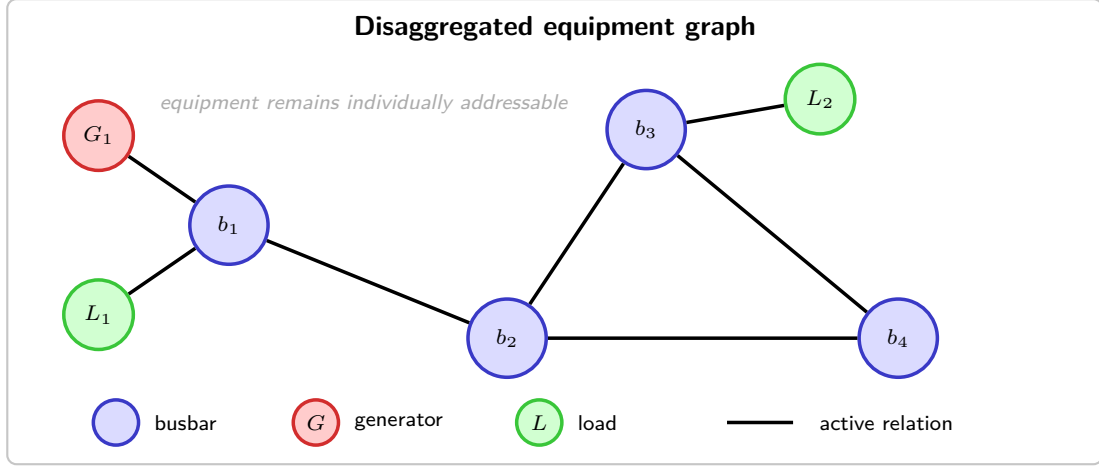


Figure 5.2: Disaggregated representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

### Nodes and features

The busbar nodes are the same as in Section 5.1.1. The graph then adds one node for every generator and load whose substation belongs to the local graph. All three node types use one common feature layout so that they can pass through the same encoder. Type indicators identify busbars, generators, and loads; physical values occupy only the columns relevant to that type. Disconnected assets remain indexed but their power and angle are zero. The ordered vector and per-type inventory are given in Appendix B.6.

### Relations and edge features

Every directed edge uses a common layout containing a physical line block and a one-hot relation block. Physical-line edges populate both; asset attachments and structural edges use a zero physical block and identify only their type and direction. This prevents a structural relation from looking like a measured line. The exact columns and activation rules are listed in Appendix B.7.

The direction of generator and load edges can be chosen independently. Each asset type can use `bidirectional`, `asset_to_busbar`, or `busbar_to_asset`. The default is `bidirectional`. Transmission-line and same-substation relations remain `bidirectional`.

### 5.1.3 Disaggregated Line-Node Representation

The disaggregated line-node graph goes one step further. Instead of a direct edge between two busbars, each transmission line becomes a node placed between them. Busbars, generators, loads, and lines are therefore all explicit nodes.

### Nodes and features

All four node types again share one ordered feature layout. Transmission-line state now occupies the line-node row, while generator and load rows carry their own power and angle. Optional maintenance values are included only when available. Appendix B.6 reports the exact columns, dimensions, and zero-filled entries.

### Relations and edge features

The edges now describe only the type and direction of each connection. They need no continuous line attributes because those values already belong to the line node. Every active edge therefore selects exactly one relation type, while inactive candidates are zero-filled and removed before message passing. The nine relation columns and activation rules are listed in Appendix B.7.

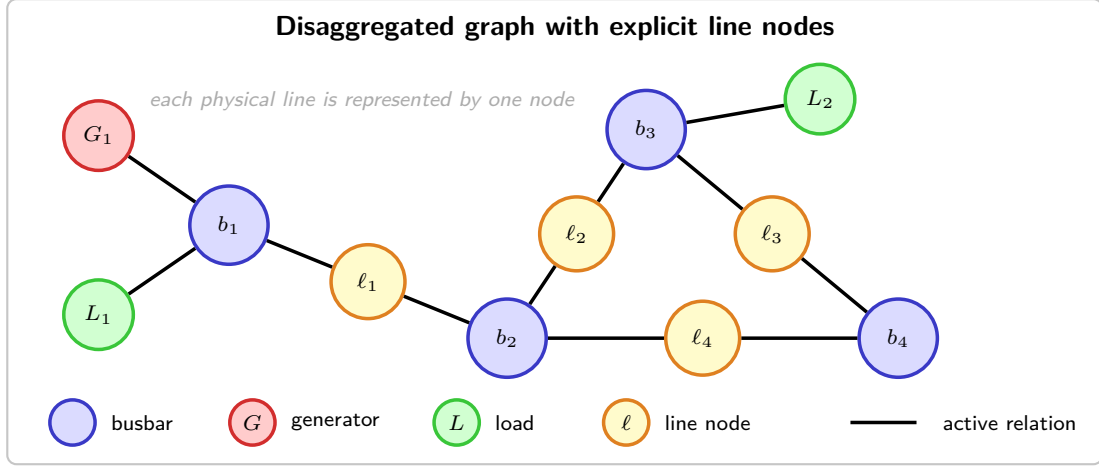


Figure 5.3: Disaggregated line-node representation. Yellow nodes represent physical lines and connect the blue busbars at their endpoints; generators and loads remain separate terminal nodes.

#### 5.1.4 Local actor information boundary and the global state graph

Agent  $a$  controls a set of substations  $\mathcal{C}_a$ . A boundary line links this region to a substation controlled by another agent. The local graph includes only the outside information needed to represent that shared line, and the exact amount depends on the graph representation.

**Why one hop, and why not more.** I use a one-hop boundary to keep the actor local while the critic still observes the full grid during training. Power flow is not truly limited to one hop, so this is a practical choice rather than a claim of optimality. The loading and angle difference of each boundary line provide the most direct interaction with the neighboring region without revealing all of its private asset states.

##### Visibility and masking

A neighboring busbar is visible only while an active boundary line connects it to the controlled region. If that connection disappears, the node features are set to zero, its edges are disabled, and the node is removed from the graph readout. The node therefore contributes neither to message passing nor to the graph embedding. A same-substation structural edge cannot make it visible again.

Controlled nodes remain visible even when they are disconnected, because the agent may reconnect equipment to them. In the line-node graph, a shared line node also remains present, but its attachment edges are disabled while the line is offline. An attachment edge is active only when the line is connected and the current busbar assignment matches that edge. Nodes for neighboring generators and loads are never created.

##### Representation-dependent information

The three representations expose different views of the neighboring region. Figure 5.4 makes this difference explicit; the corresponding inventory is retained in Appendix Table B.7.

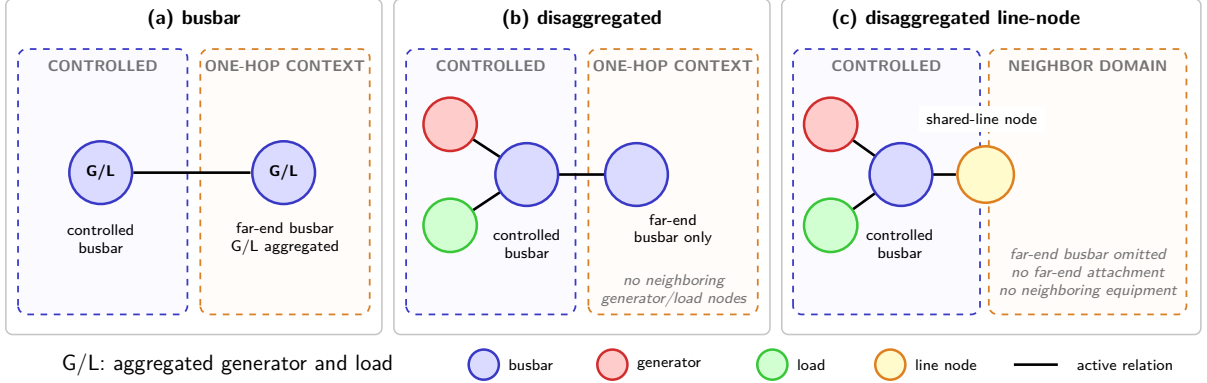


Figure 5.4: One live boundary line, seen under each representation. In every panel the blue dashed region is the agent’s controlled domain and the orange dashed region is what lies beyond it. The busbar graph (a) reaches a far-end busbar carrying the neighbor’s generation and load aggregated onto it. The disaggregated graph (b) reaches the same busbar, but no neighboring generator or load node is ever constructed, so that busbar carries no measurement of its own. The disaggregated line-node graph (c) reaches only the shared line node, and no far-end busbar exists at all.

In the busbar and disaggregated graphs, the far-end busbar is needed because the boundary line is an edge and an edge must have two endpoints. Only the busbar used by the active line is visible; the other candidate busbars remain masked. If the line disconnects, no far-end busbar remains visible. In the disaggregated graph, this neighboring busbar contains only type and domain indicators because the power and angle measurements belong to generator and load nodes, which are not created outside the controlled region. The boundary line edge is therefore the only source of neighboring measurements. The line-node graph removes the far-end busbar completely and stores the shared measurements on the line node. When the line disconnects, the line node remains with `line_status`= 0, while its attachment edges become inactive.

These information limits apply only to the actors. During training, the centralized critic still receives the normalized flat state of the full grid. At execution time, each actor uses only its own local graph. This is the centralized-training, decentralized-execution setting used by MAPPO [7].

### 5.1.5 Graph actor and centralized critic

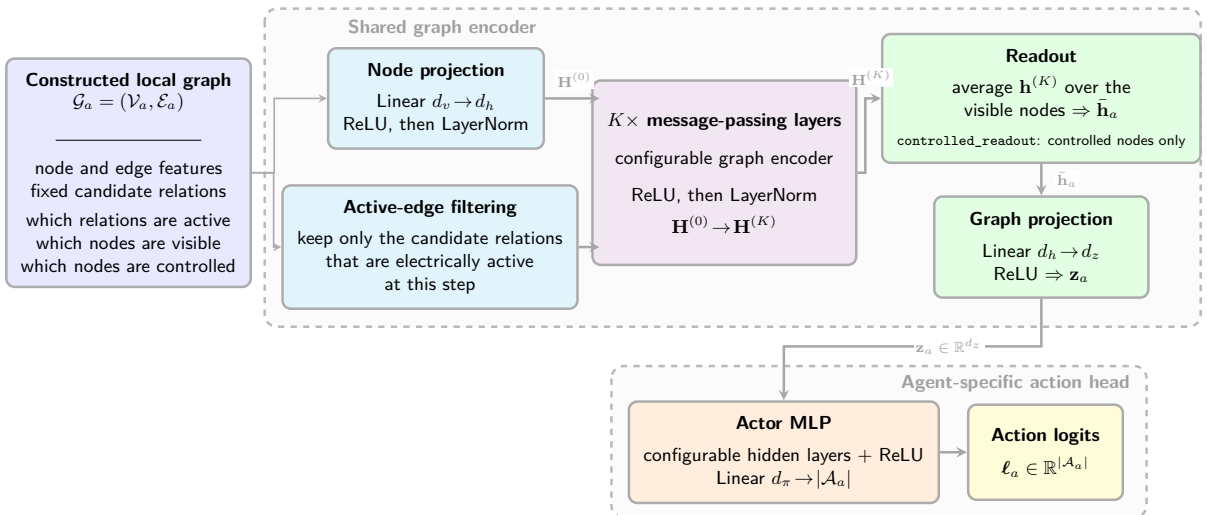


Figure 5.5: End-to-end graph-actor architecture. Candidate edges masked by the current topology are removed before message passing. The graph encoder and its output projection are shared across agents, while the final MLP has one output per action available to agent  $a$  and is therefore agent-specific.

Figure 5.5 shows the complete actor. The node features are first projected to a hidden width  $d_h$ . The encoder then applies  $K$  message-passing layers before reducing the node states to one graph embedding of width  $d_z$ . The encoder family, depth  $K$ , and hidden width  $d_h$  are experimental choices and are therefore not fixed in this chapter. They are selected in Chapter 6. The encoder and output projection are shared across agents, but the final action heads are separate because agents can have different numbers of actions.

The critic is a separate MLP that receives the normalized flat state. It does not reuse the actor’s graph embedding and is present only during training. Its normalization is independent of the graph-input transformation. The appendix records the exact encoder, graph-readout, output-projection, and action-head equations used by the evaluated configurations.

## 5.2 Substation Representation: Direct Edges and Summary Nodes

The base graphs do not directly connect the two busbars of one substation. I test two optional ways to add this internal communication. Factor  $e$  creates direct edges between the busbars. Factor  $n$  creates a learned summary node for the substation. In both cases, the added nodes and edges pass through the same projection, masking, and message-passing pipeline as the rest of the graph.

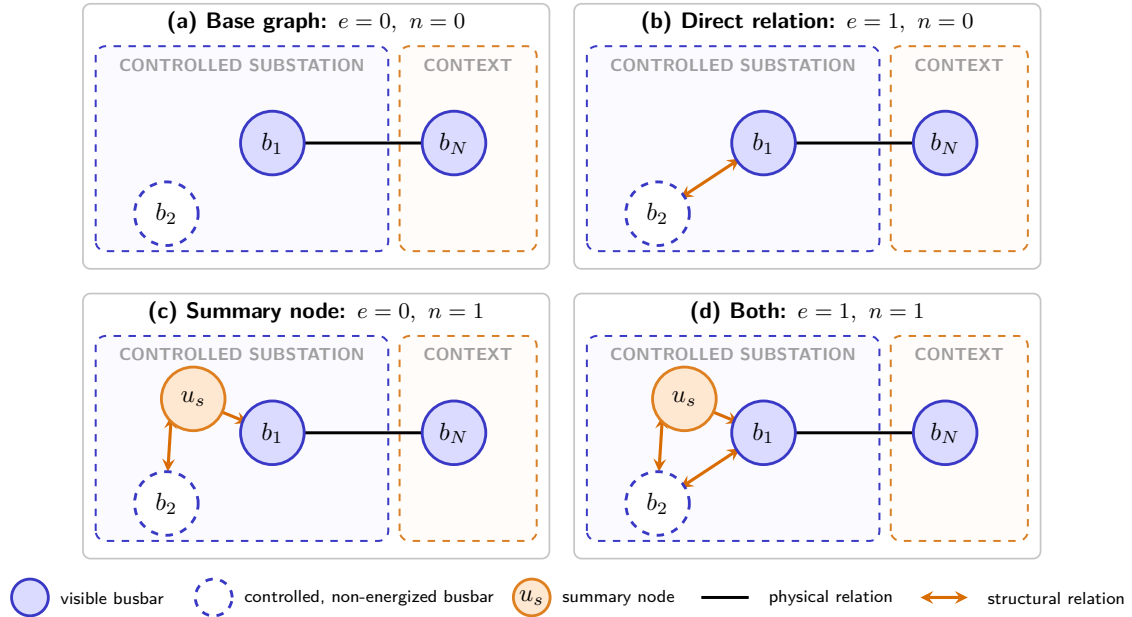


Figure 5.6: The four combinations of the independent substation factors  $e$  and  $n$ . The hollow node  $b_2$  is controlled but non-energized. Factor  $e$  adds a bidirectional relation between the controlled busbars, while factor  $n$  connects a summary node  $u_s$  to both. Neither augmentation extends to the contextual busbar  $b_N$ , which retains only its physical relation.

The placement of these augmentations in all three graph schemas, together with their exact feature values, is documented in Appendix B.8.3.

### 5.2.1 Direct same-substation edges ( $e$ )

Factor  $e$  adds the edge  $i(s, b) \rightarrow i(s, b')$  for every ordered pair of different busbars in the same controlled substation. Both directions are present and remain active throughout the episode. These are structural edges, not electrical lines. A controlled busbar can therefore exchange messages with the other busbar of its substation even when it is not energized, but it is still treated as electrically disconnected. Every physical edge value is zero because this structural relation carries information rather than power flow.

The schema-specific encoding is listed in Appendix Table B.6.

**Why controlled substations only.** The edge represents the actor’s ability to move equipment between busbars. Adding the same edge in a neighboring substation would reveal part of another agent’s action space and would create a nonphysical communication path.

### 5.2.2 Substation summary nodes (n)

Factor  $n$  adds one summary node  $u_s$  for each controlled substation. It is connected in both directions to every visible busbar of that substation, which adds one node and  $2B$  directed edges. Two busbars can then communicate through the path  $i(s, b) \rightarrow u_s \rightarrow i(s, b')$ . This serves the same purpose as the direct edge, but takes two message-passing steps instead of one. Only controlled busbars connect to the summary node, so it does not provide an extra path into a neighboring region.

The summary node is initialized from the structure of the substation, not from its identity. The vector  $\phi_s \in \mathbb{R}^4$  contains the numbers of busbars, connected lines, generators, and loads. The initialization is

$$u_s^{(0)} = \mathbf{W}_{\text{sub}} \phi_s + \mathbf{b}_{\text{sub}}, \quad \mathbf{W}_{\text{sub}} \in \mathbb{R}^{d_h \times 4}, \quad \mathbf{b}_{\text{sub}} \in \mathbb{R}^{d_h}. \quad (5.1)$$

The node starts with structural information only. Its state becomes dependent on the current observation after it receives messages from the busbars.

The construction is the same in all three graph schemas. A mean/max graph readout counts each  $u_s$  as one node. Factors  $e$  and  $n$  can be enabled together, and both can be combined with the virtual-node readout of Section 5.3.2.

## 5.3 Graph Readout

After message passing, the actor still needs one fixed-size vector for the full local graph. This step is called graph readout. It can reduce a chosen set of node embeddings with a mean or a maximum, or it can use the state of one learned virtual node. The same output projection and action head are used after either choice.

### 5.3.1 Mean and maximum graph readout

A graph readout makes two separate choices: which nodes to include and how to combine them. For each readout, let  $\mathcal{V}_{\text{sel}}$  denote the selected nodes. Depending on the readout type, this set contains all visible nodes, only controlled nodes, only energized nodes, or nodes that are both controlled and energized. A node is energized only if it touches an active electrical relation. The actor then applies either a mean or a component-wise maximum to the embeddings in  $\mathcal{V}_{\text{sel}}$ :

$$\bar{\mathbf{h}}_G^{\text{mean}} = \frac{1}{|\mathcal{V}_{\text{sel}}|} \sum_{v \in \mathcal{V}_{\text{sel}}} \mathbf{h}_v^{(K)}, \quad \left(\bar{\mathbf{h}}_G^{\text{max}}\right)_c = \max_{v \in \mathcal{V}_{\text{sel}}} \left(\mathbf{h}_v^{(K)}\right)_c. \quad (5.2)$$

The readout combines all selected node types together; it does not create a separate graph vector for each type. A node outside  $\mathcal{V}_{\text{sel}}$  does not contribute to the readout. Combining the four possible node populations with the two reducers gives the eight readouts in Table 5.2.

Table 5.2: The eight final graph readouts, named `[controlled_][energized_]{mean,max}`. Rows choose the readout population, columns the reducer. The virtual-node readout of Section 5.3.2 replaces the selected-node readout entirely.

| Selected node population       | mean                      | max                      |
|--------------------------------|---------------------------|--------------------------|
| All visible nodes              | mean                      | max                      |
| Controlled nodes               | controlled_mean           | controlled_max           |
| Energized nodes                | energized_mean            | energized_max            |
| Controlled and energized nodes | controlled_energized_mean | controlled_energized_max |

The selected population changes only the final graph readout. It does not remove nodes from message passing. A visible contextual node can still send information to controlled nodes even when it is not part of  $\mathcal{V}_{\text{sel}}$ . Screen C therefore compares graph readouts, not different information boundaries.

**What each axis trades.** The node scope and the reducer answer different questions, so their trade-offs should be considered separately (Table 5.3). Before running Screen C, the safest default was `controlled_mean`: the node population is stable, it matches the actor’s region, and it keeps empty busbars that may receive equipment. `controlled_max` is a more risk-sensitive choice when one critical node should dominate the decision. If one-hop context is disabled, the controlled and visible sets are identical.

Table 5.3: Trade-offs along each axis of Table 5.2.

| Choice                   | Main advantage                                                                                          | Main limitation                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <i>Reducer</i>           |                                                                                                         |                                                                                               |
| <b>mean</b>              | Smooth regional summary; gradients reach every pooled node.                                             | A rare but critical activation is diluted.                                                    |
| <b>max</b>               | Keeps the strongest activation and ignores how many nodes there are.                                    | Sparse gradients, and channel maxima may come from unrelated nodes.                           |
| <i>Pooled population</i> |                                                                                                         |                                                                                               |
| All visible              | Boundary conditions enter the readout directly.                                                         | Context can dilute a mean or dominate a max, and the population moves with boundary topology. |
| Controlled               | Aligned with the action domain and fixed for an episode; context still arrives through message passing. | Relies on message passing to carry every useful boundary signal.                              |
| Energized                | Focuses on the network currently carrying power.                                                        | Drops available action targets, and the population changes after each switching action.       |

### 5.3.2 Virtual-node graph readout

The virtual-node option replaces the selected-node readout with one learned summary node  $v_*$ . It can be used with any of the three graph representations. When substation-summary nodes are enabled, each controlled summary node sends one edge to  $v_*$ . Otherwise, every visible controlled busbar sends one edge to it. In all reported runs, these edges point only toward the virtual node. It can collect information but cannot send a message back into the physical graph. Contextual busbars, generators, loads, and line nodes do not connect to it directly; their information must first reach the controlled region through normal message passing.

The initial state  $\mathbf{h}_*^{(0)}$  is learned. The same message-passing layers update the virtual node and the physical nodes. After  $K$  layers,  $\mathbf{h}_*^{(K)}$  replaces the usual graph-readout vector before the output projection, so the embedding size does not change. The added edges are structural. The graph receives one such edge per visible controlled busbar, or one per controlled substation when summary nodes are used.

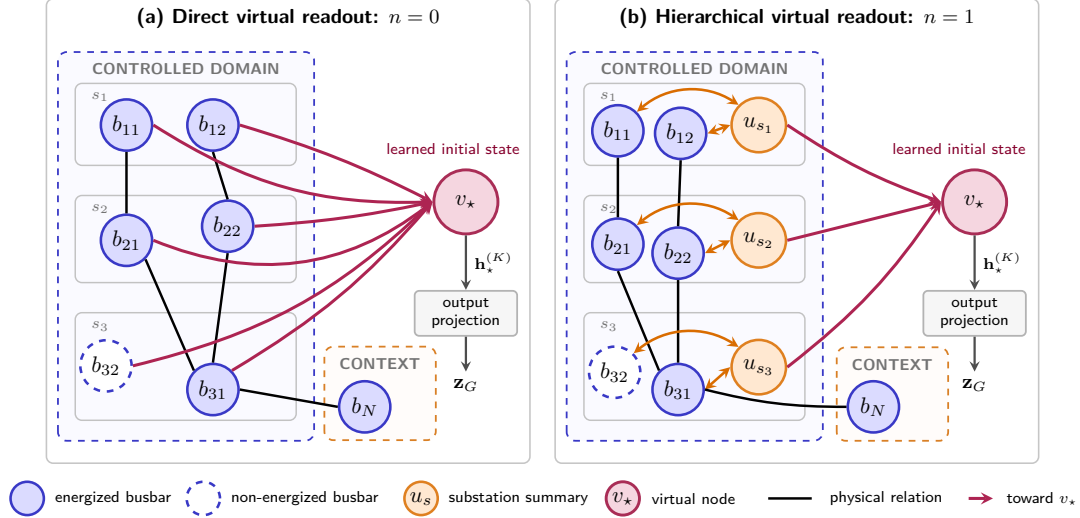


Figure 5.7: Virtual-node readout in the reported one-way setting. Without substation nodes, every controlled busbar—including non-energized  $b_{32}$ —sends directly to  $v_*$ . With augmentation  $n$ , the same busbars communicate through  $u_s$ , and only the substation nodes send to  $v_*$ . The final state  $\mathbf{h}_*^{(K)}$  replaces the selected-node readout.

## 5.4 Candidate-Action Scoring from the Disaggregated Line-Node Graph

The default action head has one output for every discrete action. Each output is tied to an index, so the network must learn separately what each index means. It does not know which physical objects the action changes. The alternative head in this section scores an action from the graph nodes that it touches. This option is available only for the disaggregated line-node graph of Section 5.1.3. The PPO loss, categorical policy, sampling procedure, and deterministic evaluation remain unchanged.

### 5.4.1 From action indices to candidate scoring

Let  $\mathbf{z}_a$  be the  $d_z$ -dimensional graph embedding of agent  $a$ . It is obtained from the local node states by the graph readout of Section 5.3, followed by the output projection in Equation B.7. The default action head receives only this single vector.

With the default head, the logits are produced directly from  $\mathbf{z}_a$ :

$$\ell_a = \text{MLP}_a(\mathbf{z}_a), \quad \ell_a \in \mathbb{R}^{|\mathcal{A}_a|}. \quad (5.3)$$

The final layer has one row for each action index. The model must therefore learn each action separately, and the head becomes wider when the action space  $|\mathcal{A}_a|$  grows.

Candidate scoring replaces these independent output rows with one scalar-valued network shared by all non-idle actions. The idle action can use the same scorer or a separate one:

$$\ell_{a,j} = \text{score} \left( \underbrace{\mathbf{z}_a}_{\text{whole graph}}, \underbrace{\mathbf{c}_j}_{\text{nodes action } j \text{ touches}}, \underbrace{\phi_j}_{\text{optional action descriptor}} \right), \quad j \geq 1. \quad (5.4)$$

The same parameters now score every non-idle action, so the number of learned scorer parameters no longer grows with  $|\mathcal{A}_a|$ . Two actions are compared through the graph nodes they affect instead of through unrelated output weights. A candidate that changes a heavily loaded line can therefore be scored using that line’s current embedding.

The candidate head has three independent choices. First, the touched nodes must be combined into an action context  $\mathbf{c}_j$ . Second, the static action descriptor  $\phi_j$  may be appended. Third, the idle action may share the scorer or use its own head. Crossing the two parameter-free candidate-pooling rules with the last two choices gives eight variants. The complete path to one action score is shown in Figure 5.8.

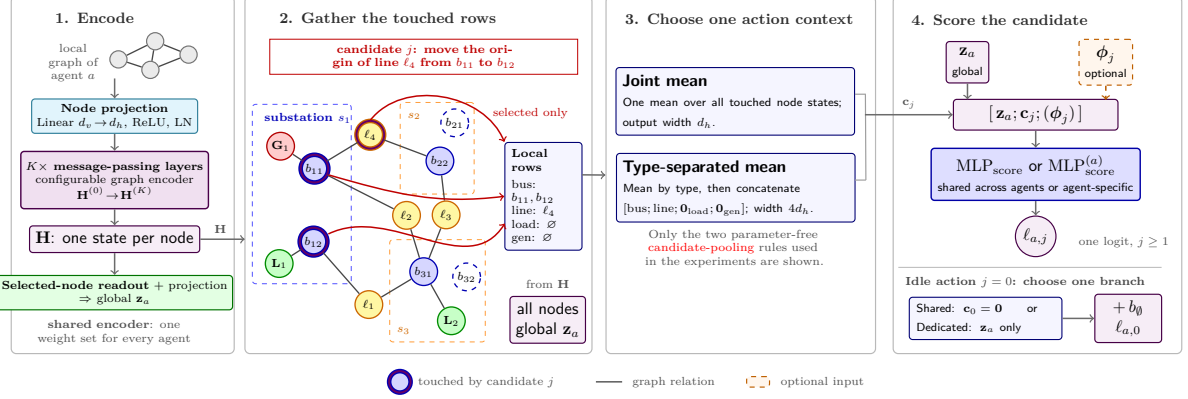


Figure 5.8: Candidate-action scoring for an endpoint-move candidate. Stage 1 is the shared GNN encoder, given in full in Figure 5.5: it maps the local graph to node states  $\mathbf{H}$ . The same stack gives the global embedding  $\mathbf{z}_a$  by selected-node readout and projection, so the two inputs to the scorer come from one set of weights rather than two. In stage 2 the violet outlines mark the only rows gathered from  $\mathbf{H}$  for the local context: both busbars of the modified substation and the touched line node. One candidate-pooling construction then produces  $\mathbf{c}_j$ . The static descriptor  $\phi_j$  may be appended before the shared non-idle scorer. The idle action uses either zero local context in that scorer or a dedicated scorer.

### 5.4.2 What each action touches

The scorer first needs to know which graph nodes belong to each action. I build one lookup table per agent, with one row for every action. The table is created once with the environment and is not recomputed during training.

A Grid2Op topology action identifies the line endpoints, generators, loads, and substations that it changes. These objects are mapped once to their graph rows. All busbars of a modified substation are included so that the destination context is represented, while a boundary action never adds private nodes from another region. The model may also receive an optional static descriptor of the operation. The row-mapping example, padding and mask rules, and complete descriptor are reported in Appendix B.5.

### 5.4.3 Candidate pooling of the touched nodes

The next step combines the touched rows into one action context  $\mathbf{c}_j$ . The global vector  $\mathbf{z}_a$  no longer contains separate node states, so the encoder also returns the full node matrix  $\mathbf{H}$ :

$$(\mathbf{z}_a, \mathbf{H}) = \text{encoder}_{\text{nodes}}(\mathcal{G}_t^{(a)}), \quad \mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times d_h}, \quad (5.5)$$

Row  $v$  of  $\mathbf{H}$  is the final embedding  $\mathbf{h}_v^{(K)}$  of node  $v$ . These embeddings have already been computed by the GNN. Only physical nodes are eligible for an action context. Substation-summary and virtual nodes are excluded because an action changes equipment, not a learned summary node.

Write  $\mathcal{N}_t(j)$  for the rows of type  $t \in \{\text{bus}, \text{line}, \text{load}, \text{gen}\}$  that action  $j$  touches. The masked mean over any such set is

$$\mathbf{c}_j^{(t)} = \frac{1}{\max(1, |\mathcal{N}_t(j)|)} \sum_{v \in \mathcal{N}_t(j)} \mathbf{h}_v^{(K)}, \quad (5.6)$$

The denominator is clamped so that an action touching no node of type  $t$  produces a zero vector instead of a division by zero. Joint-mean pooling applies the equation once to all touched nodes and produces a

vector of width  $d_h$ . Typed-mean pooling computes one mean for each node type and concatenates the four results. Its width is  $4d_h$ , and it keeps the physical roles separate (Table 5.4).

Table 5.4: Implemented candidate-pooling constructions for the action-specific node context.

| Construction                | Eligible rows                   | Aggregation                                        | Width  |
|-----------------------------|---------------------------------|----------------------------------------------------|--------|
| Joint uniform mean          | All nodes touched by the action | One equal-weight mean; node roles are mixed        | $d_h$  |
| Type-separated uniform mean | All nodes touched by the action | One equal-weight mean per type, then concatenation | $4d_h$ |

#### 5.4.4 Scoring and the idle action

One scalar-output network scores every non-idle action from the global graph embedding, the candidate context, and the optional static descriptor. For a non-idle action, the scorer input and logit are

$$\mathbf{x}_{a,j} = \begin{cases} [\mathbf{z}_a; \mathbf{c}_j; \phi_j], & \text{with the descriptor,} \\ [\mathbf{z}_a; \mathbf{c}_j], & \text{without it,} \end{cases} \quad \ell_{a,j} = \text{MLP}_{\text{score}}(\mathbf{x}_{a,j}), \quad j \geq 1. \quad (5.7)$$

The idle action either reuses this scorer with an empty candidate context or uses a dedicated head, which treats do nothing as a separate global decision:

$$\ell_{a,0} = \begin{cases} \text{MLP}_{\text{score}}(\mathbf{x}_{a,0}) + b_{\emptyset}, & \text{shared, with } \mathbf{c}_0 = \mathbf{0}, \\ \text{MLP}_{\emptyset}(\mathbf{z}_a) + b_{\emptyset}, & \text{dedicated.} \end{cases} \quad (5.8)$$

#### 5.4.5 Global-maximum and action-delta extension

The Gmax-delta variant keeps the same shared scorer but changes the information given to it. First, the global vector  $\mathbf{z}_a$  uses a component-wise maximum over energized nodes instead of a mean. Second, every exact modification in action  $j$  is encoded from the object being changed, the mean embedding of its substation busbars, the target busbar, the difference between target and source context, and static operation flags. These tokens are combined into  $\delta_j$  and appended to the scorer input. It uses the current GNN embeddings only; it does not build the post-action graph or run a power-flow simulation. The complete branch diagram is shown in Figure 5.9.

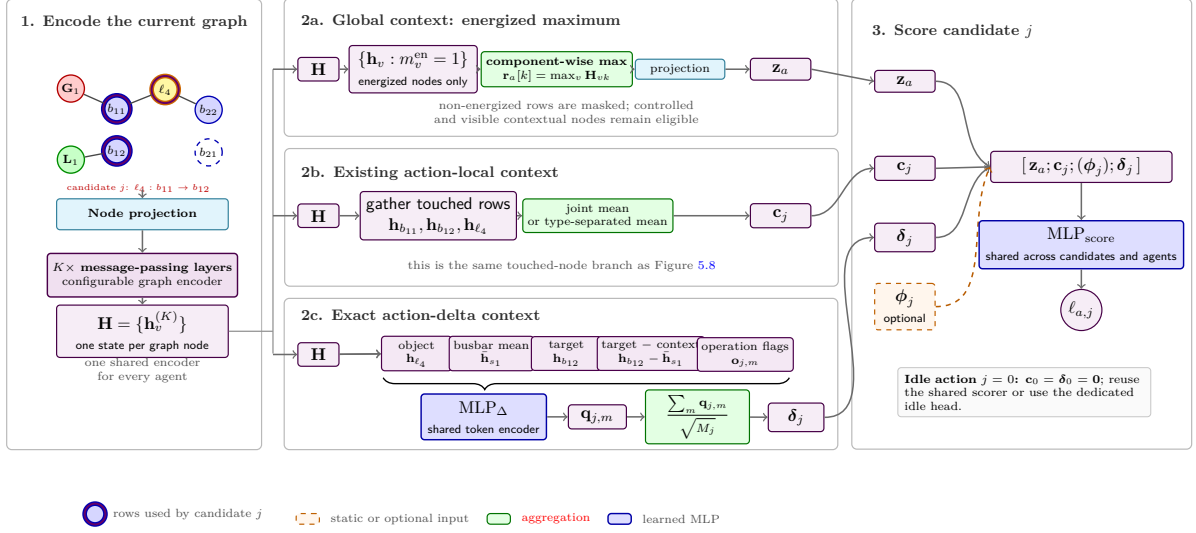


Figure 5.9: Gmax-delta extension of Figure 5.8. The shared graph encoder returns node states  $\mathbf{H}$ . The upper branch builds the global vector by maximum graph readout over energized nodes; the middle branch keeps the touched-node context  $\mathbf{c}_j$ ; and the lower branch encodes each exact object modification into  $\delta_j$ . No post-action graph or power flow is computed.

## 5.5 Cross-Grid Input Transformations

The graph architecture can be reused on grids of different sizes, but the numerical inputs can still change from one grid to another. The same physical quantity may have a different range, and voltage angles depend on the arbitrary reference bus. I therefore test two optional input transformations: fixed physical scaling and a reference-independent angle representation. They change the values given to the encoder, but not the encoder itself.

### 5.5.1 Physical scaling

For every continuous feature  $f$ , the scaled value is  $x'_f = x_f / s_f$ , where  $s_f$  is a fixed reference in the same unit. This keeps the meaning of the feature and, importantly, keeps zero equal to zero. An empty busbar therefore has zero-valued asset features on every grid. Power is divided by the rating of the largest installed generator,  $s_P = \max_g P_g^{\max}$ , while angles are divided by 180 degrees. Cooldown, overflow, and maintenance counters are divided by the limits defined by Grid2Op and listed in Table A.13. These scales are fixed when the environment is created; they are not estimated from training data. The loading ratio `rho`, binary indicators, masks, node types, and relation types are already dimensionless and are left unchanged.

### 5.5.2 Angle representation

An absolute voltage angle needs an arbitrary zero, usually defined by the slack bus. If the same constant is added to every bus angle, the physical state does not change. The difference between the two ends of a line avoids this problem:

$$\Delta\theta_\ell = \theta_\ell^{\text{or}} - \theta_\ell^{\text{ex}}.$$

This line-angle difference does not depend on the reference bus. It is also available for every observed line, whereas an absolute angle is stored only where a generator or load is connected. When this option is enabled, the absolute-angle channels are removed. Chapter 7 shows why this matters: the two grids differ by about 5 to 7° in absolute angle, mostly because they use different slack-bus references. Dividing by a fixed scale cannot remove that offset.

In the reported experiments, this option is used only with the disaggregated line-node graph of Section 5.1.3. Each physical line is a node, so its angle difference is stored directly on that node. A local actor then receives the value for every line it observes, including boundary lines. The value is set to zero when a line is disconnected.

## 5.6 Action-Space Reduction for Large Grids

The graph representation changes what an agent sees. Action-space reduction changes which decisions it can make. This step is necessary on WCCI, where the full discrete topology-action space is much larger than on bus14. Before MAPPO training, actions are ranked offline using simulated outcomes. States are collected during rollouts whenever the grid is stressed:

$$\rho_{\max} > 0.95.$$

For every collected state  $s$ , candidate actions are simulated for one step and compared with doing nothing:

$$\Delta(s, a) = \rho_{\text{after}}(s, a) - \rho_{\text{after}}(s, 0).$$

An action is useful when it reduces the post-action loading by more than  $\epsilon$ , which means  $\Delta(s, a) < -\epsilon$ . Its score is the fraction of evaluated states in which it improves on do nothing:

$$\text{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

The reduced space keeps the  $k$  highest-scoring actions for each agent and always keeps action 0. If an agent has fewer than  $k$  actions, its complete action set is retained.

The value of  $k$  is chosen empirically. I first evaluate each reduced set with a one-step greedy controller. At every risky state, it simulates the available actions and executes the one that improves loading the most. This is not a learned policy; it checks how much useful control remains after the action space has been cut. Section 8.1 reports the resulting action counts and the values of  $k$  used in the transfer experiments.

## Chapter Summary

**Purpose:** Define a graph actor whose representation, graph-level summary, and action interface can be evaluated separately and reused across grids.

- **Graph construction:** The three schemas decide which physical objects receive nodes and what one-hop boundary information is visible to a decentralized actor. The centralized critic still observes the full state.
- **Augmentation and graph readout:** Factors  $e$  and  $n$  change message passing inside controlled substations. Mean, maximum, selected-node, and virtual readouts determine how the encoded graph becomes one vector.
- **Action interface:** The default head learns one output per action index. Candidate scoring instead shares a scorer across actions and conditions each score on the nodes changed by that action.
- **Transfer preparation:** Fixed physical scales and line-angle differences improve input compatibility, while offline action ranking keeps a manageable WCCI action set whose coverage is checked by a greedy simulator.

**Takeaway:** Graph representation, information boundary, readout, action scoring, input transformation, and action-space size are separate design choices. This chapter defines the choices needed to interpret the experiments; their exact tensor layouts and implementation inventories are retained in the appendices.

# Chapter 6

## Graph-Design Screening

This chapter reports six graph-design screens (Screens A–F). All screens are evaluated on seed-0. They ask whether a specific choice in the actor of Chapter 5 changes survival on held-out chronics. Screens A–D evaluate busbar graphs, Screen E evaluates the disaggregated graph, and Screen F evaluates the disaggregated line-node graph with a transferable candidate-action interface. Comparisons are controlled within each screen; results across screens document the progression of the design rather than a single cumulative ablation. Screens A–E form the source-representation branch later tested through frozen-encoder transfer, whereas Screen F forms the complete-policy transfer branch.

### 6.1 Shared Protocol

Table 6.1 separates the common optimization protocol from the boundary and seed settings that differ by campaign. Within a screen, every setting not named as a screened factor is fixed, so each screen is a comparison against its own baseline cell.

Table 6.1: Common optimization settings and campaign-specific boundary and seed settings.

| Component      | Fixed setting                                                                                                                                        |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Environment    | bus14 (difficulty 0), topology actions, decentralized agents, normalized observations/rewards, 80/20 train/test chronic split.                       |
| Actor          | Shared GNN encoder. LayerNorm, node pre-encoder, two 128-unit action-head layers.                                                                    |
| Critic         | Centralized MLP with three 256-unit hidden layers.                                                                                                   |
| Preprocessing  | None.                                                                                                                                                |
| Budget         | 15,000,000 environment steps (72 parallel envs, 576 rollout steps).                                                                                  |
| MAPPO          | 10 epochs, 16 minibatches. LR $10^{-4}$ (linear decay). $\gamma = 0.9$ , $\lambda = 0.95$ , clip 0.2, target KL 0.02, entropy coef 0.01.             |
| Exploration    | No initial do-nothing prior; no action-0 logit bonus.                                                                                                |
| Sparse control | Disabled.                                                                                                                                            |
| Evaluation     | Deterministic, every 82,944 env steps; 20-chronic training-split rollout is used each time, and its mean survival is the checkpoint-selection score. |
| Seeds          | 0 (for Screens B, C, D, E, and F).                                                                                                                   |

#### 6.1.1 Endpoint statistics

During training, checkpoints are evaluated through rollouts on the training split. The checkpoint with the highest mean survival in these training-rollout evaluations is retained. This selected checkpoint is then tested on the complete held-out test set of 201 chronics, and that full-test score is reported in the screening results. Every run reaches the 15M-step budget, and all screens report one seed per cell.

### 6.1.2 Compute

The campaigns were executed on JED (CPU) and IZAR (GPU). Hardware and execution setup were held fixed within each factorial to ensure fair comparisons. Since the screens evaluate exactly one training seed per cell, performance differences are descriptive rather than estimates of between-seed effects.

Training throughput is dominated by stepping the 72 Grid2Op environments rather than the model’s forward and backward passes. For example, in Screen B (Section 6.3), tripling the actor parameter count from 83.6k to 244.9k has a negligible effect on wall-clock time. Parameter count is therefore not the bottleneck when selecting architectures.

**Stage I: source-grid representation.** Screens A–E hold the grid-specific action interface fixed while changing the graph encoder. This isolates what the encoder should represent on **bus14**. Their downstream test is not zero-shot policy reuse: the **bus14** action heads are removed, the source encoder is frozen, and new WCCI heads are learned in Chapter 9.

## 6.2 Screen A: Message-Passing Operator

### Question

Screen A compared GAT and GINE as the message-passing operator. The purpose was simply to select one operator for the remaining experiments.

### Design

Both actors used the busbar graph, two message-passing layers, a hidden width of 128, and the ordinary mean graph readout over visible busbars. The optimization budget and evaluation protocol were otherwise held fixed. Table 6.2 reports the seed-0 full-test score of the best checkpoint over the 201 test chronics.

Table 6.2: Screen A. Operator comparison on the busbar graph with mean graph readout.

| Operator | Graph readout | Mean survival (%) |
|----------|---------------|-------------------|
| GAT      | Mean          | 91.06             |
| GINE     | Mean          | <b>96.66</b>      |

### Reading

GINE outperforms GAT by 5.60 percentage points under this matched setting. Given this superior performance, we select GINE as the message-passing operator for all subsequent screens. This empirical advantage is consistent with theoretical expectations, as GINE is known to be highly expressive for distinguishing different graph topologies (aligning with the Weisfeiler-Lehman graph isomorphism test).

**Role downstream.** GINE becomes the common message-passing operator in both transfer branches. The encoder-only experiment therefore tests whether features learned by this selected operator remain useful after the fixed action heads are replaced.

## 6.3 Screen B: Encoder Depth and Width

### Question

This screen measures the effect of message-passing depth  $K$  and encoder width  $d_h$ . On the busbar graph, depth controls reach: one layer crosses one transmission-line edge, while two layers let each busbar combine information across paths of two physical edges. The graph-level readout then pools all valid node embeddings, so  $K$  need not equal the diameter of the local graph. Width controls how much information

the node and pooled embeddings can retain.

The result applies directly only to the busbar schema; using the same pair for graphs with explicit asset or line nodes remains an assumption.

## Design

A complete  $3 \times 4$  grid at seed 0, yielding 12 runs. This screen is executed entirely on the busbar graph representation. Table 6.3 lists the screened parameters and their levels. The graph output dimension is matched to the hidden dimension, so a cell varies one width rather than two. Every other setting is inherited from the reference configuration of Section 6.1.

Table 6.3: Screen B. Screened parameters and the values tested. All other settings are held at Table 6.1.

| Parameter                  | Setting                     | Values tested   |
|----------------------------|-----------------------------|-----------------|
| Message-passing layers $K$ | <code>gnn_layers</code>     | 1, 2, 3         |
| Encoder width $d_h$        | <code>gnn_hidden_dim</code> | 16, 32, 64, 128 |

## Result

The complete endpoint and marginal tables, the training curves, and the parameter-count diagnostic are reported in Appendix A.2.1.

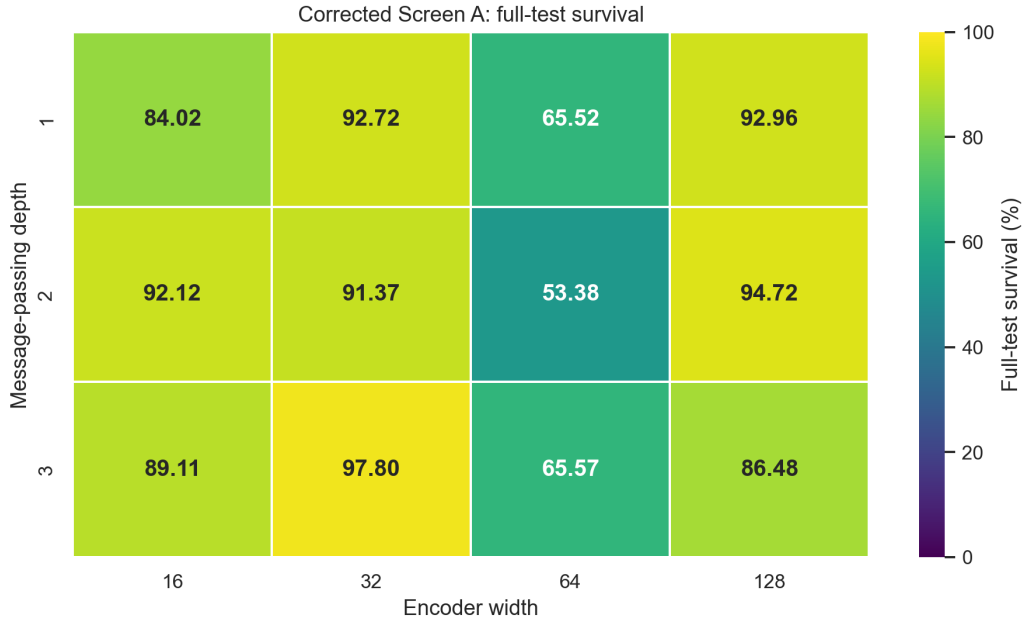


Figure 6.1: Screen B. The same twelve cells as a heatmap of the full-test **mean survival** of each run’s best checkpoint at seed 0.

## Reading

**No clear architectural rule emerges.** While `mp3_h32` achieves the highest single-checkpoint survival (97.80%), neither depth ( $K$ ) nor width ( $d_h$ ) show monotonic trends. Depth ranges overlap heavily, and width performance fluctuates unpredictably. Rather than providing a general scaling law, this screen instead serves to identify a single robust operating point for subsequent experiments.

Each endpoint covers all 201 test chronics, but it still reflects a checkpoint selected from roughly

180 ten-episode evaluations and only one training seed. Table 6.4 therefore complements it with metrics computed from the complete training curves.

Table 6.4: Screen B. Convergence measured from the training curves rather than from a single checkpoint. Steps to 90% is the first evaluation at which the smoothed test survival reaches 90%; the curve mean is the time-average of the smoothed curve over training. The final-window mean averages the raw last 20 periodic evaluations, spanning approximately the final 1.58M environment steps. This is the campaign’s late-stability window: it reduces sensitivity to the last few evaluations while remaining local to the converged regime. Dashes mark architectures that never reached the threshold.

| Architecture | Steps to 90% (M) | Curve mean (%) | Final-window mean (%) |
|--------------|------------------|----------------|-----------------------|
| mp2_h128     | <b>6.22</b>      | <b>71.40</b>   | <b>94.04</b>          |
| mp1_h16      | 6.88             | 70.92          | 84.93                 |
| mp3_h32      | 7.88             | 69.63          | 91.28                 |
| mp2_h16      | 8.79             | 69.16          | 88.21                 |
| mp1_h128     | 11.03            | 68.50          | 91.11                 |
| mp1_h32      | 6.64             | 63.13          | 93.77                 |
| mp3_h128     | —                | 58.58          | 80.20                 |
| mp1_h64      | —                | 51.60          | 63.37                 |
| mp3_h64      | —                | 49.80          | 67.58                 |
| mp3_h16      | —                | 48.82          | 60.00                 |
| mp2_h32      | 13.69            | 46.51          | 91.08                 |
| mp2_h64      | —                | 45.00          | 53.41                 |

**Why carry mp2\_h128 forward.** Physically,  $K = 2$  is a sensible choice because in a small grid like **bus14**, most substations are within two hops of each other. Empirically, **mp2\_h128** dominates all convergence and stability metrics (Table 6.4): it crosses 90% survival first, maintains the highest average performance during training, and is significantly more stable in the late-training window. While its absolute best checkpoint is slightly behind **mp3\_h32** (94.72% vs 97.80%), this gap is well within the noise variance of a single seed. We therefore select **mp2\_h128** as the robust default architecture for the remaining screens.

**Role downstream.** Two GINE layers and width 128 define the common encoder capacity for the four fixed-head representations transferred in Chapter 9 and for the candidate-action models of Screen F.

## 6.4 Screen C: Graph Readout Scope and Reducer

### Question

The local graph may contain nodes that provide message-passing context without being equally useful in the final graph summary. Should the **graph readout include** every visible node, only controlled nodes, only energized nodes, or the intersection of controlled and energized nodes? For each scope, should the valid embeddings be averaged or reduced by a per-channel maximum?

The scope mask is applied only at graph readout. Context nodes remain in the message-passing graph, so this screen changes which embeddings form the policy summary without changing what information can propagate to a controlled node.

### Design

A  $4 \times 2$  factorial crosses four readout scopes—all visible, controlled, energized, and controlled  $\cap$  energized—with mean and max reduction at seed 0. All cells use GINE with  $K = 2$  and  $d_h = 128$  on the busbar graph. Seven cells are new; the all-visible/mean cell reuses **mp2\_h128** from Screen B because it is exactly the same configuration.

## Result

Table 6.5: Screen C. Full-test **mean survival** by readout scope and reducer at seed 0. The all-visible/mean cell is inherited from Screen B.

| Readout scope               | Reducer            |              |
|-----------------------------|--------------------|--------------|
|                             | Mean               | Max          |
| All visible                 | 94.72 <sup>†</sup> | 96.31        |
| Controlled                  | 94.60              | 94.79        |
| Energized                   | 98.93              | <b>99.39</b> |
| Controlled $\cap$ energized | 91.45              | 96.47        |

<sup>†</sup>Screen-B mp2\_h128 reference; not a new run.

Table 6.6: Screen C training-curve diagnostics at seed 0, ranked by curve mean. “To 90%” is the first crossing of 90% by the five-evaluation trailing mean; curve mean is its time-weighted mean over training. Final-20 statistics use the raw final 20 periodic evaluations (approximately 1.58M environment steps).

| Scope/reducer                    | To 90% (M)  | Curve mean (%) | Final-20 mean (%) | Final-20 std (pp) |
|----------------------------------|-------------|----------------|-------------------|-------------------|
| Energized/max                    | <b>3.40</b> | <b>79.26</b>   | 97.79             | 4.12              |
| Energized/mean                   | 8.71        | 73.59          | <b>98.01</b>      | <b>2.90</b>       |
| Controlled $\cap$ energized/mean | 5.97        | 72.97          | 91.18             | 8.59              |
| All visible/mean <sup>†</sup>    | 6.22        | 71.40          | 94.04             | 5.29              |
| Controlled/mean                  | 6.47        | 70.94          | 91.21             | 6.08              |
| Controlled $\cap$ energized/max  | 8.88        | 68.65          | 86.31             | 8.80              |
| All visible/max                  | 4.73        | 68.38          | 88.57             | 8.08              |
| Controlled/max                   | 10.87       | 68.27          | 90.67             | 6.81              |

The corresponding training curves are reported in Appendix A.2.2.

## Reading

**Energized-node readout leads with both reducers.** Its **mean-readout** cell reaches 98.93% and its **max-readout** cell reaches 99.39%, only 0.46 points apart. Relative to **using all visible nodes at graph readout**, the energized-only scope improves survival by 4.21 points under mean and 3.08 points under max.

**Max is directionally consistent across scopes.** For every matched scope, **maximum graph readout exceeds mean graph readout** on the full test. The gain ranges from 0.19 points (controlled nodes) to 5.01 points (controlled-and-energized intersection). The magnitude depends strongly on scope, but the positive direction does not.

**Why carry energized/max forward.** It combines the best scope with the reducer that wins every within-scope full-test comparison. Table 6.6 also identifies it as the strongest overall training curve: it is the earliest to cross 90% (3.40M steps) and has the highest time-weighted curve mean (79.26%). While energized/mean has slightly better late-window variance, **energized\_max** has the strongest full-test endpoint (99.39%) and is the most consistently supported operating point overall. We fix it for **Screens D and E** to prevent readout from becoming an additional confounding factor.

**Role downstream.** Energized/max is retained as the global readout of the fixed-head encoders used in the encoder-transfer branch. It is not the same operation as Screen F’s candidate pooling: the former summarizes the graph state, while the latter summarizes the nodes touched by one candidate action.

## 6.5 Screen D: Structural Augmentations of the Busbar Graph

### Question

Three optional augmentations add computational structure to the busbar graph without adding any physical measurement: same-substation edges ( $e$ ), which let the busbars of one substation exchange messages directly; substation-summary nodes ( $n$ ), which insert one learned hub per substation; and virtual-node readout ( $v$ ), which replaces the **mean graph readout** with a learned graph-summary node. Each is motivated by the same intuition, shorten the path between related busbars, so the question is whether any of them earns its cost, and whether they compose.

### Design

A complete  $2^3$  factorial at seed 0, 8 runs. The graph type is busbar throughout and the baseline cell is the unaugmented **e0n0v0**. It has the same  $K = 2$ , width-128, GINE/**energized\_max** design as the reference from Screen C. Cells are named by their three switches, so **e1n0v1** means same-substation edges on, summary nodes off, virtual-node readout on.

### Result

The detailed result tables and training curves are reported in Appendix A.2.3.

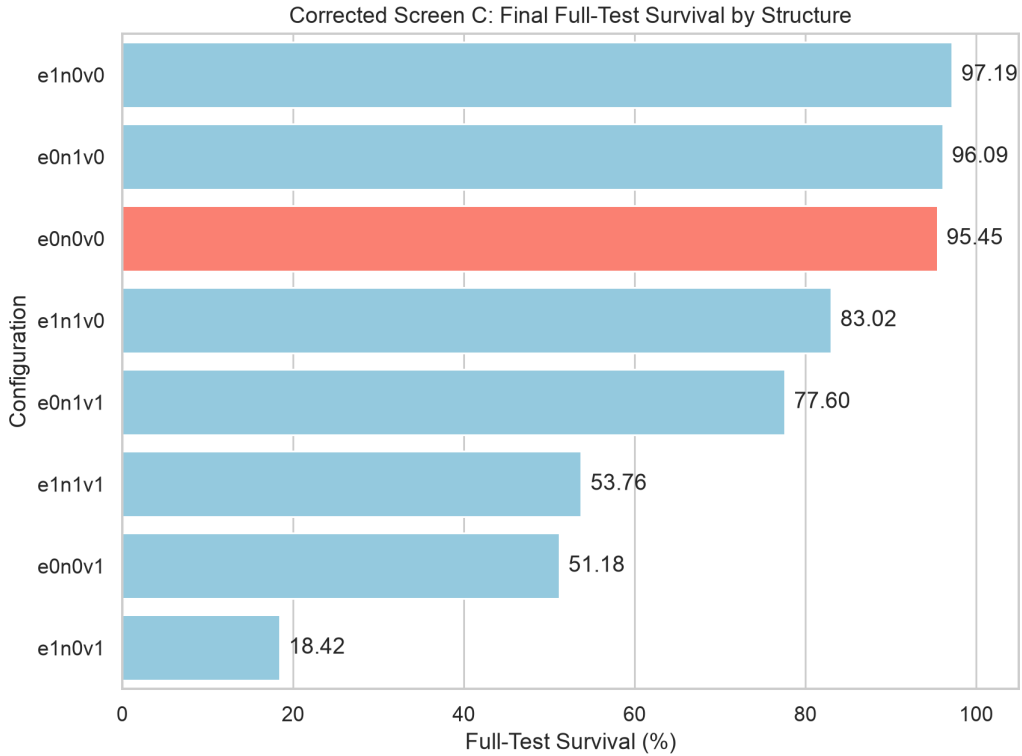


Figure 6.2: Screen D. The factorial results as a bar plot, with the baseline **e0n0v0** highlighted.

### Reading

**Modest gains do not justify increased complexity.** While same-substation edges slightly improve survival (reaching 97.19% vs. the 95.45% baseline<sup>1</sup>), this minor benefit degrades erratically when combined

<sup>1</sup>This **e0n0v0** baseline matches the best Screen C cell mathematically, but differing hardware between campaigns led to trajectory divergence.

with hierarchical factors. Furthermore, virtual-node readouts prove highly destructive (dropping survival to 18–77%), likely because stacking multiple aggregation hubs restricts information flow within our two-layer ( $K = 2$ ) encoder. Consequently, we retain the simple, unaugmented busbar graph for subsequent stages.

**Role downstream.** The unaugmented `e0n0v0` encoder and the modestly stronger `e1n0v0` variant are both transferred with frozen weights. This paired test asks whether the small source-grid benefit of same-substation edges survives a change of grid, rather than assuming that the best `bus14` endpoint must also transfer best.

## 6.6 Screen E: Generator and Load Message Direction

### Question

In the disaggregated graph, each generator and load is a node attached to its busbar. Section 5.1.2 leaves the direction of that attachment open: the asset may send messages to the busbar, receive them from it, or both. The two directions are not symmetric in effect, because the `energized_max` readout pools asset nodes as well as busbars. Does the choice matter, and do the generator and load relations behave independently?

### Design

A complete  $3 \times 3$  factorial over the generator and load directions at seed 0: 9 runs. The graph type is disaggregated throughout; the line and summary directions stay `bidirectional` and are inert for this schema. The encoder matches the reference configuration: GINE, two layers, 128 features, `energized_max` readout, no augmentations, so the only screened factors are the two attachment directions. The baseline cell is bidirectional on both relations. Directions are abbreviated `bi` (bidirectional), `a2b` (asset to busbar), and `b2a` (busbar to asset).

### Result

The training curves and complete endpoint ranking are reported in Appendix A.2.4.

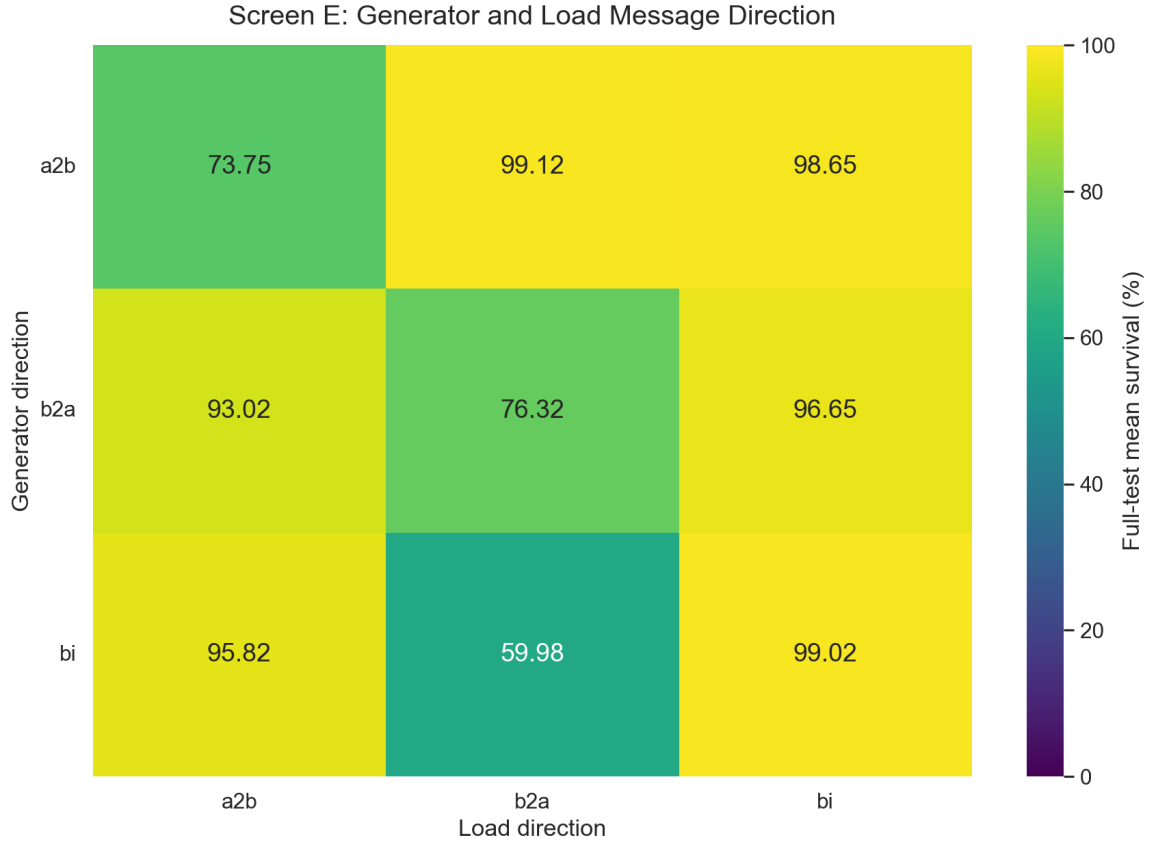


Figure 6.3: Screen E. Full-test **mean survival** of the best checkpoint by generator direction (rows) and load direction (columns). The baseline cell is **bi/bi**.

## Reading

**Asymmetric and bidirectional flows are optimal.** The best configuration uses **a2b** (asset-to-busbar) for generators and **b2a** (busbar-to-asset) for loads (99.12% survival), closely followed by the fully bidirectional configuration (**bi/bi**) at 99.02% survival. Other symmetric or reversed configurations perform significantly worse. The success of the asymmetric **gen\_a2b/load\_b2a** flow makes physical sense: because the **energized\_max graph readout** includes generator and load nodes, pushing generator information inward to the busbars and pulling busbar information outward to the loads allows the network to correctly route production capacity into the core topology while load nodes contextualize their demand. This prevents the dilution of the pooled graph embedding that occurs when edges incorrectly blur these distinct physical roles.

**Role downstream.** Both near-leading configurations are retained for frozen-encoder transfer: **gbi\_lbi** and **ga2b\_lb2a**. Their 0.10-point source-grid gap is too small at one seed to justify declaring a universal direction, so the WCCI experiment tests whether either relation pattern produces more reusable features.

**Stage II: transferable action interface.** Screen F changes the question. Instead of replacing a grid-specific action head on WCCI, it asks whether one scorer can follow a new candidate list and therefore transfer as part of the complete actor.

## 6.7 Screen F: Candidate-Action Scoring

### Question

The candidate-action head of Section 5.4 scores every action from the encoded nodes it modifies rather than from a fixed-width layer over an action index. Three choices in that head are unresolved: how

**candidate pooling reduces the touched nodes**, whether the static action descriptor is appended to the score, and whether the idle action gets its own head. This screen varies all three.

## Design

A  $2^3$  factorial on `bus14`, seed 0, eight runs. **Candidate pooling** is `mean` or `typed_mean`; the action descriptor is appended (`f1`) or withheld (`f0`); the idle action is scored by a dedicated head (`a0h1`) or by the shared scorer (`a0h0`). Every run uses the disaggregated line-node graph without one-hop context, a two-layer GINE encoder of width 128, and the 15M-step budget of Section 6.1. All eight reached 15M steps.

## Result

Table 6.7 reports the full-test **mean survival** of each best checkpoint over the 201 held-out chronics. The corresponding training curves and endpoint ranking are reported in Appendix A.2.5.

Table 6.7: Screen F, full-test **mean survival** of the best checkpoint on 201 held-out chronics. One seed per cell.

| <b>Candidate pooling</b> | Descriptor      | Idle head         | <b>Mean survival (%)</b> | Best checkpoint |
|--------------------------|-----------------|-------------------|--------------------------|-----------------|
| <code>mean</code>        | <code>f0</code> | <code>a0h1</code> | 99.52                    | 14,929,920      |
| <code>typed_mean</code>  | <code>f1</code> | <code>a0h0</code> | 99.50                    | 14,929,920      |
| <code>mean</code>        | <code>f0</code> | <code>a0h0</code> | 99.18                    | 14,929,920      |
| <code>mean</code>        | <code>f1</code> | <code>a0h1</code> | 98.66                    | 14,929,920      |
| <code>mean</code>        | <code>f1</code> | <code>a0h0</code> | 98.32                    | 14,846,976      |
| <code>typed_mean</code>  | <code>f0</code> | <code>a0h1</code> | 98.17                    | 14,764,032      |
| <code>typed_mean</code>  | <code>f0</code> | <code>a0h0</code> | 92.50                    | 12,358,656      |
| <code>typed_mean</code>  | <code>f1</code> | <code>a0h1</code> | 57.84                    | 11,612,160      |

## Reading

**The candidate-action head is a viable actor.** Six of the eight cells reach 98.2% or better and three exceed 99%. No single factor strictly separates the performance. The apparent marginal gaps—such as `mean` at 98.9% against `typed_mean` at 87.0%, and `f0` at 97.3% against `f1` at 88.6%—are **driven by two low-performing interaction cells**; removing the single failure at 57.84% brings the **candidate-pooling** gap to about a point. At one seed, the defensible statement is that the head works and that the three choices do not cleanly order themselves, though withholding the action descriptor (`f0`) and using joint-mean pooling (`mean`) avoid the catastrophic failure cases and emerge as the most robust configuration.

**Role downstream.** Screen F supplies the complete-policy branch of Chapter 10. Its source encoder and shared candidate scorer are reused together; no WCCI-specific fixed-index action layer is required.

## How the screens feed the transfer study

Table 6.8: The downstream role of the six screens. A–E contribute to the frozen-encoder experiment; F contributes to complete-policy transfer.

| Screen | Evidence retained                                         | Downstream use                                                           |
|--------|-----------------------------------------------------------|--------------------------------------------------------------------------|
| A–C    | GINE, $K = 2$ , width 128, energized/max global readout   | Common backbone and readout for the fixed-head encoder-transfer variants |
| D      | e0n0v0 and e1n0v0                                         | Busbar encoders transferred with original and corrected inputs           |
| E      | gbi_1bi and ga2b_1b2a                                     | Disaggregated encoders transferred with original and corrected inputs    |
| F      | Disaggregated line-node graph and shared candidate scorer | Complete-policy zero-shot transfer and target adaptation                 |

### Chapter 6 Summary: Two Design Branches

**Purpose:** To identify effective source-grid graph representations under a controlled fixed action interface, then separately test an action interface designed for complete-policy transfer.

**Content:** Screens A–E study message-passing operator, encoder capacity, global graph readout, structural augmentation, and asset message direction with a fixed-index action head. Their selected busbar and disaggregated encoders form the frozen-encoder experiment of Chapter 9. Screen F instead tests whether a disaggregated line-node graph with a candidate-action interface can retain strong bus14 performance while removing the fixed output dimension that prevents complete-policy transfer.

**Takeaway & Transfer to WCCI:** The screens create two complementary tests. Screens A–E ask whether a learned source representation survives after replacing the incompatible action heads; Screen F asks whether the encoder and action-selection mechanism can cross grids together. The subsequent chapters therefore use:

- e0n0v0, e1n0v0, gbi\_1bi, and ga2b\_1b2a for encoder-only transfer with new WCCI heads;
- the candidate-action scoring (cas\_h1) runs for complete-policy transfer.

# Chapter 7

## What the Shared Encoder Reads

One encoder serves every agent, and the same encoder is what a transfer study moves between grids. Its inputs are raw physical quantities, so before either form of sharing can be interpreted the inputs themselves have to be measured. This chapter establishes what the encoder consumes, how far those distributions differ between two grids, which of the differences a network absorbs and which it does not, and what corrections follow. Without those measurements, a deficit under transfer could not be told apart from a representation reading miscalibrated inputs. [Chapter 8 defines the target controls](#), [Chapter 9 tests frozen fixed-head encoders with original and corrected preprocessing](#), and [Chapter 10 tests complete-policy transfer](#).

The actor and the critic do not face the same problem. The critic consumes the normalized flat observation and is grid-specific in any case; the encoder reads raw graph features, with physical scaling and running standardization initially disabled, so a transferred `bus14` encoder meets WCCI's raw physical quantities directly. Only that second distribution is at issue here.

### 7.1 Measurement protocol

Both environments were rolled out under a do-nothing policy for 6,000 environment steps, with a cap of 300 steps per chronic so the sample spreads over roughly twenty scenarios per grid rather than concentrating in one. At each step the per-agent local graphs were rebuilt and every visible node recorded, pooling agents together because a single encoder is shared across them. This yields 426,000 node observations for `bus14` and 1,236,000 for WCCI.

Only nodes are measured, because on this schema only nodes carry measurements: the nine edge channels are relation indicators, exactly one of which is set on any active edge, with no continuous content to compare. The transfer runs also set `gnn_include_neighbors = false`, so every node in a local graph is controlled and `domain_mask` is constant at one throughout.

Four statistics summarize each feature. Writing  $F_A$  and  $F_B$  for the two empirical distributions with means  $\mu_A, \mu_B$  and standard deviations  $\sigma_A, \sigma_B$ , and  $\sigma_p = \sqrt{(\sigma_A^2 + \sigma_B^2)/2}$  for their pooled spread:

$$\text{offset} = \frac{\mu_B - \mu_A}{\sigma_p}, \quad \text{scale ratio} = \frac{\sigma_B}{\sigma_A}, \quad \text{KS} = \sup_x |F_A(x) - F_B(x)|. \quad (7.1)$$

**Conventions.**  $A$  is always `bus14` and  $B$  always WCCI, so an offset is positive when WCCI sits higher and a scale ratio above one means WCCI is the wider of the two.  $F_A$  and  $F_B$  are the empirical CDFs built from the samples of Section 7.1, and the Kolmogorov–Smirnov statistic is the largest gap between them: it lies in  $[0, 1]$  and is drawn in the last panel of Figure 7.1.

The pairing is deliberate: the offset is blind to a change in spread and the scale ratio is blind to a displacement, so a channel is only unproblematic when both sit near their neutral values, and KS summarises whatever disagreement remains. Section 7.4 shows why the two failures need separating.

## 7.2 What the encoder reads

The measurements are taken on the disaggregated line-node graph, the schema the transfer study uses. Its twelve node channels are specified in Table B.3: four type indicators, the shared `p` and `theta` columns, four line-state columns, the substation cooldown, and `domain_mask`.

Every node type shares one feature vector, so a channel is filled only on the type that owns it and is zero-filled elsewhere. That makes the sparsity structural and severe: `p` and `theta` carry a value only on generator and load nodes, which are 24.0 % of rows on `bus14` and 28.7 % on WCCI, while `rho` and the other line-state channels are filled only on transmission-line nodes, 36.6 % and 36.4 % of rows. A statistic taken over all rows therefore mixes a measurement with the zeros of three other node types, which is why every channel below is reported twice.

Three channels are constant or near-constant in this sample rather than sparse: the two cooldown columns, which a do-nothing policy never triggers, and `timestep_overflow`. One channel is dimensionless by construction: `rho` divides each line’s flow by that line’s own thermal limit, while every other physical channel carries the units of the grid it came from.

## 7.3 Results

Table 7.1 reports the shift statistics twice: over all sampled rows, and over only the node type that carries the channel. The two readings differ for the reason given above — a shared column mixes a point mass at zero with the measurement of the type that owns it.

Table 7.1: Distribution shift between the two grids on the disaggregated line-node graph, ordered by KS. Offset is in pooled standard deviations and the scale ratio is WCCI over `bus14`. The right-hand block restricts each measured channel to the node type that owns it: generator and load nodes for `p` and `theta`, transmission-line nodes for the line state. *The type indicators and `domain_mask` own no type, so the restriction does not apply; `constant` marks a channel with no variation in this sample, whose ratio is undefined.*

| Channel                                | Zeros (%)          |      | All sampled rows |       |       | Owning node type |             |              |
|----------------------------------------|--------------------|------|------------------|-------|-------|------------------|-------------|--------------|
|                                        | <code>bus14</code> | WCCI | offset           | ratio | KS    | offset           | ratio       | KS           |
| <code>theta</code>                     | 77.5               | 73.8 | +0.653           | 0.83  | 0.157 | +1.864           | 1.43        | <b>0.678</b> |
| <code>p</code>                         | 77.8               | 76.7 | −0.037           | 2.14  | 0.042 | −0.158           | <b>2.54</b> | 0.201        |
| <code>rho</code>                       | 63.4               | 63.6 | −0.088           | 0.92  | 0.053 | −0.260           | 1.05        | 0.141        |
| <code>line_status</code>               | 63.4               | 63.6 | −0.005           | 1.00  | 0.002 | constant at one  |             |              |
| <code>timestep_overflow</code>         | 100                | 100  | −0.002           | 0.70  | 0.000 | −0.004           | 0.70        | 0.000        |
| <code>time_before_cooldown_line</code> | 100                | 100  | —                | —     | 0.000 | constant         |             |              |
| <code>time_before_cooldown_sub</code>  | 100                | 100  | —                | —     | 0.000 | constant         |             |              |
| <code>is_busbar</code>                 | 60.6               | 65.0 | −0.093           | 0.98  | 0.045 | —                |             |              |
| <code>is_generator</code>              | 91.5               | 89.3 | +0.076           | 1.11  | 0.022 | —                |             |              |
| <code>is_load</code>                   | 84.5               | 82.0 | +0.066           | 1.06  | 0.025 | —                |             |              |
| <code>is_transmission_line</code>      | 63.4               | 63.6 | −0.004           | 1.00  | 0.002 | —                |             |              |
| <code>domain_mask</code>               | 0                  | 0    | —                | —     | 0.000 | constant at one  |             |              |

Read on all rows the table looks harmless: no channel reaches  $KS = 0.2$ . That is the type mixing, not agreement. Restricted to the owning type, `theta` reaches 0.678 with +1.86 standard deviations of displacement, and `p` widens by a factor of 2.54. The four type indicators differ only slightly, which says the two grids have a similar mix of busbars, assets, and lines per local graph rather than saying anything about their physics.

The extremes make the same point concretely. `p` spans  $[0, 97]$  MW on `bus14` and  $[-281, 399]$  MW on WCCI, where 7.8 % of asset rows are negative and `bus14` has none. `theta` spans  $[-11.6, 1.5]$  degrees against  $[-7.1, 16.8]$ : not a wider version of the same distribution but a displaced one.

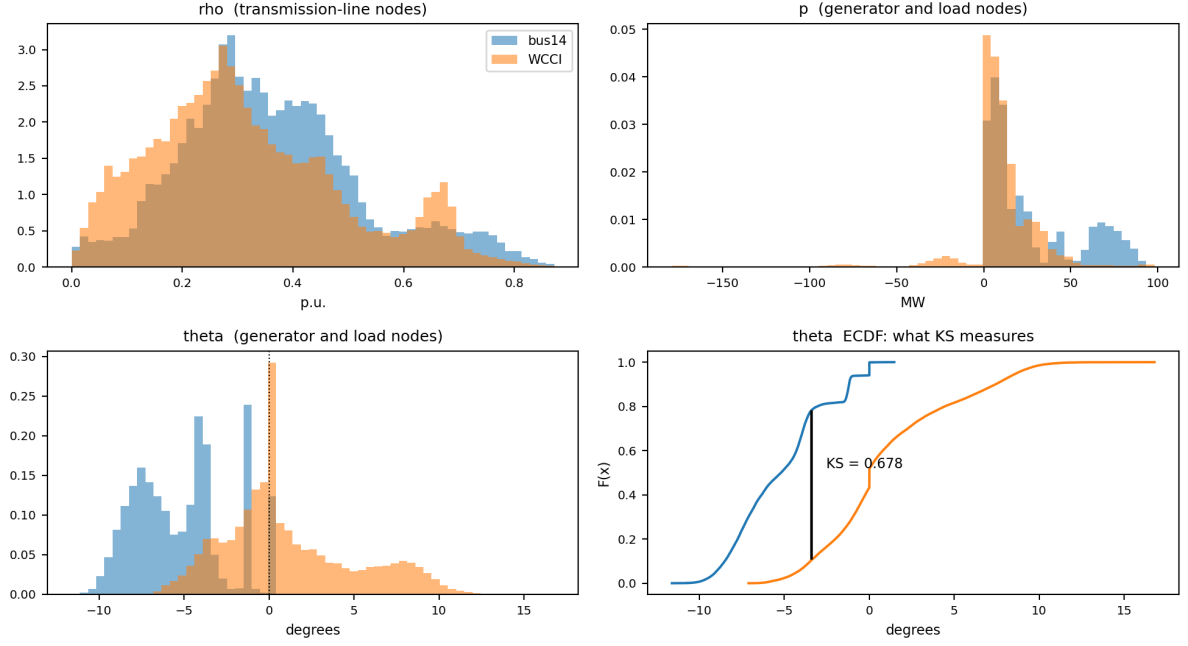


Figure 7.1: The measured channels on the node type that owns them. `rho` lands almost on top of itself, as expected from a quantity already normalized by each grid’s own thermal limits. `p` shares a centre but not a width, and WCCI alone produces negative values. `theta` is displaced rather than rescaled: `bus14` sits almost entirely on the negative side. The last panel reads that displacement as the KS statistic. Full ranges on log density are in Figure A.11.

## 7.4 Two failure modes

The left panel of Figure 7.2 separates the channels along the two axes; the right panel repeats the diagnostic after the corrections and is read in Section 7.5.4. The distinction is not cosmetic, because the architecture responds to the two axes differently.

**Reading the axes.** Each point is one channel, placed by the two statistics of Equation (7.1) over all sampled entries: horizontally the offset magnitude  $|\mu_B - \mu_A|/\sigma_p$ , vertically the scale ratio  $\sigma_B/\sigma_A$  on a log axis so that halving and doubling sit equidistant from 1. A channel needing no correction sits at the bottom left, so distance from that corner is distance from comparability and its direction names which of the two failures is at work.

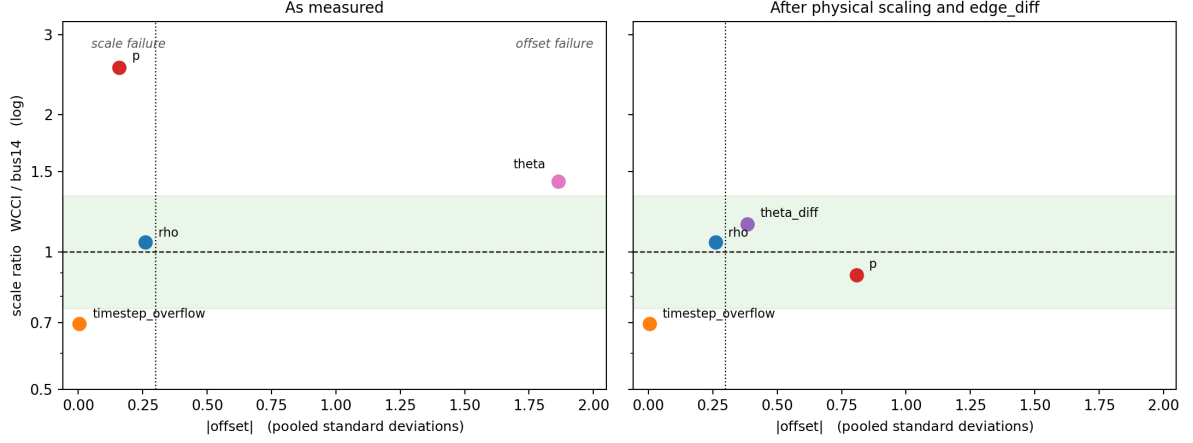


Figure 7.2: Absolute offset against scale ratio, over all sampled entries. The shaded band and the dashed line mark a scale ratio within a third of unity; the dotted vertical line marks an offset of 0.3 pooled standard deviations. *Left*: as measured. Channels outside those bounds fail in one of two distinct ways — the power channels keep their centre and widen, the angles keep their width and move. *Right*: the same diagnostic after the two corrections of Section 7.5, discussed in Section 7.5.4. Points are placed on the owning node type. Every channel now sits inside the scale band and the worst displacement falls from 1.86 to 0.81, but *p* acquires an offset it did not have.

The actor’s node pre-encoder is a linear map followed by ReLU and LayerNorm. LayerNorm effectively absorbs a uniform change in *scale*, as multiplying inputs by a constant leaves the normalized activations largely unchanged. However, it cannot absorb an *offset*: displacing a channel moves the pre-activations across the ReLU threshold, fundamentally altering which units fire. Post-hoc normalization cannot undo this destroyed activation pattern.

This makes the angle, not the power, the critical failure point. The power channel *p* differs in width by a factor of 2.54, which LayerNorm partially self-corrects (though WCCI’s deep negative region remains unseen territory). Conversely, *theta* is displaced by 1.86 standard deviations, an offset the architecture cannot correct.

*rho*, meanwhile, needs no correction at all: scale ratio 1.050, offset  $-0.26$  — the same values the busbar schema gives, since it is the same set of lines carrying the same loadings. The single most decision-relevant channel is already comparable across grids because it was defined in per-unit terms, an argument for expressing features in grid-relative units wherever the physics permits it.

## 7.5 Correcting the inputs

Two preprocessing mechanisms exist in the codebase. Deterministic physical scaling divides each channel by a per-environment physical constant. Running standardization maintains per-environment, per-node-type statistics over *p* and *theta*, excluding *rho*, which the measurements above confirm needs no help. On the diagnosis alone the second looks like the match, being the only one that can move a displaced channel. It is not.

### 7.5.1 Running standardization makes the mismatch worse

Standardizing each environment with its own statistics aligns the aggregate moments exactly: every channel comes out at offset 0.00 and scale ratio 1.00. It also moves both channels it touches an order of magnitude further apart (Table 7.2).

Table 7.2: KS between the two grids before and after per-environment standardization, over all sampled rows. The last column is the number a node that does *not* own the channel presents to the encoder once standardized. Raw, such a row reads 0 on both grids; standardizing sends it to  $-\mu/\sigma$ , a property of the grid.  $\rho$  is shown for contrast and is excluded from the running-statistics feature set.

| Channel               | KS raw | KS standardized | “Not this type” encoding<br>bus14 vs WCCI |
|-----------------------|--------|-----------------|-------------------------------------------|
| <b>p</b>              | 0.042  | <b>0.773</b>    | −0.371 vs −0.145                          |
| <b>theta</b>          | 0.157  | <b>0.673</b>    | +0.481 vs −0.143                          |
| <b>rho</b> (excluded) | 0.053  | 0.634           | −0.658 vs −0.622                          |

The cause is the shared-column structure of Section 7.2. Subtracting a per-grid mean moves each channel’s point mass at zero to a grid-dependent location, and since three quarters of all rows are zeros for **p** and **theta**, the encoder’s single most frequent input stops meaning the same thing on the two grids — a larger mismatch than the displacement being corrected. On **theta** a row that does not own the channel reads +0.481 on bus14 and −0.143 on WCCI, so the same absence lands on opposite sides of zero.

Two further properties rule the mechanism out independently of these numbers. Standardization is scale-invariant, so it erases any scaling applied before it — dividing by a constant and then standardizing reproduces standardizing alone to four decimals — meaning the two mechanisms cannot be combined. And running statistics begin empty and converge during training, so a transferred encoder would spend early target training reading a distribution that is itself still moving, where a deterministic scaling is fixed at construction and identical on both grids from the first step.

### 7.5.2 Physical scaling for the power channels

Physical scaling, defined in Section 5.5, divides the power channels by each environment’s own maximum generator capacity. Two properties answer the failure diagnosed above: being multiplicative it leaves zeros at zero, so the point mass stays aligned on both grids, and being fixed at construction it does not drift the way estimated moments do. The divisor is a single generator’s nameplate rating, a crude proxy for grid scale, so it is worth checking against the alternatives (Table 7.3); it is the closest of the four to the divisor that would equalize the two spreads exactly.

Table 7.3: Candidate power normalizers and the p scale ratio each would produce on asset nodes. A ratio of 1.00 would be exact; the raw ratio is 2.54.

| Divisor                                     | bus14 | WCCI  | Resulting ratio |
|---------------------------------------------|-------|-------|-----------------|
| Exact                                       | —     | —     | 1.00            |
| <b>Maximum <math>P_{\max}</math> (used)</b> | 140   | 400   | <b>0.89</b>     |
| Total $P_{\max}$                            | 540   | 2,450 | 0.56            |
| Generator count                             | 6     | 22    | 0.69            |
| Median $P_{\max}$                           | 85    | 71    | 3.04            |

### 7.5.3 The angle displacement is a gauge artifact

Scaling cannot repair the angle displacement. Absolute voltage angles are referenced to the slack bus, and because the two grids place their slack buses differently, the displacement observed in Figure 7.1 is a gauge artifact rather than a physical difference. No rescaling or recentering can fix a reference-dependent quantity; it must be replaced entirely.

The solution is to use the angle drop along each line (Section 5.5.2). As a difference, it is gauge-invariant. On the disaggregated graph, this feature lives on the transmission-line node (the most populous type), gracefully trading a feature owned by a quarter of the rows for one owned by a third. We therefore replace absolute angles with line-angle differences in both environments.

### 7.5.4 The corrected distribution

Table 7.4 measures both changes together, under the same protocol as Section 7.1, and the right panel of Figure 7.2 places the result back on the offset–scale diagnostic. The angle channel lands in the band occupied by  $\rho$ , and the power spread comes within 11 % of parity.

Table 7.4: Distribution shift with `gnn_angle_representation = edge_diff` and `gnn_physical_scaling = true`, against the uncorrected baseline, on the node type owning each channel. The offset column is included because scaling moves it: dividing each grid by its own constant is not offset-preserving when the two constants stand in a different relation to typical operation.

| Channel           | Uncorrected |       |       | Corrected |             |              |
|-------------------|-------------|-------|-------|-----------|-------------|--------------|
|                   | offset      | ratio | KS    | offset    | ratio       | KS           |
| <b>theta</b>      | +1.864      | 1.43  | 0.678 | replaced  |             |              |
| <b>theta_diff</b> | —           | —     | —     | −0.383    | <b>1.15</b> | <b>0.230</b> |
| <b>p</b>          | −0.158      | 2.54  | 0.201 | −0.808    | <b>0.89</b> | 0.511        |
| <b>rho</b>        | −0.260      | 1.05  | 0.141 | −0.260    | 1.05        | 0.141        |

The angle correction is unambiguous: replacing a reference-dependent quantity by a gauge-invariant one takes the worst channel in the chapter from 0.678 to 0.230, and its scale ratio stays near one throughout.

The power correction is a genuine exchange rather than a repair. Dividing by each grid’s largest generator rating brings the spread from 2.54 to 0.89, but the raw means, nearly equal at  $-0.158$ , are not preserved: **bus14** runs its generators at 39 % of its fleet maximum against WCCI’s 20 %, so a divisor taken from the largest unit lands the two grids at different normalized levels and the displacement grows to  $-0.808$ . KS rises with it, from 0.201 to 0.511. Section 7.4 argued that offsets, not scale errors, are what the architecture cannot absorb, so this moves **p** onto the axis that binds.

The exchange is still favourable: the worst displacement across all channels falls from 1.864 to 0.808, every scale ratio ends inside the band, and no channel remains outside both bounds at once. But it is an exchange, not an elimination — a divisor reflecting typical rather than peak generation would plausibly do better on both axes, and selecting one against the offset as well as the scale ratio is left open. **The two transfer branches test whether the residual displacement matters in practice. Chapter 9 compares frozen source encoders trained with the two input conditions, while Section 10.1 repeats the comparison for the complete candidate-action policy.**

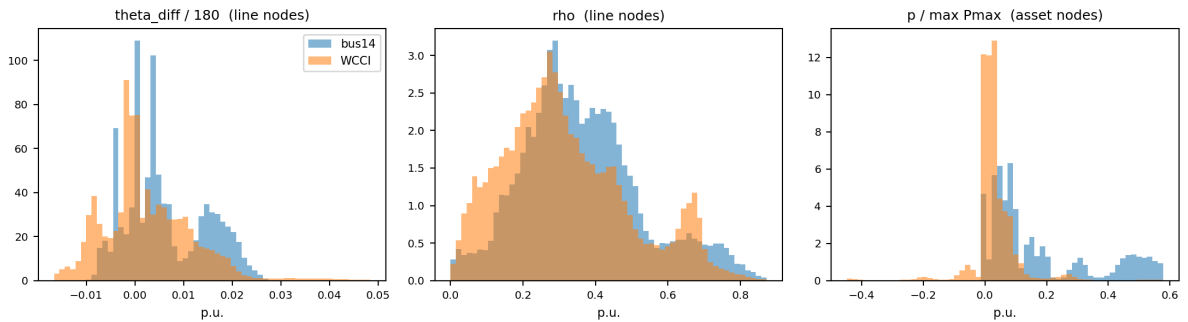


Figure 7.3: Corrected distributions, for comparison with Figure 7.1. The angle drop is centred on both grids, and the scaled power channels overlap over most of their range instead of differing by a factor of 2.5 in width.

## Chapter 7 Summary: What the Shared Encoder Reads

**Purpose:** To measure what the shared encoder actually reads on both grids, so that a deficit under transfer can be told apart from a representation reading miscalibrated inputs.

**Content:** Do-nothing rollouts on both grids (426,000 and 1,236,000 node observations) compared channel by channel on offset, scale ratio and KS. The pre-encoder’s LayerNorm absorbs a change in *scale* but not an *offset*, which moves pre-activations across the ReLU threshold — so the two axes are not equally serious.

### Findings:

- **rho** needs no correction (ratio 1.05, offset  $-0.26$ ): it is already in per-unit terms.
- **theta** is displaced by  $1.864\sigma$ , a gauge artifact of each grid’s slack bus. The gauge-invariant angle drop takes it from  $KS = 0.678$  to  $0.230$ .
- Physical scaling of **p** is an exchange, not a repair: spread  $2.54 \rightarrow 0.89$ , but offset  $-0.158 \rightarrow -0.808$ . The worst displacement overall still falls from  $1.864$  to  $0.808$ .
- Running standardization is rejected: it aligns the moments exactly yet triples KS on **p** ( $0.042 \rightarrow 0.773$ ), because a per-grid mean displaces the point mass at zero that three quarters of all rows carry.
- The action descriptor was normalized per agent, so the same action read  $0.60$  for one agent and  $0.50$  for another — which explains why the descriptor-free **f0** cells were robust in **Screen F**.

**Takeaway:** Deterministic physical scaling and the edge-difference angle are adopted, running standardization is not. **Since the power correction trades a scale error for a displacement, both input conditions are carried into the frozen-encoder experiment of Chapter 9 and the complete-policy experiment of Chapter 10.**

## Chapter 8

# WCCI36: Target Environment and Controls

The experiments now move from `bus14` to the larger WCCI 2020 grid. This chapter defines the target setting and checks which models can learn on it before testing transfer. **The following chapters then separate encoder-only transfer, which learns new WCCI action heads, from complete-policy transfer, which retains the candidate-action interface.**

### 8.1 Target environment

WCCI has more substations, lines and agents than `bus14` (Table 8.1). The agents still control separate areas, but their areas are not equal in size.

Table 8.1: Source and target environments. Both use topology actions, decentralized agents and the same train/test split rule.

|                | <code>bus14</code> | <code>bus36_wcci_nomaint</code> |
|----------------|--------------------|---------------------------------|
| Substations    | 14                 | 36                              |
| Lines          | 20                 | 59                              |
| Agents         | 3                  | 4                               |
| Chronics       | 1,004              | 2,880                           |
| Episode length | up to 8,064 steps  | up to 8,062 steps               |
| Maintenance    | none               | disabled (Section 8.1.1)        |

#### 8.1.1 Removing maintenance

The WCCI environment normally includes scheduled line maintenance, while `bus14` does not. Maintenance is disabled here so that a transferred policy is not tested on a hazard absent from its source environment. This also keeps the observation spaces aligned.

On the full WCCI test split, removing maintenance raises do-nothing mean survival from 27.3% to 57.3% (Table 8.2). The change is large enough that maintenance-on and maintenance-off results should not be compared directly.

Table 8.2: Do-nothing performance on the full WCCI test split. The 576 chronics are the same in both rows.

| Setting              | Mean survival | Median survival | Complete episodes |
|----------------------|---------------|-----------------|-------------------|
| Maintenance enabled  | 27.3%         | 12.1%           | 39/576 (6.8%)     |
| Maintenance disabled | 57.3%         | 100.0%          | 303/576 (52.6%)   |

### 8.1.2 Reduced action spaces

The full decentralized action space contains almost 67,000 actions and is strongly unbalanced across agents. The candidate-action models are therefore evaluated with reduced spaces of  $k \in \{32, 64, 128, 256\}$ . Here,  $k$  is a cap: agents with fewer valid actions keep their complete list.

Table 8.3: Per-agent action counts on `bus36_wcci_nomaint` and one-step greedy **mean survival** on the 24 difficult test chronics. Agent headings give the number of controlled substations.

| Action space | agent_0 (5) | agent_1 (5) | agent_2 (9) | agent_3 (17) | Total  | <b>Greedy mean survival</b> |
|--------------|-------------|-------------|-------------|--------------|--------|-----------------------------|
| Full list    | 77          | 65,642      | 127         | 1,119        | 66,965 | –                           |
| $k = 32$     | 32          | 32          | 32          | 32           | 128    | 33.1%                       |
| $k = 64$     | 64          | 64          | 64          | 64           | 256    | 67.9%                       |
| $k = 128$    | 77          | 128         | 127         | 128          | 460    | 68.2%                       |
| $k = 256$    | 77          | 256         | 127         | 256          | 716    | 94.9%                       |

A one-step greedy policy is used only as a reference for what each reduced space contains. Its difficult-cohort **mean survival** rises from 33.1% at  $k = 32$  to 94.9% at  $k = 256$ . A larger action list therefore gives more useful choices, but it also gives the learned policy a harder selection problem. This trade-off is measured in Chapter 10.

## 8.2 Evaluation protocol

All reported transfer evaluations use the same 50 WCCI chronics. Do nothing **completes 26 of them, giving a 52% completion rate**. These 26 form the *easy* cohort; the remaining 24 form the *difficult* cohort. Do-nothing mean survival is 56.0% overall and 8.34% on the difficult cohort.

Reporting both values separates two goals: preserving cases that already need no action and improving cases where inaction fails. The number of difficult chronics completed is reported as *rescues*. The one-step greedy policy is a strong reference, not an upper bound, because it queries the simulator at every step.

## 8.3 Inputs used for transfer

Chapter 7 found two input changes that make the graph features more comparable across grids. Power is divided by the largest installed generator on each grid, and voltage angle is represented by the difference across each line. Running standardization is not used. The study keeps both the original inputs (NL) and these corrected inputs (NLS) so that the effect of preprocessing can be measured.

## 8.4 Controls trained on WCCI

Figure 8.1 compares three controls trained directly on WCCI: a flat MLP, fixed-list GNN actors from the earlier screens, and the candidate-action scorer. Final checkpoints are used and the local heuristic is disabled. **The fixed-list controls are the target-training references for the frozen-encoder experiment of Chapter 9; the scratch candidate scorers are the references for complete-policy transfer in Chapter 10.**

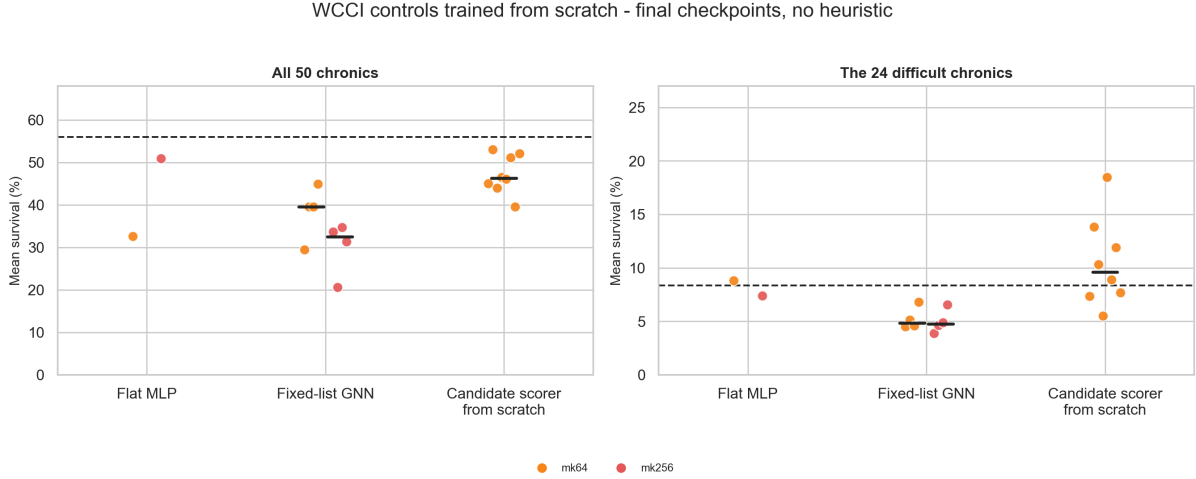


Figure 8.1: WCCI controls trained from scratch. Each point is one model; colours show the reduced action-space size. Short black lines show medians when several models share a setting. The dashed line is do nothing.

The flat MLP and fixed-list GNNs remain below do nothing overall. The fixed-list GNNs also stay below it on the difficult cohort and produce no rescues. The candidate scorer is more promising: at  $k = 64$ , its median is 9.61% on the difficult cohort and its best cell reaches 18.46%. However, its overall score stays below do nothing because it loses some easy chronics.

The candidate scorer is also the only graph actor here whose complete policy can cross grids. Its shared scorer is applied to each candidate, so its output size does not depend on the number of agents or actions. A fixed-list head has one output per action index and cannot be reused unchanged on WCCI.

Exact flat-MLP and fixed-list GNN results are given in Appendix A.4.

This separation avoids comparing two different transfer claims as if they were the same. A fixed-list source policy cannot be deployed unchanged because its output rows name **bus14** actions, but its encoder may still provide reusable features after those rows are replaced. The candidate scorer faces the stricter test because its encoder and action interface are transferred together.

**Purpose:** To define a WCCI target setting comparable with `bus14`, establish an evaluation protocol that separates difficult cases from already-safe ones, and provide scratch-trained controls for both encoder-only and complete-policy transfer.

**Content:** We compare the source and target grids, quantify the effect of disabling WCCI maintenance, reduce the 66,965-action decentralized space to per-agent caps  $k \in \{32, 64, 128, 256\}$ , and define a fixed 50-chronic test with easy and difficult cohorts. We retain both original (NL) and corrected (NLS) inputs, then evaluate final ungated checkpoints of a flat MLP, fixed-list GNNs and the candidate-action scorer trained directly on WCCI.

**Findings:**

- Disabling maintenance raises do-nothing **mean survival** on the complete test split from 27.3% to 57.3%; maintenance-on and maintenance-off results are therefore not directly comparable.
- Larger candidate sets contain much stronger actions: one-step greedy difficult-cohort **mean survival** rises from 33.1% at  $k = 32$  to 94.9% at  $k = 256$ . They simultaneously make action ranking harder for the actor.
- Among the 50 evaluation chronics, do nothing completes 26 easy cases but obtains only 8.34% mean survival on the remaining 24 difficult cases, despite reaching 56.0% overall. Both cohorts must therefore be reported.
- The flat MLP and fixed-list GNNs remain below do nothing overall, and the fixed-list GNNs produce no difficult rescues. The candidate scorer is stronger, with its best scratch-trained cell reaching 18.46% difficult- cohort **mean survival**.

**Takeaway:** The target controls support two downstream comparisons. Chapter 9 asks whether the fixed-list encoders from Screens A–E are better starting representations than random target encoders after learning new WCCI heads. Chapter 10 asks whether Screen F’s grid-size-independent candidate actor can cross grids as a complete policy, and whether target adaptation improves it.

## Chapter 9

# Encoder-Only Transfer Across Grids

Screens A–E identify graph encoders that solve the fixed `bus14` control problem well, but their action heads cannot be moved to WCCI: each output row names one action from the source action list. This chapter isolates the reusable part of those policies. The `bus14` encoder is retained and frozen, its action heads are discarded, and new WCCI heads are learned. Screen F and Chapter 10 address the stricter case in which the encoder and action-selection interface are transferred together.

### 9.1 Question and hypotheses

The main question is whether a graph encoder learned on `bus14` provides more useful WCCI features than a target encoder without source training. A second question is whether the corrected inputs of Chapter 7 make those features more reusable. The study therefore separates three effects:

1. source pretraining versus random encoder features;
2. original inputs (NL) versus corrected inputs (NLS);
3. frozen source features versus an architecture trained end to end on WCCI.

The four transferred representations come directly from Screens D and E: `e0n0v0`, `e1n0v0`, `gbi_lbi`, and `ga2b_1b2a`. They share the decisions retained from Screens A–C: GINE, two message-passing layers, width 128, and energized/max global readout. The two Screen-D variants test whether same-substation edges improve reuse; the two Screen-E variants test whether bidirectional or asymmetric asset relations produce more reusable representations.

### 9.2 Transferred components and controls

The frozen block contains the node and edge input projections, type embeddings when present, GINE layers, normalization layers, and any learned representation projection before the grid-specific action head. The source action heads are removed. Each WCCI agent receives a newly initialized head with the target action dimension. The centralized critic is also newly initialized and remains trainable because it is target-specific.

Table 9.1: Training routes for the encoder-only transfer study.

| Route               | Encoder initialization                | Trainable actor parameters on WCCI | Purpose                       |
|---------------------|---------------------------------------|------------------------------------|-------------------------------|
| Target scratch      | Random                                | Encoder and new heads              | Target-learning reference     |
| Random frozen       | Random                                | New heads only                     | Frozen-feature control        |
| NL frozen transfer  | Existing NL <b>bus14</b> checkpoint   | New heads only                     | Original-input encoder reuse  |
| NLS frozen transfer | Retrained NLS <b>bus14</b> checkpoint | New heads only                     | Corrected-input encoder reuse |

The random-frozen route is the key diagnostic for representation transfer. A source-frozen actor may learn something through its new heads even if the encoder features are arbitrary. The source encoder provides evidence of reuse only when it improves on that control. The fully trainable scratch route gives the target performance available when the same architecture is allowed to adapt end to end.

Table 9.2: Protocol fields to complete before running the matched WCCI experiment. Every route must use the same target values.

| Field                            | Fixed value                                             |
|----------------------------------|---------------------------------------------------------|
| WCCI reduced action space        | <i>[pending: action cap and dataset path]</i>           |
| WCCI training budget             | <i>[pending: environment steps]</i>                     |
| Source and target seeds          | <i>[pending: seed set]</i>                              |
| Head optimizer and learning rate | <i>[pending: matched head-training recipe]</i>          |
| Checkpoint selection             | <i>[pending: selection score and rollout frequency]</i> |
| Final evaluation                 | 50 fixed test chronics; 24 difficult chronics           |
| Maintenance and heuristic        | Maintenance disabled; <i>[pending: gate condition]</i>  |

The NL route uses the preprocessing under which the existing Screens A–E checkpoints were trained. The NLS route retrains the same **bus14** architectures with the corrected physical inputs, then uses exactly that NLS preprocessing on WCCI. Scaling is never switched only at deployment, because a frozen encoder must receive the input convention on which its weights were learned.

## 9.3 Results

### 9.3.1 Learning dynamics

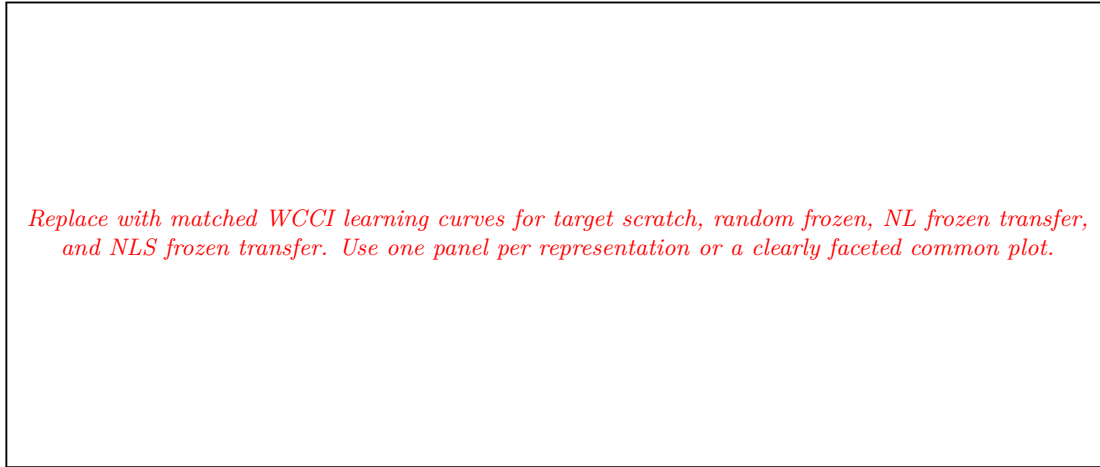


Figure 9.1: Learning dynamics of the encoder-transfer routes. Curves should use the same WCCI training budget and report the checkpoint-selection score at the same evaluation steps.

The learning curves will first test sample efficiency. An NLS source encoder that reaches useful survival earlier than both target scratch and random frozen provides evidence that source training supplies reusable features, even if a fully trainable scratch actor later catches up.

### 9.3.2 Full-test comparison

Table 9.3: Primary endpoint for encoder-only transfer: mean survival on the 24 difficult WCCI chronics. All entries are placeholders until the matched evaluations are complete.

| Representation | Target scratch | Random frozen | NL frozen  | NLS frozen |
|----------------|----------------|---------------|------------|------------|
| e0n0v0         | <i>TBD</i>     | <i>TBD</i>    | <i>TBD</i> | <i>TBD</i> |
| e1n0v0         | <i>TBD</i>     | <i>TBD</i>    | <i>TBD</i> | <i>TBD</i> |
| gbi_lbi        | <i>TBD</i>     | <i>TBD</i>    | <i>TBD</i> | <i>TBD</i> |
| ga2b_lb2a      | <i>TBD</i>     | <i>TBD</i>    | <i>TBD</i> | <i>TBD</i> |

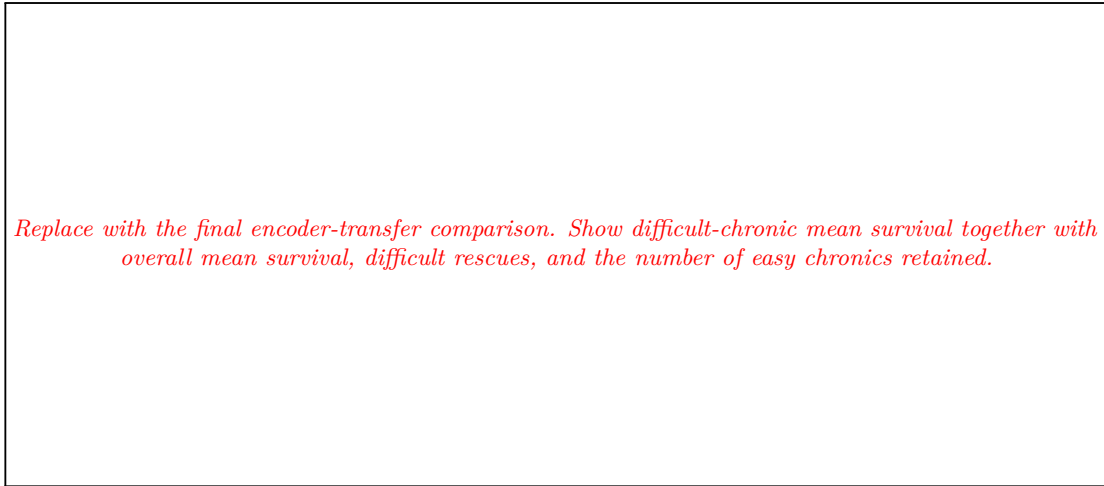


Figure 9.2: Final WCCI comparison of the four source representations and four training routes. Do nothing and the matching scratch controls should remain visible as references.

**Finding.** *[pending: state whether either pretrained frozen encoder beats random frozen, whether NLS improves over NL within each representation, and how close the best frozen encoder comes to target scratch]*

## 9.4 Interpretation rules

The result is interpreted through matched comparisons rather than the largest single endpoint:

- If both NL and NLS frozen encoders beat random frozen, the source GNN has learned reusable features; an NLS advantage then measures the additional effect of corrected preprocessing.
- If only NLS beats random frozen, compatible preprocessing is necessary for the learned representation to cross grids in frozen form.
- If a pretrained frozen encoder beats random frozen but remains below target scratch, the representation transfers partially but still benefits from target adaptation.
- If pretrained frozen and random frozen are indistinguishable while target scratch learns, the architecture can fit WCCI but the source features show no useful frozen transfer under this protocol.

With one training seed, these statements remain descriptive. The strongest claim should be based on the matched difference from random frozen and on consistent learning-curve behavior, not on a small difference between unrelated endpoints.

### Chapter 9 Summary: Encoder-Only Transfer

**Purpose:** To test whether graph representations selected on `bus14` remain useful on WCCI after replacing the incompatible fixed-index action heads.

**Content:** Four source encoders from Screens D and E are transferred with frozen weights. Existing NL checkpoints are compared with matched NLS retraining, a random-frozen diagnostic, and WCCI training from scratch.

**Finding:** *[pending: insert the matched NL/NLS, random-frozen, and scratch results, including difficult-cohort survival and learning speed]*

**Takeaway:** *[pending: state whether the source encoder transfers, whether corrected inputs are required, and whether the remaining limitation lies in representation adaptation or in the grid-specific action interface]*

## Chapter 10

# Complete-Policy Transfer and Target Adaptation

Chapter 9 tested whether a source encoder remains useful after its incompatible action heads are replaced. This chapter asks the stricter question: can the shared candidate-action actor learned on `bus14` control WCCI while retaining both its encoder and its action-selection interface? It first evaluates the complete actor without target training, then studies decision-time changes and target adaptation.

### 10.1 Comparison rules

The transferred object is the complete candidate-action actor, not only its graph encoder. The actor varies along four binary choices: original or corrected inputs, mean or typed-mean candidate pooling, action descriptor off or on, and a shared or dedicated idle-action head. Table 10.1 defines the three training routes.

Table 10.1: Training routes used in the transfer study.

| Route        | Initial actor                   | WCCI training         |
|--------------|---------------------------------|-----------------------|
| Zero-shot    | Shared <code>bus14</code> actor | None                  |
| From scratch | Random weights                  | 15M target steps      |
| Fine-tuned   | Shared <code>bus14</code> actor | About 5M target steps |

Final checkpoints are used for actors trained on WCCI. Zero-shot evaluation uses the source checkpoint selected on `bus14` by mean survival over the 20 training-split chronics in each periodic rollout, with the chronic set changing between evaluations. The optional heuristic blocks an agent’s action when its local maximum line loading is below 0.95. Every comparison uses the same 50 chronics and seed 0.

### 10.2 What controls zero-shot transfer?

Figure 10.1 separates three questions. The top panel gives mean survival over all 50 chronics. The lower-left panel restricts the score to the 24 difficult chronics. The lower-right panel then adjusts that difficult-cohort score to the greedy reference available at the same  $k$ :

$$R_{k,\pi} = 100 \frac{S_{\pi} - S_{\text{DN}}}{S_{\text{greedy},k} - S_{\text{DN}}}. \quad (10.1)$$

Here 0% means do-nothing performance and 100% means the greedy result at the same action count. A negative value is worse than do nothing. This avoids crediting a policy simply because it receives a larger candidate list.



Architecture effects depend on the gate condition

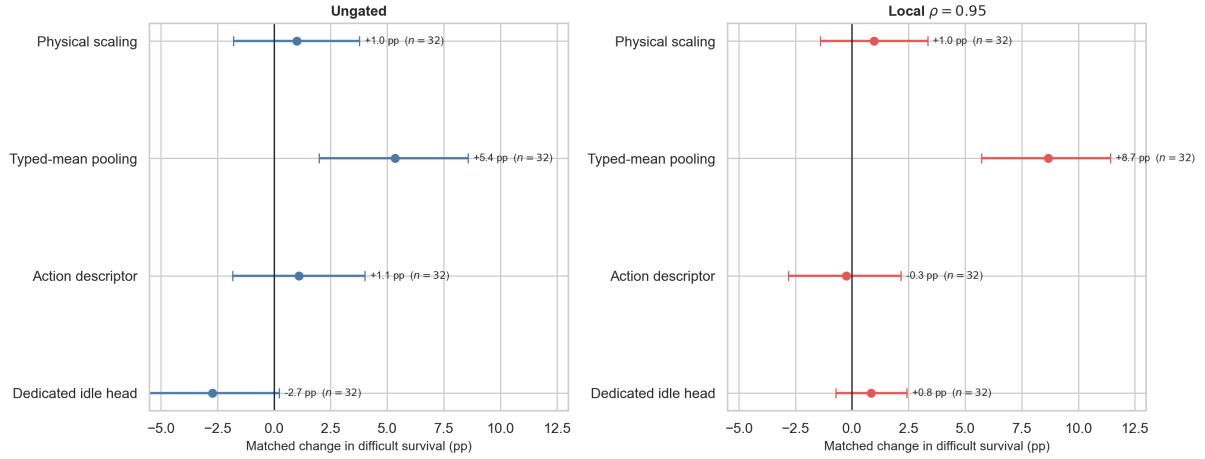


Figure 10.2: Matched architecture effects on the difficult cohort, shown separately for ungated actors and the local- $\rho = 0.95$  heuristic. Positive values improve survival. Error bars are bootstrap intervals over matched architecture cells;  $n$  is the number of available pairs.

These comparisons use one seed. They identify useful directions but do not establish a final architecture ranking.

### 10.3 Policy changes and the local heuristic

Two changes are tested around the zero-shot actor. The first uses the global-maximum readout and exact action-delta encoder of Figure 5.9. The second uses an adaptive intervention budget. Figure 10.3 first shows their absolute performance in one view. Each action-space cluster contains four distributions: the two policy changes, with and without the local- $\rho$  heuristic.

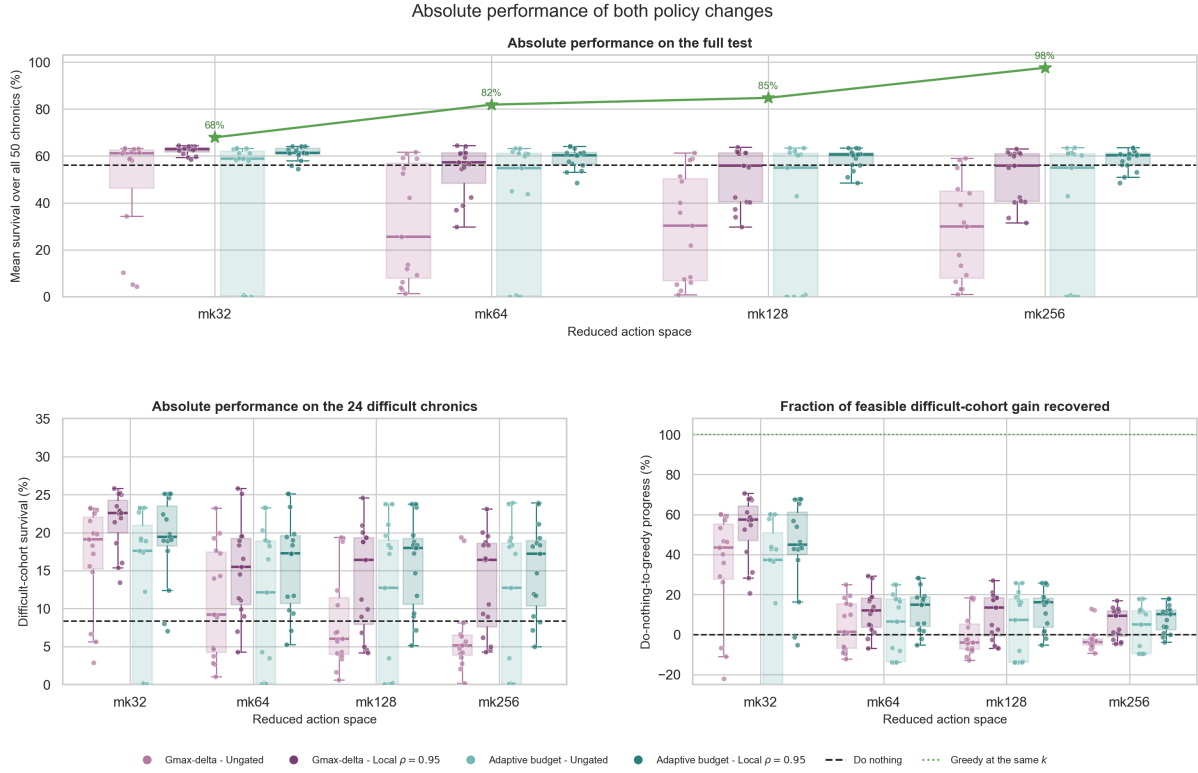


Figure 10.3: Absolute performance of the Gmax-delta and adaptive-budget actors at each action-space size, with four distributions per cluster. Boxes and points summarize the architecture cells for ungated and local- $\rho = 0.95$  evaluation. The dashed line marks do-nothing; the green reference gives the greedy ceiling or, in the relative panel, its full attainable gain.

The absolute view shows whether a positive change is large enough to produce a useful policy. On difficult chronics, ungated Gmax-delta falls from a 19.1% median at  $k = 32$  to 5.1% at  $k = 256$ , whereas the adaptive budget retains 12.7% at  $k = 256$ . With the heuristic, both modified actors remain above the do-nothing reference at every  $k$ . Figure 10.4 then isolates the matched change from the plain actor and the effect of the local- $\rho$  heuristic.

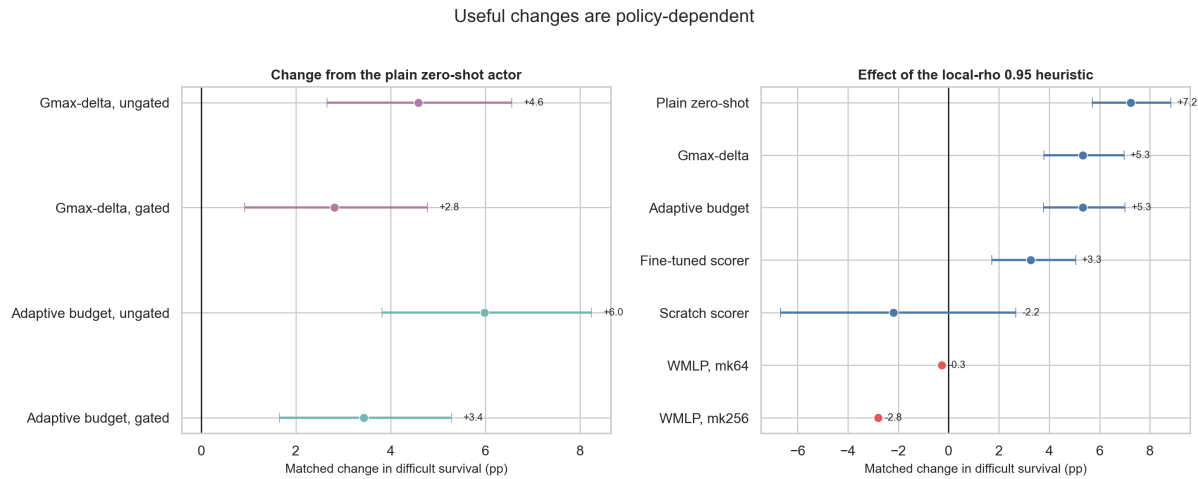


Figure 10.4: Matched changes on the difficult cohort. Positive values are better. The Gmax-delta and adaptive-budget blocks are compared with the same plain zero-shot architectures.

Without the heuristic, Gmax-delta adds 4.6 points and the adaptive budget adds 6.0 points. Across gate conditions, the heuristic adds 7.2 points to the plain zero-shot actor and 5.3 points to both modified

actors. It is not a universal safety layer: it reduces the scratch scorer by 2.2 points and the flat MLP by up to 2.8 points. The gate must therefore be checked with each policy.

The plain block contains all 64 matched architecture- $k$  pairs. Gmax-delta and the adaptive budget each contain 60 pairs because their complete designs use 15 architectures rather than 16.

## 10.4 Does candidate-policy target training help?

Figure 10.5 compares the same eight NLS architectures at  $k = 64$ . The left panel tracks each architecture across zero-shot, scratch and fine-tuned routes. The right panel shows the main trade-off: **mean survival** on the difficult cohort against the number of easy chronics kept.

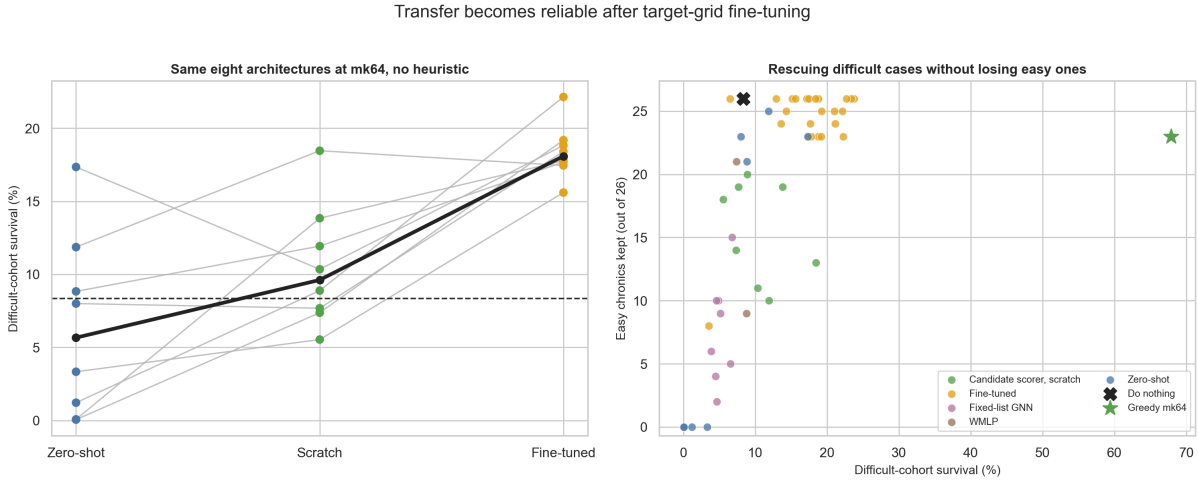


Figure 10.5: Effect of target-grid training. Grey lines join the same architecture in the left panel. The black cross and green star in the right panel are do nothing and the greedy  $k = 64$  reference.

Fine-tuning is the most consistent route. It beats training from scratch on seven of the eight difficult-cohort comparisons, with a mean gain of 7.9 points. It also beats zero-shot transfer in all eight cells, by 12.0 points on average. The **median difficult-cohort mean survival across architectures** rises from 5.7% zero-shot to 9.6% from scratch and 18.1% after fine-tuning.

The right panel adds an important qualification. Some actors improve difficult chronics while losing cases that do nothing already solves. The useful region is therefore the upper-right corner, not simply the highest difficult-cohort score. The target-trained controls remain far below the greedy policy, showing that much more performance is available in the action space.

## 10.5 Limits of the study

The study uses one training seed and 50 evaluation chronics. Bootstrap intervals describe variation across architecture cells, not across independent training runs. Some campaigns also use different target-training budgets. The results support comparisons inside the matched blocks, but small gaps between unrelated blocks should not be overinterpreted. Full matrices and exact control tables are provided in Appendix A.4.

**Purpose:** To determine how well the complete shared candidate-action actor learned on bus14 transfers to WCCI, identify which architectural and decision-time choices improve zero-shot behavior, and measure whether WCCI training benefits more from source initialization than from random weights.

**Content:** Unlike Chapter 9, the source action interface is not replaced. We evaluate 16 NL/NLS architecture cells over  $k \in \{32, 64, 128, 256\}$ , with and without a local- $\rho = 0.95$  gate, using both absolute mean survival and progress from do nothing to the same- $k$  greedy reference. We then test Gmax-delta and adaptive-budget policy changes before comparing the same eight NLS architectures at  $k = 64$  under zero-shot transfer, training from scratch and target fine-tuning.

**Findings:**

- Zero-shot transfer is strongest with the smallest action set. With the gate,  $k = 32$  reaches 62.5% overall and 21.9% on difficult chronics, recovering 54.6% of the available same- $k$  gain; the recovered fraction falls to 2.9% at  $k = 256$ . Ungated medians remain below do nothing.
- Typed-mean candidate pooling is the clearest positive architecture effect, adding 5.4 difficult-cohort mean-survival points ungated and 8.7 gated. Physical scaling adds about one point, while the descriptor is small on average and a dedicated idle head is harmful ungated and nearly neutral gated.
- Gmax-delta and the adaptive budget add 4.6 and 6.0 matched points without the gate. The gate improves 58 of 64 plain cells and adds 7.2 points on average, but it hurts the scratch scorer and flat MLP; it is therefore policy-dependent rather than a universal safety mechanism.
- Fine-tuning is the strongest adaptation route. It beats zero-shot in all eight matched cells by 12.0 points on average and beats scratch in seven cells by 7.9 points. Median difficult-cohort mean survival across architectures progresses from 5.7% zero-shot to 9.6% scratch and 18.1% fine-tuned.

**Takeaway:** Reliable transfer requires controlling the action-selection problem, not merely enlarging the available action set. Small candidate sets and typed-mean candidate pooling are the most robust zero-shot choices; local gating must be validated per policy; and conservative target fine-tuning is more effective than either zero-shot deployment or training the same actor from scratch. These conclusions are based on one seed and 50 chronics, so matched effects are more credible than small gaps between unrelated cells.

# Appendix A

## Additional Experimental Evidence

This appendix collects the additional evidence behind the experimental narrative. It follows the order of the thesis: sparse control, graph-design screening, cross-grid input analysis, and WCCI transfer. Each block states its own evaluation protocol because the number of seeds and chronics differs between campaigns. The main chapters retain the compact comparisons needed for the discussion.

### A.1 Sparse Intervention Control

The joint survival/action-0 comparison is retained in Figure 4.1; the tables below provide the exact aggregate and per-agent values.

#### A.1.1 Additional method details

The intervention-gated actor uses two deterministic decoders. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \quad P(a_i = a > 0) = P(g_i = 1)P(b_i = a \mid g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision and selects the most likely non-idle action only when the gate chooses intervention. The original coupled entropy is

$$H_i = H(g_i) + P(g_i = 1)H(b_i),$$

whereas the separated variant used by the gate-plus-budget diagnostic is

$$H_i = \alpha_{gate}H(g_i) + \alpha_{nonidle}H(b_i).$$

Separating the terms avoids rewarding intervention merely because a larger  $P(g_i = 1)$  exposes more non-idle entropy.

The fixed-penalty experiment crosses

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \quad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \quad \text{seed} \in \{0, 1, 2\},$$

for  $2 \times 4 \times 3 = 24$  runs. The state-dependent safety penalty is disabled in every condition, so this penalty is not rho-weighted.

For AIB, the multiplier settings are:

- `intervention_budget_lr`: 0.02;
- `intervention_budget_init_lambda`: 0.0;
- `intervention_budget_max_lambda`: 10.0.

The five main conditions are flat local-safe AIB with  $d \in \{0.10, 0.20, 0.35\}$ ; gated local-safe AIB with  $d = 0.20$ , hierarchical-greedy evaluation, and separated entropy; and flat plain non-idle AIB with  $d = 0.20$ .

For the teacher-student diagnostic, the teacher is a MAPPO policy evaluated with the local rho heuristic. The student is trained by behavioral cloning to match the teacher’s executed actions and is then evaluated without the heuristic. Two balancing settings address the strong action-0 class imbalance; their exact outcomes are reported in Table A.6.

### A.1.2 Exact bus14 results

The following tables report average full-test episodic survival and action-0 fractions over the three evaluated seeds. The all-agent value is the arithmetic mean of the three per-agent fractions. The do-nothing row is a reference policy that always selects action 0.

Table A.1: Full-test summary for evaluation-time rho heuristics.

| Run type                                   | Avg. episodic survival (%) | Avg. action-0 all agents | Avg. action-0 agent 0 | Avg. action-0 agent 1 | Avg. action-0 agent 2 |
|--------------------------------------------|----------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| Do-nothing policy                          | 20.25                      | 1.000                    | 1.000                 | 1.000                 | 1.000                 |
| Baseline flat policy                       | 93.67                      | 0.434                    | 0.554                 | 0.385                 | 0.363                 |
| Global rho heuristic, $\rho_{safe} = 0.90$ | 98.36                      | 0.936                    | 0.933                 | 0.936                 | 0.940                 |
| Local rho heuristic, $\rho_{safe} = 0.90$  | 97.07                      | 0.968                    | 0.992                 | 0.985                 | 0.928                 |

Table A.2: Full-test summary for intervention-gated actors.

| Run type                               | Avg. episodic survival (%) | Avg. action-0 all agents | Avg. action-0 agent 0 | Avg. action-0 agent 1 | Avg. action-0 agent 2 |
|----------------------------------------|----------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| Do-nothing policy                      | 20.25                      | 1.000                    | 1.000                 | 1.000                 | 1.000                 |
| Baseline flat policy                   | 93.67                      | 0.434                    | 0.554                 | 0.385                 | 0.363                 |
| Intervention gate, final-action MAP    | 89.82                      | 0.802                    | 0.826                 | 0.889                 | 0.690                 |
| Intervention gate, hierarchical greedy | 93.66                      | 0.395                    | 0.066                 | 0.844                 | 0.276                 |

Table A.3: Full-test summary for flat-policy fixed-penalty runs.

| Run type                               | Avg. episodic survival (%) | Avg. action-0 all agents | Avg. action-0 agent 0 | Avg. action-0 agent 1 | Avg. action-0 agent 2 |
|----------------------------------------|----------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| Do-nothing policy                      | 20.25                      | 1.000                    | 1.000                 | 1.000                 | 1.000                 |
| Baseline flat policy                   | 93.67                      | 0.434                    | 0.554                 | 0.385                 | 0.363                 |
| Flat policy, $\lambda_{fixed} = 0.000$ | 98.23                      | 0.494                    | 0.524                 | 0.463                 | 0.496                 |
| Flat policy, $\lambda_{fixed} = 0.001$ | 96.75                      | 0.486                    | 0.642                 | 0.401                 | 0.416                 |
| Flat policy, $\lambda_{fixed} = 0.003$ | 93.97                      | 0.575                    | 0.691                 | 0.346                 | 0.688                 |
| Flat policy, $\lambda_{fixed} = 0.010$ | 97.44                      | 0.905                    | 0.907                 | 0.954                 | 0.853                 |

Table A.4: Full-test summary for gated-policy fixed-penalty runs.

| Run type                                | Avg. episodic survival (%) | Avg. action-0 all agents | Avg. action-0 agent 0 | Avg. action-0 agent 1 | Avg. action-0 agent 2 |
|-----------------------------------------|----------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| Do-nothing policy                       | 20.25                      | 1.000                    | 1.000                 | 1.000                 | 1.000                 |
| Baseline flat policy                    | 93.67                      | 0.434                    | 0.554                 | 0.385                 | 0.363                 |
| Gated policy, $\lambda_{fixed} = 0.000$ | 78.51                      | 0.787                    | 0.747                 | 0.949                 | 0.665                 |
| Gated policy, $\lambda_{fixed} = 0.001$ | 73.82                      | 0.862                    | 0.956                 | 0.832                 | 0.799                 |
| Gated policy, $\lambda_{fixed} = 0.003$ | 94.59                      | 0.887                    | 0.934                 | 0.929                 | 0.799                 |
| Gated policy, $\lambda_{fixed} = 0.010$ | 85.69                      | 0.974                    | 0.990                 | 0.984                 | 0.947                 |

Table A.5: Full-test summary for adaptive intervention budget runs.

| Run type                                                  | Avg. episodic survival (%) | Avg. action-0 all agents | Avg. action-0 agent 0 | Avg. action-0 agent 1 | Avg. action-0 agent 2 |
|-----------------------------------------------------------|----------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| Do-nothing policy                                         | 20.25                      | 1.000                    | 1.000                 | 1.000                 | 1.000                 |
| Baseline flat policy                                      | 93.67                      | 0.434                    | 0.554                 | 0.385                 | 0.363                 |
| Flat local-safe AIB, $d = 0.20$                           | 98.31                      | 0.978                    | 0.980                 | 0.989                 | 0.966                 |
| Flat local-safe AIB, $d = 0.10$                           | 98.39                      | 0.979                    | 0.992                 | 0.995                 | 0.949                 |
| Flat local-safe AIB, $d = 0.35$                           | 96.61                      | 0.934                    | 0.959                 | 0.947                 | 0.896                 |
| Gated local-safe AIB, $d = 0.20$ ,<br>hierarchical greedy | 33.87                      | 0.999                    | 1.000                 | 1.000                 | 0.998                 |
| Flat non-idle AIB, $d = 0.20$                             | 95.08                      | 0.986                    | 0.997                 | 0.992                 | 0.970                 |

Table A.6: Full-test summary for the teacher-student distillation runs.

| Run type                                               | Avg. episodic survival (%) | Avg. action-0 all agents | Avg. action-0 agent 0 | Avg. action-0 agent 1 | Avg. action-0 agent 2 |
|--------------------------------------------------------|----------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| Do-nothing policy                                      | 20.25                      | 1.000                    | 1.000                 | 1.000                 | 1.000                 |
| Teacher: MAPPO with local rho heuristic                | 97.07                      | 0.968                    | 0.992                 | 0.985                 | 0.928                 |
| Student BC, balanced non-idle 0.50, weight 5, aux 0.50 | 92.14                      | 0.939                    | 0.974                 | 0.925                 | 0.919                 |
| Student BC, balanced non-idle 0.20, weight 3, aux 0.25 | 90.76                      | 0.957                    | 0.982                 | 0.948                 | 0.940                 |

### A.1.3 Preliminary maintenance-enabled WCCI pilot

Representative sparse-control mechanisms were retrained from scratch with an MLP actor on the maintenance-enabled WCCI grid. No `bus14` weights were transferred. All runs use the same reduced action space, three seeds, 60M training steps, and learning-rate schedule. The values below are the final deterministic test evaluation, averaged over seeds and ten held-out episodes per seed; they are not full-test estimates.

Table A.7: Preliminary WCCI stress test with scheduled maintenance at the final training-time test evaluation.

| Method                                        | Mean survival (%) | Action-0 fraction |
|-----------------------------------------------|-------------------|-------------------|
| Matched MLP baseline                          | $23.75 \pm 7.70$  | 0.168             |
| Global rho heuristic, $\rho_{safe} = 0.90$    | $28.35 \pm 11.06$ | 0.974             |
| Local rho heuristic, $\rho_{safe} = 0.90$     | $9.46 \pm 3.54$   | 0.995             |
| Intervention gate, final-action MAP           | $25.92 \pm 10.69$ | 0.400             |
| Flat fixed penalty, $\lambda_{fixed} = 0.010$ | $27.00 \pm 12.13$ | 0.489             |
| Flat local-safe AIB, $d = 0.20$               | $21.65 \pm 2.21$  | 0.927             |

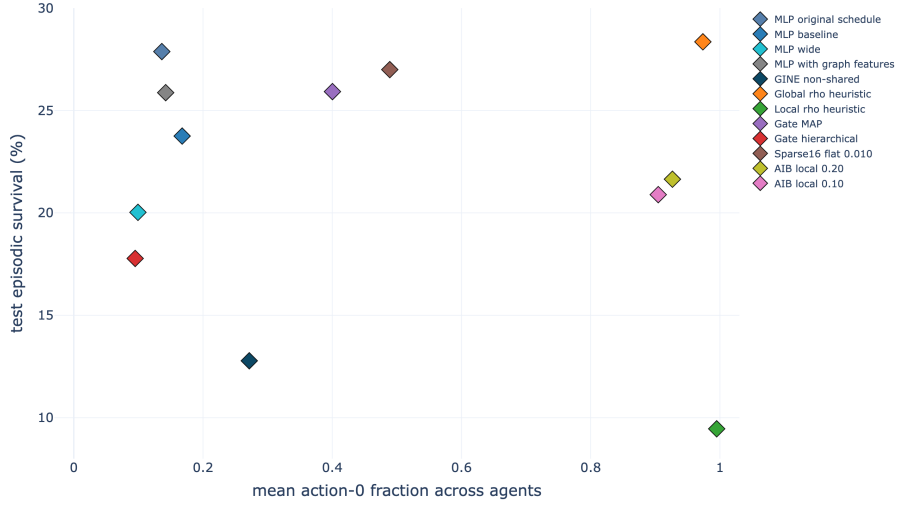


Figure A.1: Action-0/mean-survival trade-off for the early maintenance-enabled WCCI runs. Each point is a seed-aggregated run variant. No method combines high sparsity with strong survival on this larger grid.

The local rho heuristic and AIB are highly sparse but remain weak controllers, which suggests that sparsity transfers more easily than control quality. Since the pilot changes grid size, action space, and maintenance simultaneously, it cannot identify the cause of the weak result.

## A.2 Graph-Design Screening

Screens B–F use the common 15M-step `bus14` protocol of Section 6.1. Each factorial cell uses seed 0; training checkpoints are selected from changing 20-chronic training-split rollouts and then evaluated over the 201 held-out chronics. Multi-seed curves are shown only where a later replication campaign is explicitly identified.

### A.2.1 Screen B: Encoder Depth and Width

Table A.8 gives the complete endpoint grid, while Table A.9 summarizes each factor across the other. Figures A.2 and A.3 provide the corresponding training-curve and parameter-count diagnostics.

Table A.8: Screen B. Full-test survival of the best checkpoint, one seed per cell, by message-passing depth and encoder width. Screen C reuses `mp2_h128` as its all-visible/mean reference.

| Layers $K$ | Encoder width $d_h$ |              |       |       |
|------------|---------------------|--------------|-------|-------|
|            | 16                  | 32           | 64    | 128   |
| 1          | 84.02               | 92.72        | 65.52 | 92.96 |
| 2          | 92.12               | 91.37        | 53.38 | 94.72 |
| 3          | 89.11               | <b>97.80</b> | 65.57 | 86.48 |

Table A.9: Screen B. Marginal effect of each factor, averaged over the other. Four runs per depth, three per width.

| Level                        | Mean (%) | Min (%) | Max (%) |
|------------------------------|----------|---------|---------|
| <i>Message-passing depth</i> |          |         |         |
| $K = 1$                      | 83.80    | 65.52   | 92.96   |
| $K = 2$                      | 82.90    | 53.38   | 94.72   |
| $K = 3$                      | 84.74    | 65.57   | 97.80   |
| <i>Encoder width</i>         |          |         |         |
| $d_h = 16$                   | 88.42    | 84.02   | 92.12   |
| $d_h = 32$                   | 93.97    | 91.37   | 97.80   |
| $d_h = 64$                   | 61.49    | 53.38   | 65.57   |
| $d_h = 128$                  | 91.39    | 86.48   | 94.72   |

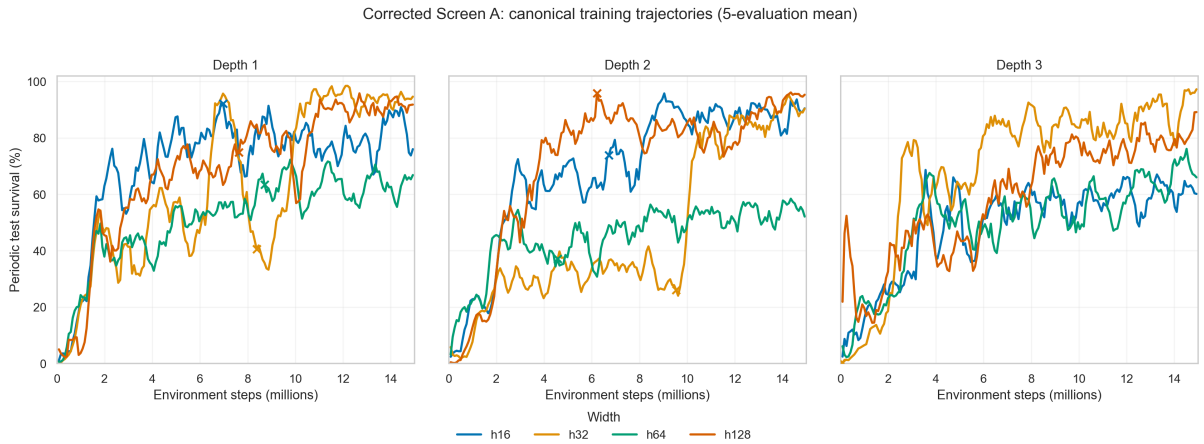


Figure A.2: Screen B. Test episodic survival during training, one panel per message-passing depth, with the four widths compared inside each panel. Restarted runs use the canonical history in which the continuation replaces the overlapping post-checkpoint branch.

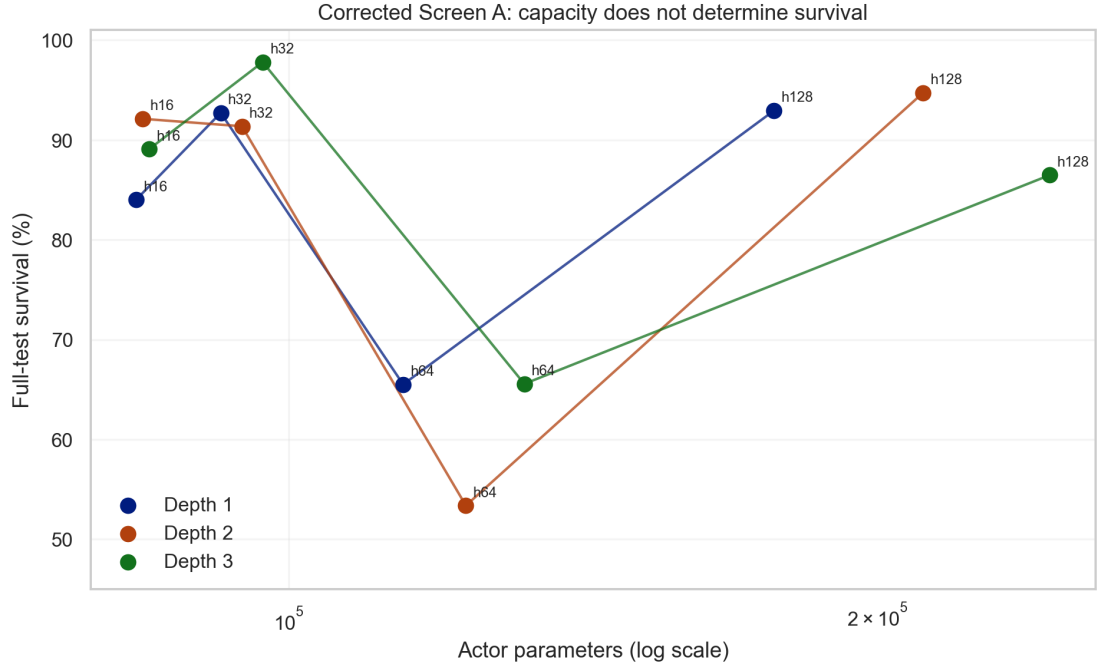


Figure A.3: Screen B. Full-test survival against actor parameter count on a logarithmic axis, joined within each depth. The grid spans 83.6k to 244.9k actor parameters, a factor of three.

## A.2.2 Screen C: Graph Readout Scope and Reducer

Figures A.4 and A.5 complement the endpoint comparison in Table 6.5 with the eight training trajectories and the full-test interaction between readout scope and reducer.

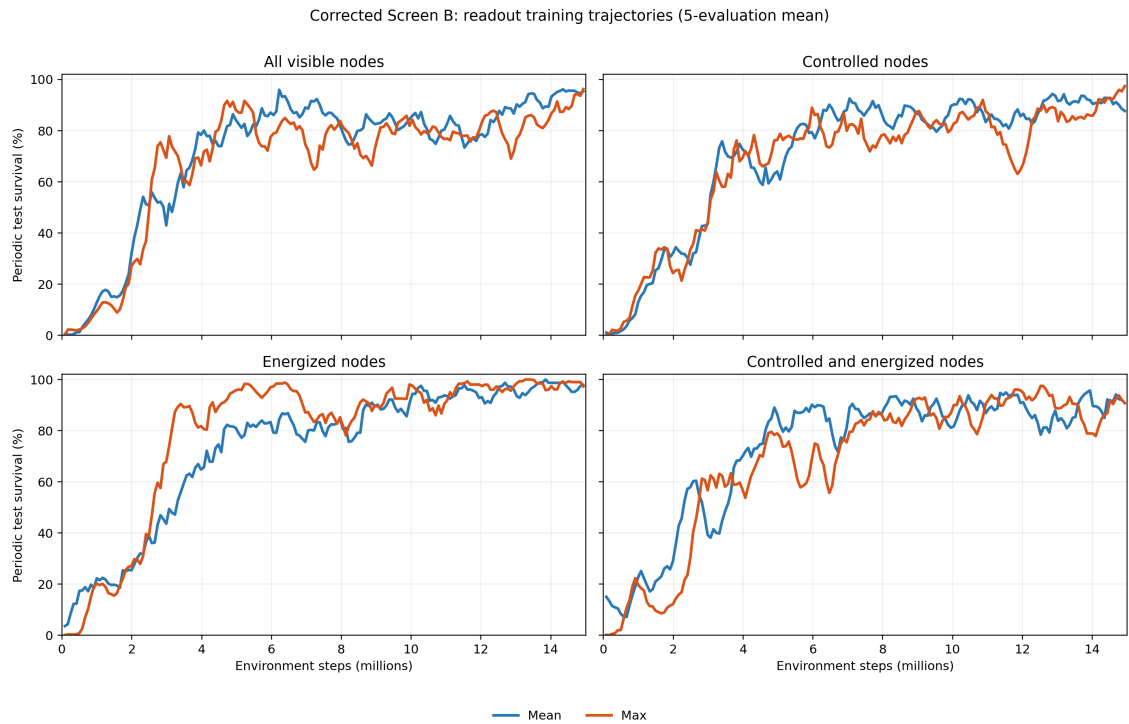


Figure A.4: Screen C. Smoothed training episodic survival curves by readout scope and reducer (seed 0).

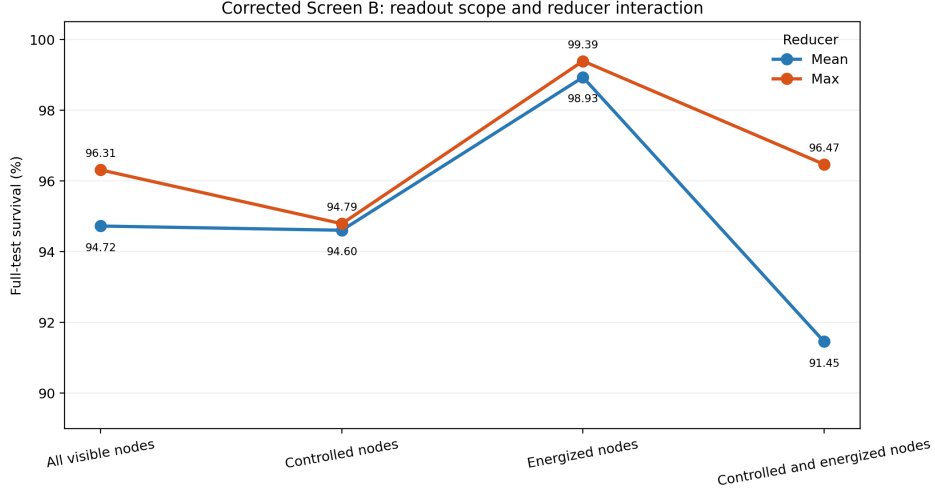


Figure A.5: Screen C. Full-test survival of each best checkpoint. Connecting the two reducers within a scope makes the scope-reducer interaction visible; energized-only readout leads under both reductions.

### A.2.3 Screen D: Structural Augmentations of the Busbar Graph

Tables A.10 and A.11 and Figures A.6 and A.7 complement the factorial cube retained in Section 6.5 with the complete results.

Table A.10: Screen D. Cells ranked by full-test survival, with the difference from the unaugmented `e0n0v0` baseline.

| Cell                | Mean (%) | Std (pp) | Min (%) | Max (%) | $\Delta$ baseline (pp) |
|---------------------|----------|----------|---------|---------|------------------------|
| <code>e0n0v0</code> | 97.31    | 1.22     | 96.55   | 98.72   | 0.00                   |
| <code>e1n0v0</code> | 96.20    | 3.94     | 91.66   | 98.68   | -1.11                  |
| <code>e0n0v1</code> | 95.63    | 6.19     | 88.52   | 99.82   | -1.68                  |
| <code>e0n1v0</code> | 93.51    | 6.39     | 86.72   | 99.40   | -3.80                  |
| <code>e1n1v0</code> | 90.01    | 4.92     | 85.42   | 95.19   | -7.30                  |
| <code>e1n0v1</code> | 89.92    | 9.42     | 80.48   | 99.33   | -7.39                  |
| <code>e0n1v1</code> | 68.91    | 5.77     | 62.59   | 73.91   | -28.40                 |
| <code>e1n1v1</code> | 66.11    | 33.95    | 27.86   | 92.68   | -31.20                 |

Table A.11: Screen D. Main effect of each factor (12 runs per level) and the four single-factor flips of the cube, each one enabling that factor with the other two held fixed. **Spread** is the range of those four flips.

| Factor                      | Off (%) | On (%) | Effect (pp) | Flips (pp)               | Mean flip (pp) | Spread (pp) |
|-----------------------------|---------|--------|-------------|--------------------------|----------------|-------------|
| <i>e</i> (substation edges) | 88.84   | 85.56  | -3.28       | -1.1, -5.7, -3.5, -2.8   | -3.28          | 4.60        |
| <i>v</i> (virtual readout)  | 94.26   | 80.14  | -14.11      | -1.7, -24.6, -6.3, -23.9 | -14.11         | 22.92       |
| <i>n</i> (summary nodes)    | 94.77   | 79.63  | -15.13      | -3.8, -26.7, -6.2, -23.8 | -15.13         | 22.92       |

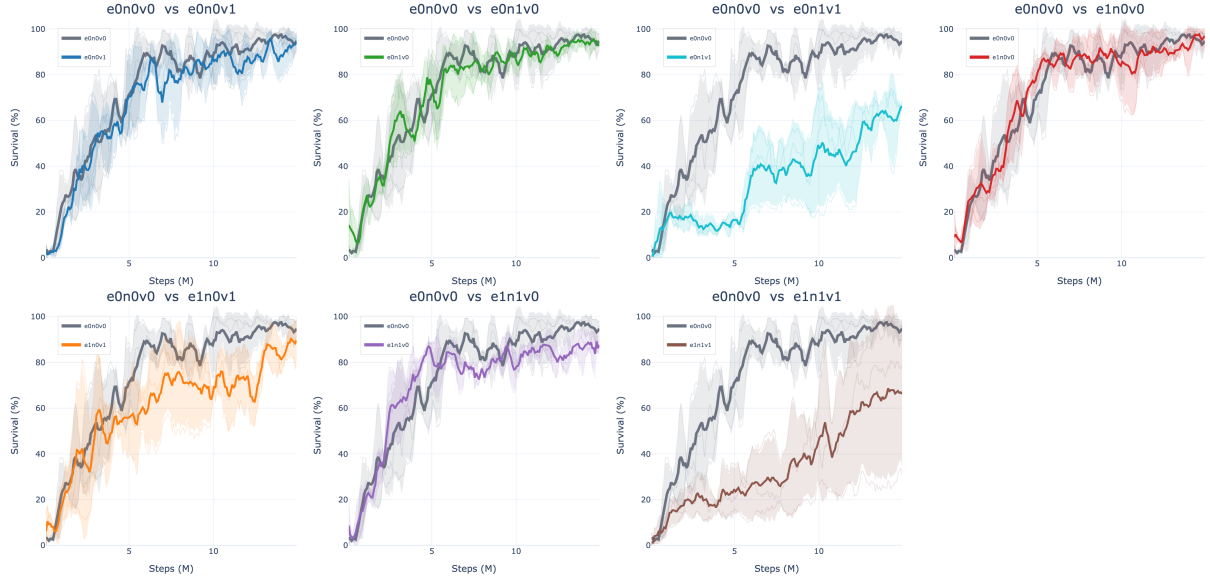


Figure A.6: Screen D. Test episodic survival during training, one panel per augmented cell against the unaugmented **e0n0v0** baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.



Figure A.7: Screen D. The same eight cells as a heatmap of the full-test survival of each run's best checkpoint, with the three individual seed values printed under each cell mean.

## A.2.4 Screen E: Generator and Load Message Direction

Figure A.8 shows the training trajectories behind the endpoint heatmap retained in Section 6.6.

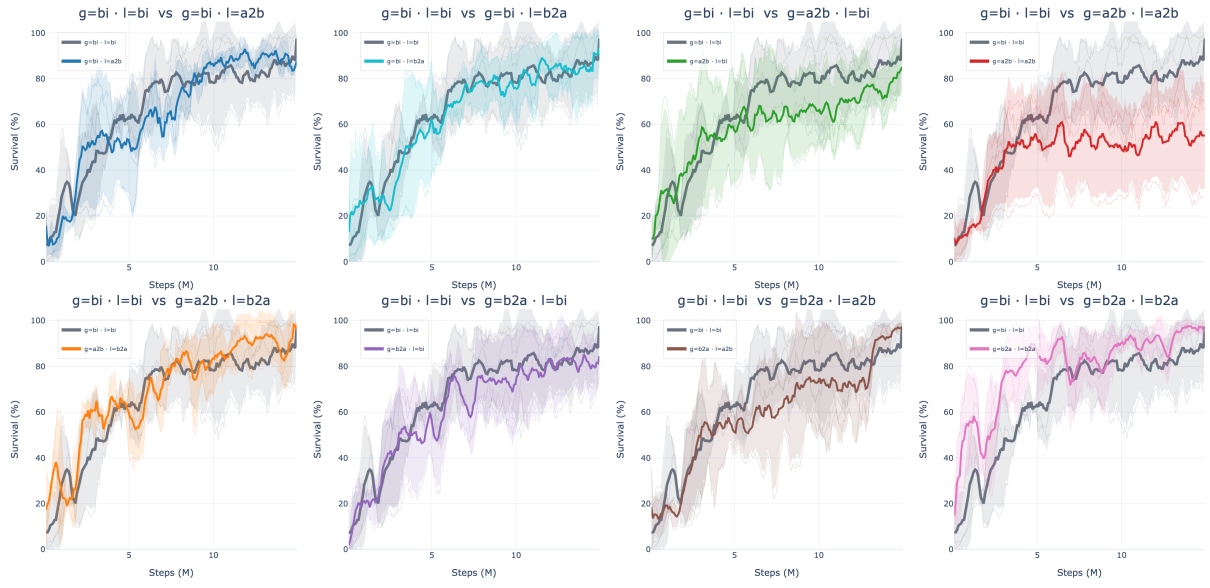


Figure A.8: Screen E. Test episodic survival during training, one panel per direction pair against the bidirectional baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

### A.2.5 Screen F: Candidate-Action Scoring

Figures A.9 and A.10 complement the full-test table retained in Section 6.7 with the training trajectories and endpoint ordering.

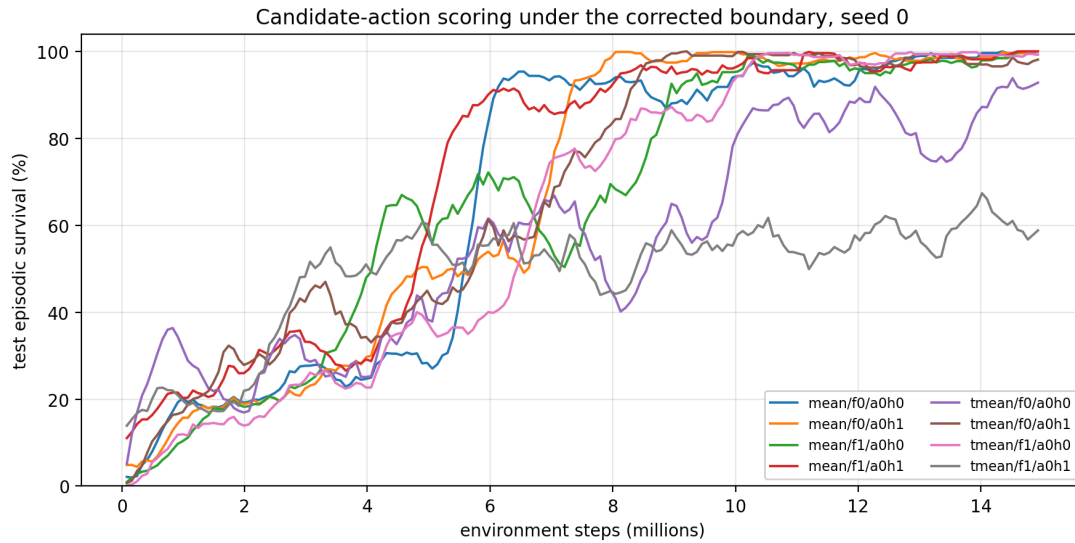


Figure A.9: Screen F. Test episodic survival during training, smoothed over nine evaluations. Six of the eight cells converge to the top of the range and stay there; `typed_mean/f1/a0h1` never does.

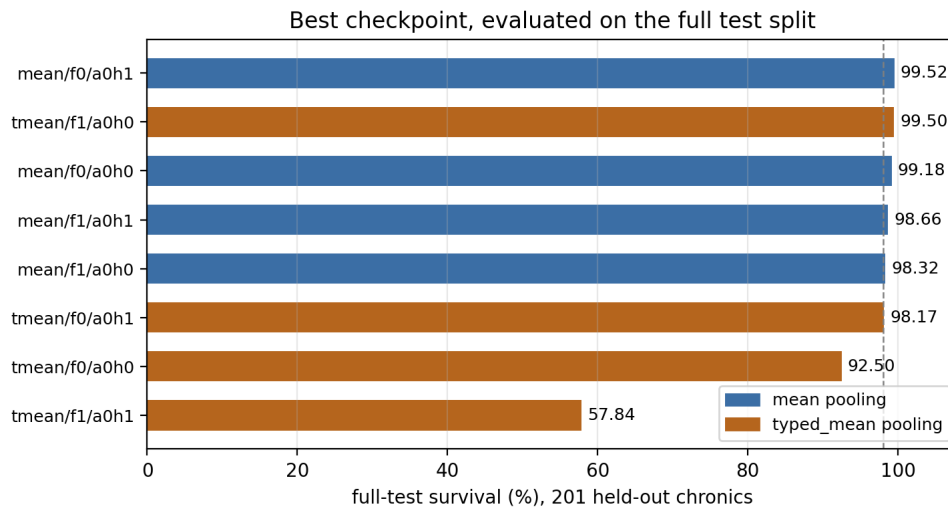


Figure A.10: Screen F. Full-test survival of each best checkpoint. The dashed line marks 98%.

## A.3 Cross-Grid Input Analysis

This block supports Chapter 7. Both grids are measured under the same do-nothing protocol: 6,000 environment steps, at most 300 steps per chronic, and pooled local-graph nodes from the shared encoder. This yields 426,000 **bus14** node observations and 1,236,000 WCCI node observations.

### A.3.1 Encoder Input Distributions

Chapter 7 retains the compact shift table (Table 7.1) and the consolidated distribution panel (Figure 7.1). This section provides the raw moments behind those statistics and the tail behaviour that linear axes compress. All values are measured on the disaggregated line-node graph under the protocol of Section 7.1.

Table A.12: Raw moments behind Table 7.1, with the Wasserstein distance. “Owning” restricts each channel to the node type that carries it.  $W_1/\sigma_p$  tracks the offset almost exactly on the displaced channel, which is why the main table omits it: for distributions that differ mainly by a shift the two statistics carry the same information.

| Channel | Rows   | bus14  |        | WCCI   |        | offset | ratio | $W_1/\sigma_p$ |
|---------|--------|--------|--------|--------|--------|--------|-------|----------------|
|         |        | mean   | sd     | mean   | sd     |        |       |                |
| p       | all    | 6.538  | 17.615 | 5.461  | 37.718 | −0.037 | 2.141 | 0.204          |
|         | owning | 27.306 | 26.998 | 19.067 | 68.613 | −0.158 | 2.541 | 0.485          |
| theta   | all    | −1.236 | 2.570  | 0.305  | 2.127  | +0.653 | 0.827 | 0.653          |
|         | owning | −5.162 | 2.707  | 1.064  | 3.871  | +1.864 | 1.430 | 1.864          |
| rho     | all    | 0.133  | 0.203  | 0.116  | 0.187  | −0.088 | 0.922 | 0.088          |
|         | owning | 0.364  | 0.168  | 0.319  | 0.176  | −0.260 | 1.050 | 0.261          |

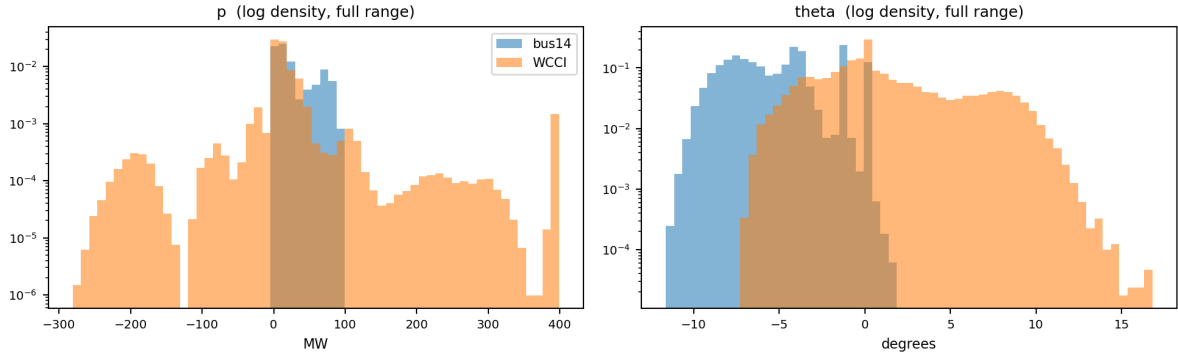


Figure A.11: Full range of the two asset channels on log density. WCCI carries a generator pinned near its 399 MW rating and a load region reaching −281 MW, neither of which occurs anywhere on **bus14**: a transferred encoder has no calibration for either. The angle panel shows the displacement rather than a difference in width.

### A.3.2 Deterministic Feature Scales

Section 5.5 gives the two references that carry physical content, the power base and the angular scale. The complete list of divisors, including the discrete counters, is below.

Table A.13: Deterministic physical scales used for continuous graph features. The power base is the largest installed generator rating on the grid; the remaining discrete channels use the corresponding Grid2Op parameter bound.

| Features                                                                                           | Scale $s_f$                    | Source                              |
|----------------------------------------------------------------------------------------------------|--------------------------------|-------------------------------------|
| <code>gen_p</code> , <code>load_p</code> , <code>p</code>                                          | Power base in MW               | Largest installed generator rating. |
| <code>gen_theta</code> , <code>load_theta</code> , <code>theta</code> ,<br><code>theta_diff</code> | 180 degrees                    | Fixed angular scale.                |
| <code>time_before_cooldown_sub</code>                                                              | Maximum substation<br>cooldown | Grid2Op environment parameter.      |
| <code>time_before_cooldown_line</code>                                                             | Maximum line cooldown          | Grid2Op environment parameter.      |
| <code>timestep_overflow</code>                                                                     | Allowed overflow duration      | Grid2Op environment parameter.      |
| Maintenance time and duration                                                                      | Episode horizon                | Grid2Op chronic horizon.            |

## A.4 WCCI Transfer and Target-Control Results

This section gives the detailed results behind Chapters 8–10. The encoder-only transfer tables will be added after the pending experiments in Chapter 9. Every survival value uses the same 50 WCCI chronics and the easy/difficult split defined in Section 8.2.

### A.4.1 Zero-Shot and Intervention Matrices

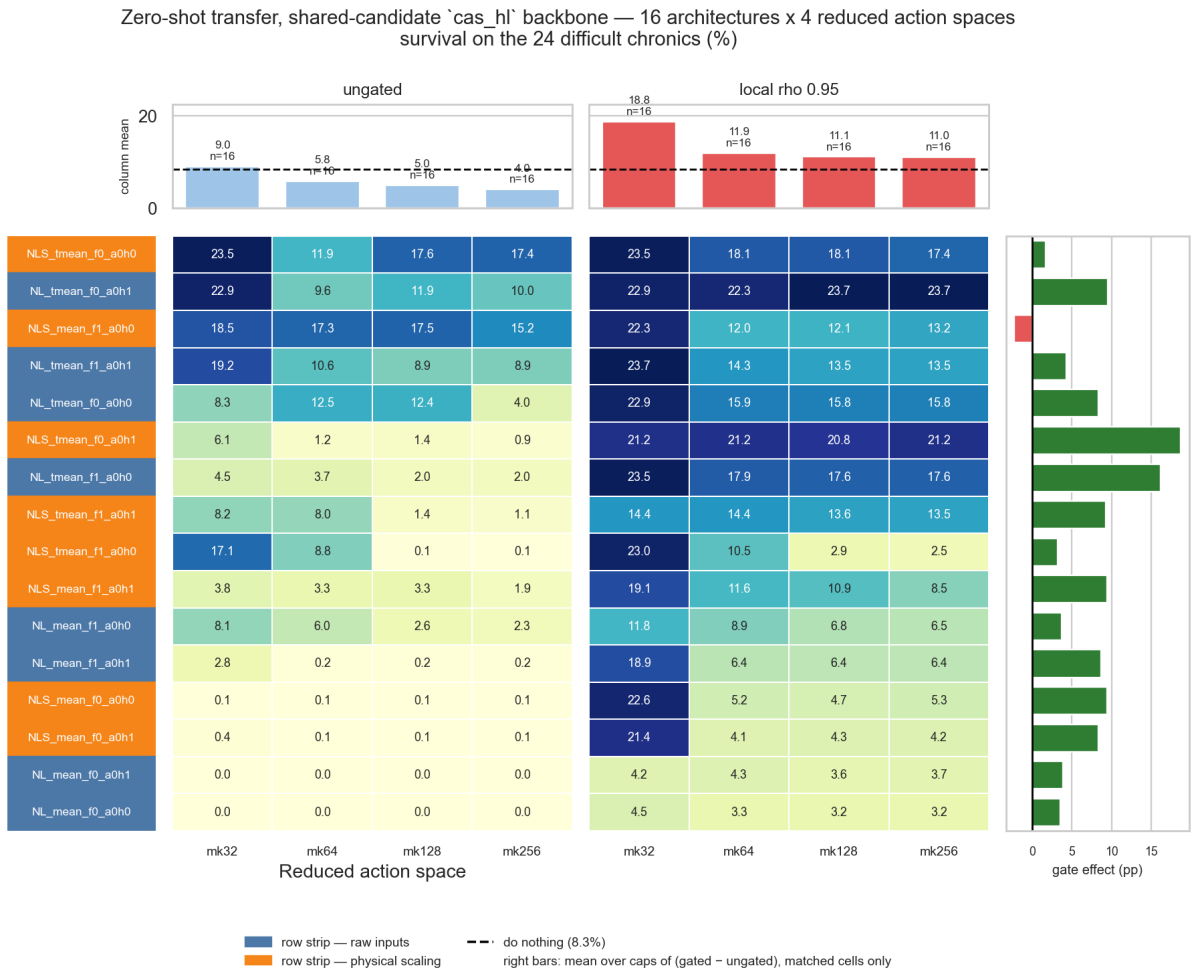


Figure A.12: Zero-shot difficult-cohort survival across input, candidate pooling, descriptor, idle-head and action-space choices.

AIB zero-shot transfer — 15 architectures x 4 reduced action spaces, both gate conditions complete survival on the 24 difficult chronics (%)

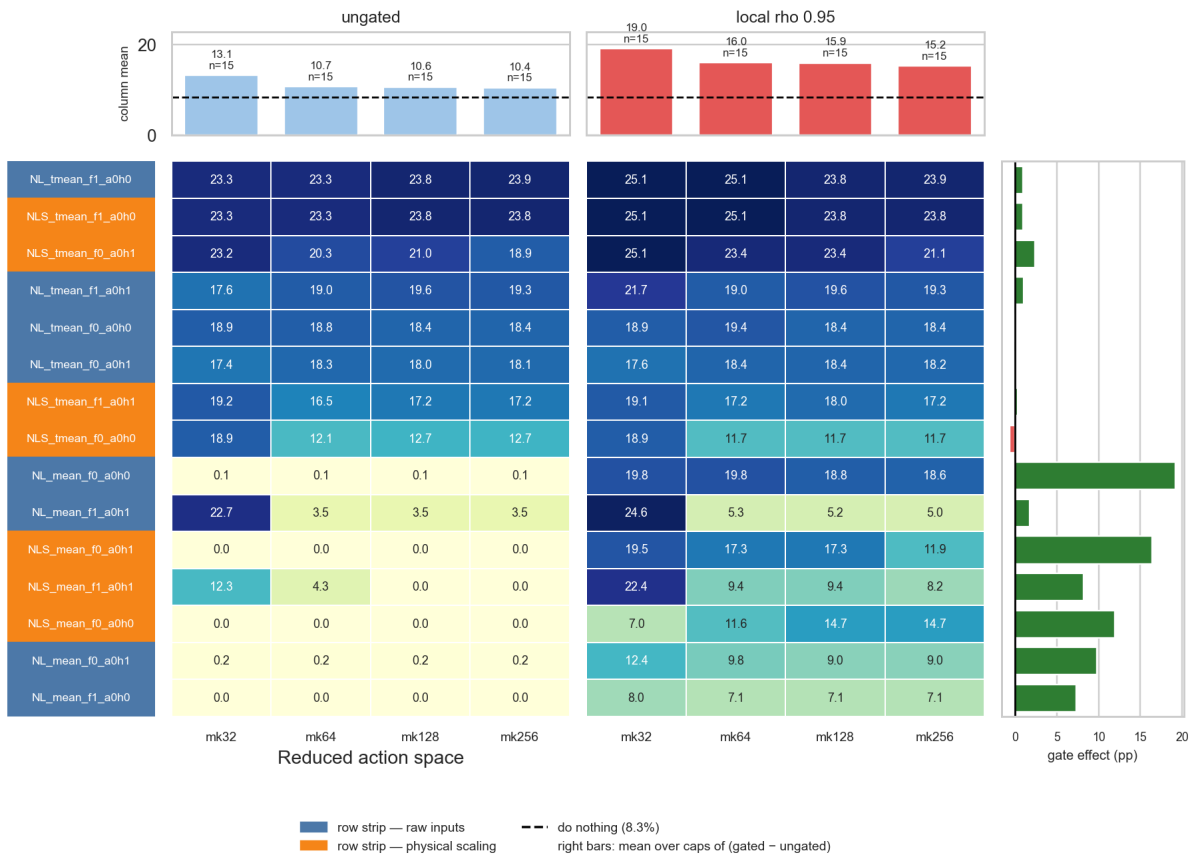


Figure A.13: Difficult-cohort survival with the adaptive intervention budget.

gmax-delta backbone, zero-shot transfer — 15 architectures x 4 reduced action spaces, both gate conditions complete survival on the 24 difficult chronics (%)

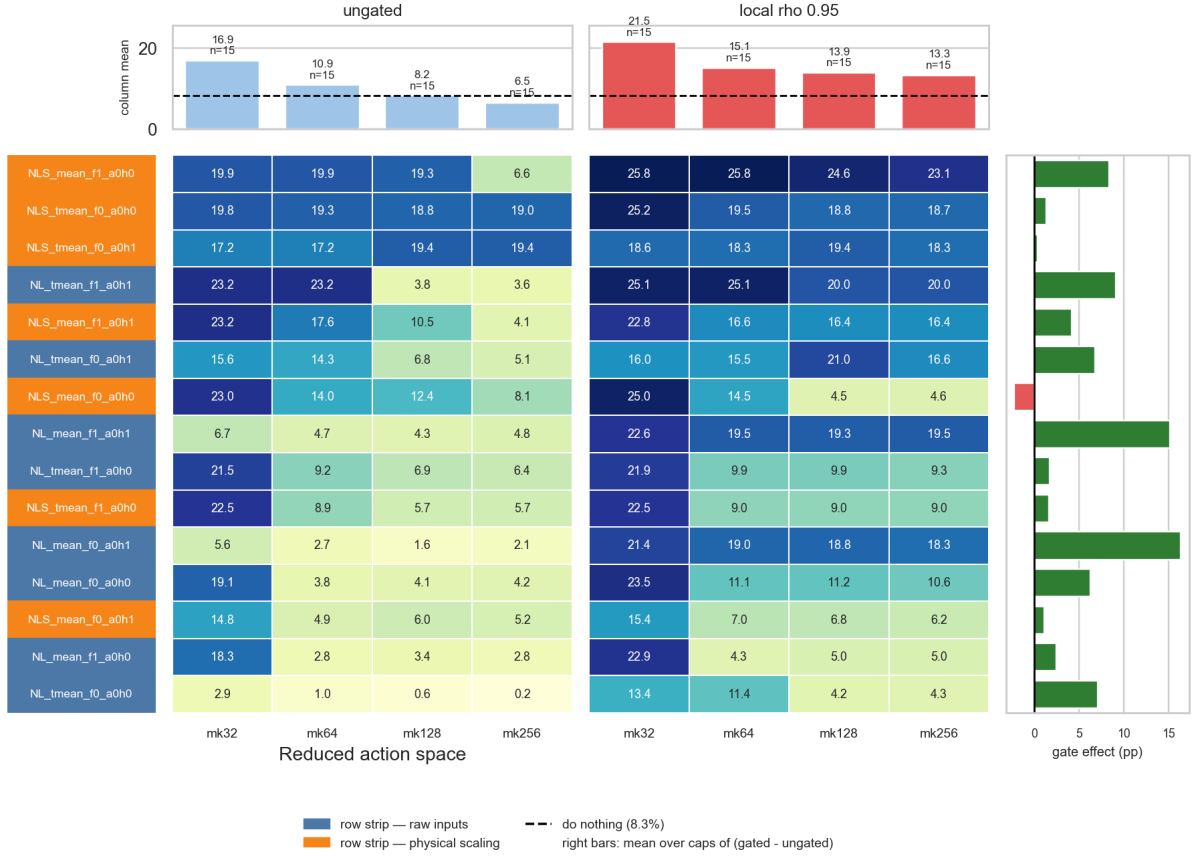


Figure A.14: Difficult-cohort survival with the Gmax-delta action feature.

#### A.4.2 Target-trained controls

Table A.14: Flat-MLP control results. *Easy kept* counts complete episodes among the 26 chronics completed by do nothing.

| $k$ | Checkpoint | Heuristic           | Overall (%) | Difficult (%) | Rescues | Easy kept |
|-----|------------|---------------------|-------------|---------------|---------|-----------|
| 64  | best test  | local $\rho = 0.95$ | 43.20       | 9.79          | 1       | 15        |
| 64  | best test  | none                | 38.22       | 8.50          | 0       | 11        |
| 64  | final      | local $\rho = 0.95$ | 45.33       | 8.55          | 0       | 16        |
| 64  | final      | none                | 32.64       | 8.82          | 0       | 9         |
| 256 | best test  | local $\rho = 0.95$ | 29.80       | 5.39          | 0       | 10        |
| 256 | best test  | none                | 48.55       | 8.92          | 0       | 18        |
| 256 | final      | local $\rho = 0.95$ | 28.20       | 4.59          | 0       | 10        |
| 256 | final      | none                | 50.95       | 7.38          | 0       | 21        |

Table A.15: Fixed-list GNN controls trained directly on WCCI.

| Graph         | $k$ | Checkpoint | Overall (%) | Difficult (%) | Rescues | Easy kept |
|---------------|-----|------------|-------------|---------------|---------|-----------|
| bus_e0n0v0    | 64  | best test  | 20.05       | 3.55          | 0       | 2         |
| bus_e0n0v0    | 64  | final      | 29.47       | 4.50          | 0       | 4         |
| bus_e0n0v0    | 256 | best test  | 33.35       | 4.58          | 0       | 7         |
| bus_e0n0v0    | 256 | final      | 33.67       | 3.90          | 0       | 6         |
| bus_e1n0v0    | 64  | best test  | 37.94       | 5.10          | 0       | 12        |
| bus_e1n0v0    | 64  | final      | 39.63       | 5.15          | 0       | 9         |
| bus_e1n0v0    | 256 | best test  | 26.54       | 4.24          | 0       | 7         |
| bus_e1n0v0    | 256 | final      | 20.63       | 4.64          | 0       | 2         |
| het_ga2b_lb2a | 64  | best test  | 36.05       | 3.08          | 0       | 9         |
| het_ga2b_lb2a | 64  | final      | 39.54       | 4.59          | 0       | 10        |
| het_ga2b_lb2a | 256 | best test  | 31.31       | 4.41          | 0       | 8         |
| het_ga2b_lb2a | 256 | final      | 34.78       | 4.90          | 0       | 10        |
| het_gbi_lbi   | 64  | best test  | 51.35       | 7.21          | 0       | 21        |
| het_gbi_lbi   | 64  | final      | 44.95       | 6.80          | 0       | 15        |
| het_gbi_lbi   | 256 | best test  | 23.62       | 4.70          | 0       | 4         |
| het_gbi_lbi   | 256 | final      | 31.32       | 6.55          | 0       | 5         |

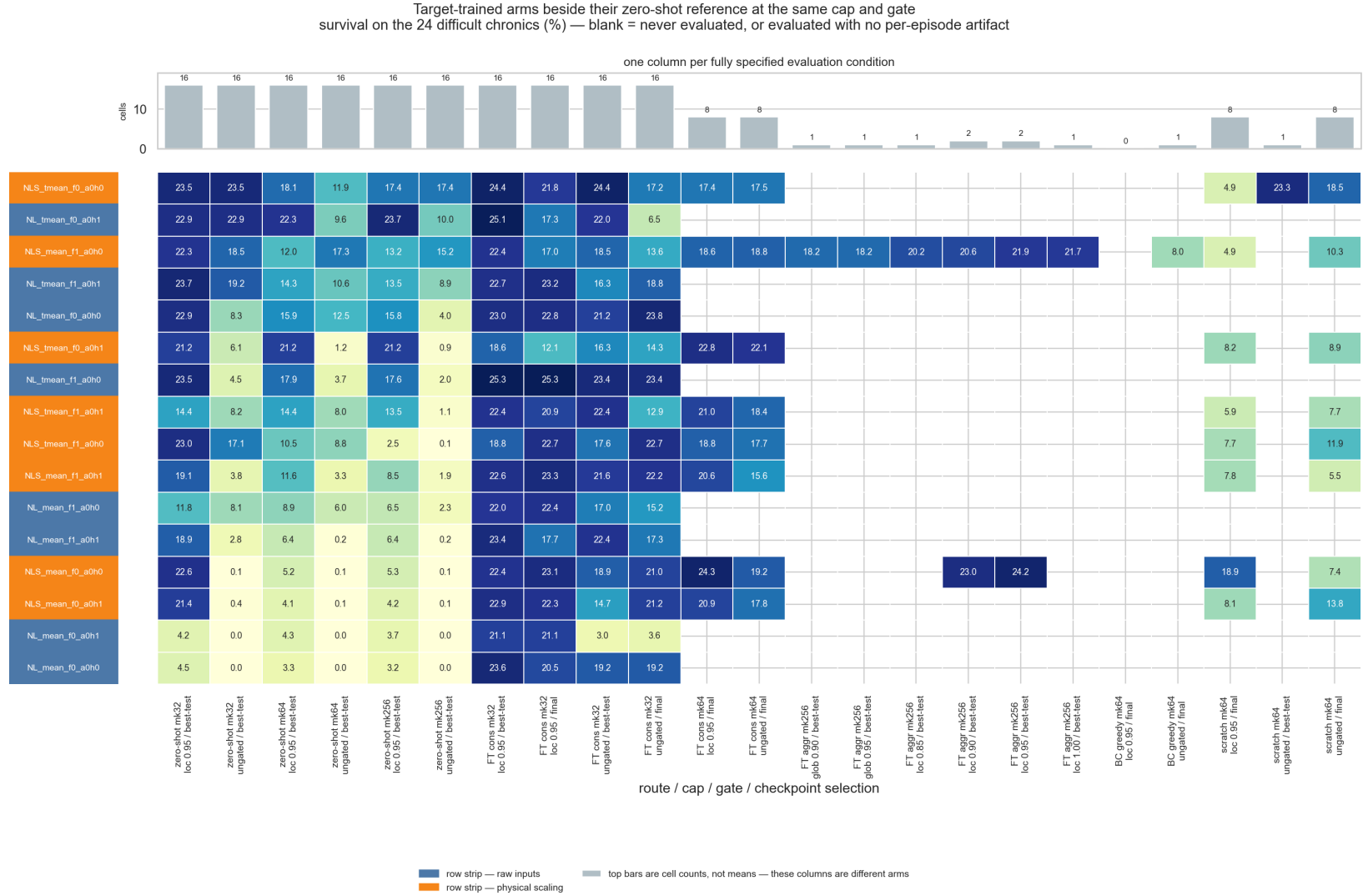


Figure A.15: Detailed comparison of target-trained WCCI controls.

### A.4.3 Per-chronic diagnostic

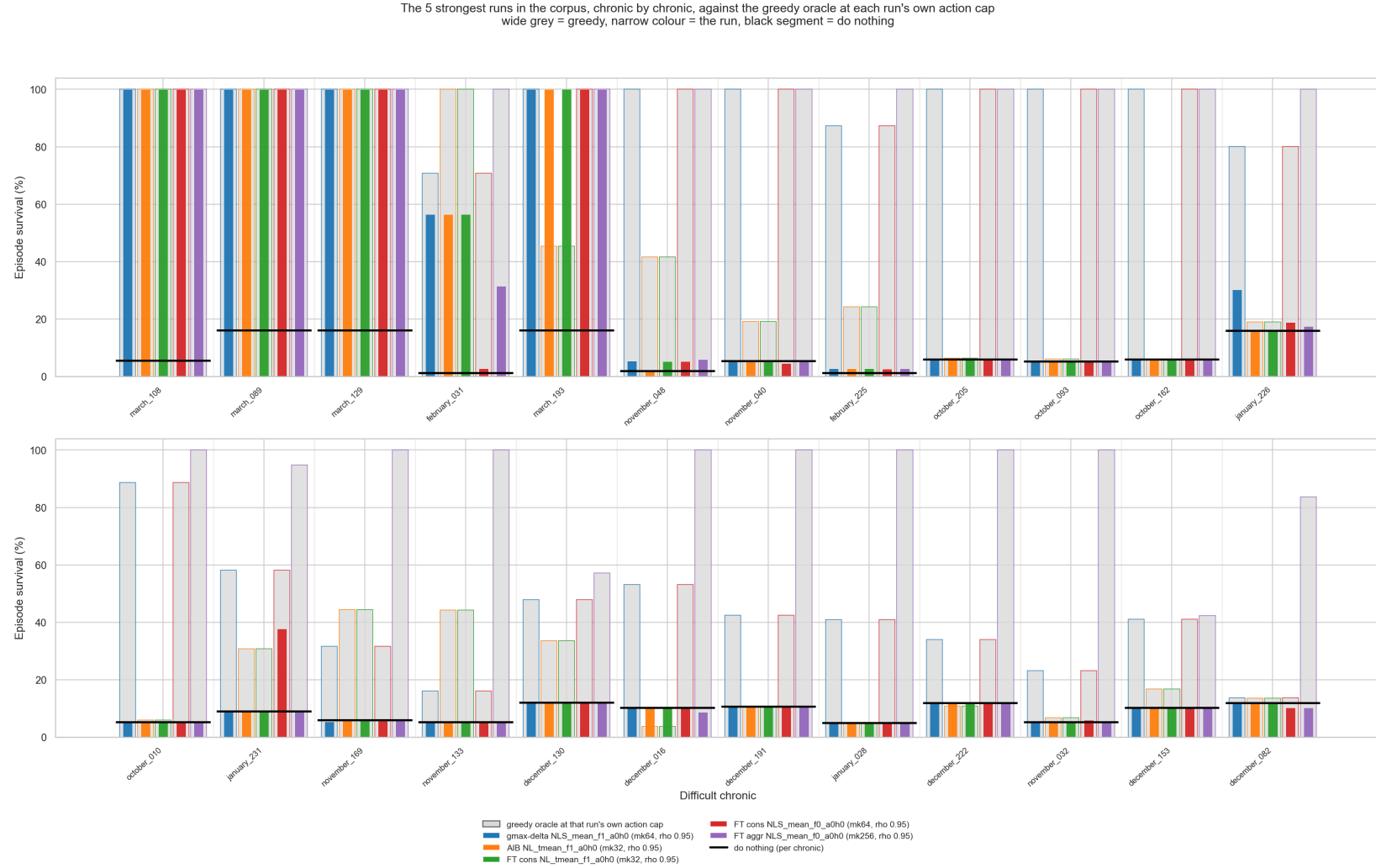


Figure A.16: Per-chronic survival for selected policies on the difficult cohort, with do nothing and the greedy reference.

## Appendix B

# Graph-Policy Implementation Details

This appendix is the implementation reference for the graph policy used throughout the experiments. It brings together the shared encoder, exact feature inventories, candidate-action metadata, graph construction, masking, augmentations, and decentralized information boundaries.

### B.1 Shared input and encoder pipeline

For agent  $a$ , the encoder receives the local graph  $\mathcal{G}_t^{(a)}$ . Candidate edges with `edge_mask`=0 are removed before message passing, so the layer sees only the active directed set  $\mathcal{E}_t^{(a)}$ . The source of  $u \rightarrow v$  sends a message and  $v$  aggregates it. Node and edge masks do not become learned features.

Each raw node vector  $\mathbf{x}_v$  first passes through

$$\mathbf{h}_v^{(0)} = \text{LayerNorm}(\text{ReLU}(\mathbf{W}_x \mathbf{x}_v + \mathbf{b}_x)), \quad (\text{B.1})$$

which produces a 128-dimensional node state. Learned global node-identifier embeddings and the optional external edge pre-encoder are disabled. Every GINE layer nevertheless contains its own trainable projection of the edge vector to the dimension required by that layer.

Two message-passing layers are used. After each layer, the implementation applies ReLU followed by LayerNorm:

$$\mathbf{h}_v^{(k+1)} = \text{LayerNorm}\left(\text{ReLU}(\tilde{\mathbf{h}}_v^{(k+1)})\right). \quad (\text{B.2})$$

The same encoder parameters are shared by all agents. Each agent nevertheless passes its own local graph, with its own node set, active edges, and one-hop context, through that shared encoder.

### B.2 GINE message passing

For an active edge  $u \rightarrow v$ , layer  $k$  projects the complete edge feature vector and adds it to the sender state:

$$\tilde{\mathbf{e}}_{uv}^{(k)} = \mathbf{W}_e^{(k)} \mathbf{e}_{uv} + \mathbf{b}_e^{(k)}, \quad (\text{B.3})$$

$$\mathbf{m}_{u \rightarrow v}^{(k)} = \text{ReLU}\left(\mathbf{h}_u^{(k)} + \tilde{\mathbf{e}}_{uv}^{(k)}\right), \quad (\text{B.4})$$

$$\tilde{\mathbf{h}}_v^{(k+1)} = \text{MLP}^{(k)}\left((1 + \epsilon)\mathbf{h}_v^{(k)} + \sum_{u:(u,v) \in \mathcal{E}_t} \mathbf{m}_{u \rightarrow v}^{(k)}\right). \quad (\text{B.5})$$

The implementation uses the PyTorch Geometric default  $\epsilon = 0$  without a trainable epsilon parameter. Each internal GINE MLP is Linear–ReLU–Linear and outputs 128 features. Equation B.5 shows why edge attributes are part of the message content. Two identical node states can send different messages when one edge is an overloaded physical line and the other is an asset-attachment relation. Likewise,

reversing a typed disaggregated edge activates a different relation-one-hot column and can produce a different message.

GINE uses an unweighted sum over incoming active edges. It does not learn a softmax competition between neighbors. The edge embedding changes what is sent; the number of incoming messages and their vector sum determine the aggregate.

### B.3 Graph readout and output

With a **mean graph readout**, the graph representation after the second message-passing layer is

$$\bar{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \quad (\text{B.6})$$

where  $m_v$  is the dynamic visibility mask. This is one mean over all visible node types, not a separate mean per type. Therefore:

- the busbar graph pools controlled busbars and only the far-end busbars currently attached by live boundary lines;
- the disaggregated graph additionally pools controlled generators and loads, but never neighboring private assets;
- the disaggregated line-node graph additionally pools transmission-line nodes and contains no neighboring equipment.

An inactive contextual candidate contributes neither to the numerator nor to the denominator. The ordinary GINE mean readout includes visible context; **controlled\_mean** replaces  $m_v$  by the product of the visibility and controlled-node masks. Visible context can still influence controlled nodes through message passing, but it is not aggregated directly. **Sum and maximum graph readouts**, including their **controlled\_\*** variants, change only the readout and do not change message passing or the information boundary.

The **read-out** 128-dimensional vector then passes through the shared graph output projection

$$\mathbf{z}_a = \text{ReLU}(\mathbf{W}_o \bar{\mathbf{h}}_G + \mathbf{b}_o), \quad \mathbf{z}_a \in \mathbb{R}^{128}. \quad (\text{B.7})$$

When virtual-node readout is selected, the final virtual-node state replaces  $\bar{\mathbf{h}}_G$ ; the output projection and its dimension stay the same.

### B.4 Agent-specific action head

The flat observation is not concatenated to  $\mathbf{z}_a$ . Each agent has its own two-layer MLP action head:

$$\mathbf{q}_a^{(1)} = \text{ReLU}(\mathbf{W}_a^{(1)} \mathbf{z}_a + \mathbf{b}_a^{(1)}), \quad (\text{B.8})$$

$$\mathbf{q}_a^{(2)} = \text{ReLU}(\mathbf{W}_a^{(2)} \mathbf{q}_a^{(1)} + \mathbf{b}_a^{(2)}), \quad (\text{B.9})$$

$$\boldsymbol{\ell}_a = \mathbf{W}_a^{(3)} \mathbf{q}_a^{(2)} + \mathbf{b}_a^{(3)}, \quad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}, \quad (\text{B.10})$$

$$\pi_a(\cdot \mid \mathcal{G}_t^{(a)}) = \text{Categorical}(\text{logits} = \boldsymbol{\ell}_a). \quad (\text{B.11})$$

Both hidden layers contain 128 units. During training, an action is sampled from this categorical distribution. Deterministic evaluation selects  $\arg \max_j \ell_{a,j}$ . Action zero is the local do-nothing action.

Only the GNN encoder and graph output projection are shared across agents. The action heads are agent-specific because agents can have different local action sets and therefore different output widths  $|\mathcal{A}_a|$ . The critic is a separate MLP over the centralized flat state and does not reuse the actor graph embedding.

Equation B.11 is the default head. The optional candidate-action head of Section 5.4 keeps this pipeline up to  $\mathbf{z}_a$  and additionally consumes the post-message-passing node states  $\mathbf{h}_v^{(K)}$ , which are the same states that Equation B.6 reduces through the graph readout.

## B.5 Candidate-action scorer implementation

A Grid2Op topology action lists the topology-vector entries that it changes. Each entry belongs to a known line endpoint, generator, or load, so the graph builder can map the action once to the node rows used by the scorer. When an action changes a substation, all of its busbar rows are included so that both the current and destination contexts are available. A boundary action includes only the shared line node and controlled endpoint rows already present in the local graph.

Table B.1: One action, from Grid2Op to node rows: moving the origin end of line 4 onto the other busbar of substation 6. Row indices are illustrative.

| Step                       | Result for this action                                                             |
|----------------------------|------------------------------------------------------------------------------------|
| Objects the action changes | Substation 6, line 4                                                               |
| Their rows in the graph    | Both busbars of substation 6, here 12 and 13, and the line node of line 4, here 22 |
| Stored for scoring         | Busbar [12, 13], line [22], generator and load empty                               |

Candidate actions may involve different numbers of nodes. Their node-index lists are therefore padded to the maximum list length, and a Boolean mask excludes padding during aggregation. This fixed tensor shape allows all candidate embeddings to be gathered and processed in parallel.

The optional 12-dimensional descriptor  $\phi_j$  describes the operation rather than the current grid state. Its indicators are binary, and its counts are divided by the largest value in the agent’s action space.

Table B.2: The static action-feature vector  $\phi_j$ .

| Entry                                | Meaning                                |
|--------------------------------------|----------------------------------------|
| <i>Indicators</i>                    |                                        |
| <code>is_do_nothing</code>           | The action is the idle action          |
| <code>has_busbar_context</code>      | It modifies at least one substation    |
| <code>has_line</code>                | It touches at least one line           |
| <code>has_load</code>                | It touches at least one load           |
| <code>has_generator</code>           | It touches at least one generator      |
| <i>Scaled counts</i>                 |                                        |
| <code>n_substations</code>           | Substations modified                   |
| <code>n_topology_changes</code>      | Topology-vector entries changed        |
| <code>n_line_status_changes</code>   | Lines connected or disconnected        |
| <code>n_line_endpoint_changes</code> | Line endpoints moved to another busbar |
| <code>n_generator_changes</code>     | Generators moved to another busbar     |
| <code>n_load_changes</code>          | Loads moved to another busbar          |
| <code>n_other_changes</code>         | Changed objects of any other type      |

## B.6 Exact node inputs by representation

This section consolidates the exact feature inventories moved out of Chapter 5. It is the implementation-level reference for the busbar, disaggregated, and disaggregated line-node schemas.

All node types of one representation receive the same feature columns. Columns not listed for a node type are zero. Type identity reaches GINE through the `is_*` columns. The inventory below describes visible nodes. For an inactive contextual candidate, the complete row—including type and role columns—is zero, all incident edges are inactive, and the node is excluded from readout.

Table B.3: Node types and nonzero input features received by GINE, under the default `gnn_angle_representation = node`. Under `edge_diff` the `gen_theta`, `load_theta` and `theta` entries leave the node rows in every representation; the busbar and disaggregated graphs move the angle drop onto the physical line edge, while the disaggregated line-node graph writes it on the transmission-line row as `theta_diff` (Section 5.5.2).

| Representation          | Node type          | Populated input features                                                                                                                                                                                         |
|-------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bus                     | Busbar             | <code>gen_p</code> , <code>gen_theta</code> , <code>load_p</code> , <code>load_theta</code> , <code>time_before_cooldown_sub</code> , <code>domain_mask</code>                                                   |
| disaggregated           | Busbar             | <code>is_busbar</code> , <code>time_before_cooldown_sub</code> , <code>domain_mask</code>                                                                                                                        |
|                         | Generator          | <code>is_generator</code> , <code>p=gen_p</code> , <code>theta=gen_theta</code> , <code>domain_mask</code>                                                                                                       |
|                         | Load               | <code>is_load</code> , <code>p=load_p</code> , <code>theta=load_theta</code> , <code>domain_mask</code>                                                                                                          |
| disaggregated line-node | Busbar             | <code>is_busbar</code> , <code>time_before_cooldown_sub</code> , <code>domain_mask</code>                                                                                                                        |
|                         | Generator          | <code>is_generator</code> , <code>p=gen_p</code> , <code>theta=gen_theta</code> , <code>domain_mask</code>                                                                                                       |
|                         | Load               | <code>is_load</code> , <code>p=load_p</code> , <code>theta=load_theta</code> , <code>domain_mask</code>                                                                                                          |
|                         | Transmission line  | <code>is_transmission_line</code> , <code>line_status</code> , <code>rho</code> , <code>timestep_overflow</code> , <code>time_before_cooldown_line</code> , optional maintenance times, <code>domain_mask</code> |
| Any augmented schema    | Substation summary | Learned projection of the substation structural descriptor; no raw physical feature row                                                                                                                          |
|                         | Virtual node       | One learned vector shared across graphs; no raw physical feature row                                                                                                                                             |

## B.7 Exact edge inputs by representation

The table below likewise consolidates the physical edge blocks, typed attachment relations, and zero-filled structural relations previously listed separately in Chapter 5.

The edge vector contains all edge information consumed by GINE. Inactive candidates are filtered out before the vectors in Table B.4 reach a convolution.

Table B.4: Directed edge relations and edge-feature vectors received by GINE, under the default `gnn_angle_representation = node`. “Relation” denotes one active relation-one-hot column. Under `edge_diff` the physical-line block of the busbar and disaggregated graphs gains a `theta_diff` column; the line-node rows are unchanged, because there the drop is a line-node feature and the attachment edges stay free of physical channels.

| Representation          | Directed relation                                           | Edge features                                                                                                                                                           |
|-------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bus                     | Physical busbar $\leftrightarrow$ busbar                    | <code>line_status</code> , <code>rho</code> , <code>timestep_overflow</code> , <code>time_before_cooldown_line</code> , optional maintenance times; no relation one-hot |
|                         | Optional self or same-substation                            | Same width; all physical values zero; no relation one-hot                                                                                                               |
| disaggregated           | Physical busbar $\leftrightarrow$ busbar                    | Complete measured physical-line block plus <code>relation_physical_line</code>                                                                                          |
|                         | Generator $\leftrightarrow$ busbar                          | Neutral physical block plus direction-specific generator relation                                                                                                       |
|                         | Load $\leftrightarrow$ busbar                               | Neutral physical block plus direction-specific load relation                                                                                                            |
|                         | Optional self or same-substation                            | Neutral physical block plus the corresponding relation                                                                                                                  |
| disaggregated line-node | Busbar $\leftrightarrow$ line node                          | One origin/extremity-and-direction relation; no line-state attributes                                                                                                   |
|                         | Generator $\leftrightarrow$ busbar                          | One direction-specific generator relation                                                                                                                               |
|                         | Load $\leftrightarrow$ busbar                               | One direction-specific load relation                                                                                                                                    |
|                         | Optional self or same-substation                            | One corresponding relation                                                                                                                                              |
| Any augmented schema    | Busbar $\rightarrow$ summary/virtual, with optional reverse | Zero edge vector; the relation carries no flow                                                                                                                          |

The complete relation names and activation rules are given in Tables B.4 and B.4. Those tables distinguish, for example, generator-to-busbar from busbar-to-generator and origin from extremity line attachments. These distinctions matter directly to an edge-aware model such as GINE, which folds the edge vector into every message.

## B.8 Graph Construction and Local Boundaries

This section collects the construction and sizing details of the three graph schemas of Section 5.1: how large each candidate graph is, how inactive candidates are neutralized, and how a local actor graph is cut out of the full grid. None of it changes the representations themselves.

### B.8.1 Candidate graph sizes

Let  $S$  be the number of included substations,  $B$  the busbars per substation,  $L$  the included physical lines, and  $N_g, N_d$  the generators and loads. Write  $d_g, d_d, d_\ell \in \{1, 2\}$  for the number of retained directions of the generator, load, and line-node attachments: 2 for **bidirectional** and 1 for either one-way mode. Let  $P_\ell$  be the number of represented line endpoints: an internal line contributes two and a boundary line in a local disaggregated line-node graph contributes only its controlled endpoint. Thus  $L \leq P_\ell \leq 2L$ . Table B.5 gives the size of the fixed candidate graph before any mask is applied.

Table B.5: Size of the candidate graph of each schema. Enabling self edges adds  $|\mathcal{V}|$  directed edges and enabling same-substation edges adds  $SB(B - 1)$ , in every schema.

| Schema                  | Candidate nodes      | Candidate directed edges              |
|-------------------------|----------------------|---------------------------------------|
| <b>bus</b>              | $SB$                 | $2B^2L$                               |
| disaggregated           | $SB + N_g + N_d$     | $2B^2L + B(d_gN_g + d_dN_d)$          |
| disaggregated line-node | $SB + N_g + N_d + L$ | $Bd_\ell P_\ell + B(d_gN_g + d_dN_d)$ |

Three consequences are worth naming. A busbar graph with both optional relations enabled therefore has  $2B^2L + SB + SB(B - 1)$  candidate directed edges. The disaggregated schema adds  $N_g + N_d$  nodes and between  $B(N_g + N_d)$  and  $2B(N_g + N_d)$  asset candidates to the busbar graph, and buys individual asset states with them. For a complete state graph,  $P_\ell = 2L$ , so the disaggregated line-node schema replaces the  $2B^2L$  busbar-to-busbar term by  $2Bd_\ell L$ : identical at  $B = 2$  and cheaper for  $B > 2$ . A local boundary line has candidates only at its controlled endpoint, which reduces  $P_\ell$  and prevents construction of a far-end attachment. The schema adds  $L$  line nodes and one message-passing hop for internal endpoint exchange.

The hierarchical augmentations of Section 5.2 add on top of any of these: one substation-summary node per substation, with  $SB$  directed edges for **toward\_summary** or  $2SB$  for **bidirectional**; and one virtual node, with  $N_{\text{bus}}$  or  $2N_{\text{bus}}$  edges, or  $S$  or  $2S$  when summary nodes are also enabled.

### B.8.2 Indexing and masking conventions

Grid2Op numbers busbars from one in its topology vector; those identifiers are converted to the zero-based index  $b$  used in the node identifier  $i(s, b)$ .

An inactive candidate edge keeps its row in the edge-feature tensor, so the tensor shape is fixed for an episode, but the row is zero-filled and its **edge\_mask** entry is 0. Masked edges are removed before the encoder is called, so those zero rows never generate a message. This is what allows a topology change to alter connectivity without rebuilding the graph. An inactive contextual node likewise retains its candidate row, but its complete feature vector is zero and its **node\_mask** entry is 0. Every incident physical or structural edge is deactivated. The row therefore participates in neither message passing nor graph readout; a mean readout excludes it from both numerator and denominator.

### B.8.3 Substation augmentation details

Figure B.1 shows where the direct busbar relation and substation-summary node are inserted in each graph schema. Both augmentations remain restricted to controlled substations.

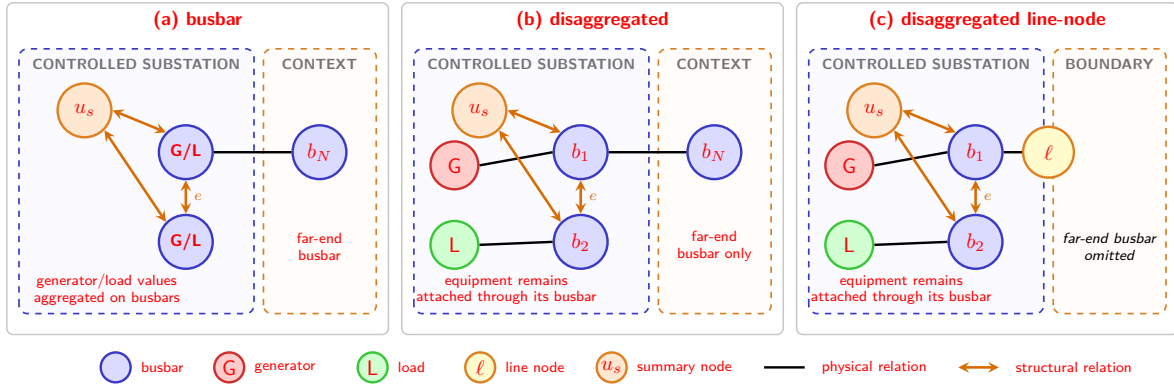


Figure B.1: Placement of the two augmentations when  $e = n = 1$ . In every graph representation, the direct relation and summary node attach only to controlled busbars. Equipment and line nodes retain their ordinary busbar relations, and no augmentation is added in the contextual domain.

Table B.6: Node and edge features introduced by the direct same-substation augmentation. “None” means that the augmentation neither adds nodes nor changes the features of existing nodes.

| Graph schema            | Added node features | Feature values on each added directed edge                                                                                                                                                      |
|-------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Busbar                  | None                | Every entry is 0: this schema has no relation columns. An active physical line always carries <code>line_status=1</code> , so an all-zero row is itself the signature of a structural relation. |
| Disaggregated           | None                | <code>relation_same_substation_busbar=1</code> ; every other entry is 0.                                                                                                                        |
| Disaggregated line-node | None                | <code>relation_same_substation_busbar=1</code> ; every other entry is 0.                                                                                                                        |

#### B.8.4 Local actor graphs

A local actor graph always contains the controlled substations and every line touching them. Additional membership is schema-specific. The `bus` graph contains only the far-end busbar currently attached by each live boundary line, and that busbar exposes aggregated neighboring power and angle. The direct disaggregated graph contains the same far-end busbar, but never its generator or load nodes. The disaggregated line-node graph represents the shared line itself and therefore constructs no neighboring busbar or private asset and no far-end attachment edge. The centralized state graph is intentionally unrestricted for centralized training. The ordinary graph readout aggregates all visible nodes, whereas the `controlled_*` graph readout restricts readout to action-domain nodes; neither choice changes the construction boundary.

Table B.7: Information from another agent’s domain visible to a local actor.

| Representation          | Neighbor information available                                                                           |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| busbar                  | Shared-line edge and attached far-end busbar, including aggregated generator/load power and angles       |
| disaggregated           | Shared-line edge and attached far-end busbar; no neighboring generator/load nodes or private-asset state |
| disaggregated line-node | Shared-line node only; no neighboring busbar, equipment, or far-end attachment                           |

### B.8.5 Boundary verification

The regression suite checks that contextual visibility follows a live busbar attachment, moves when the attachment is switched, and disappears after a line outage; controlled nodes remain present; structural relations cannot make a hidden candidate visible; inactive rows are zero; neighboring private assets are absent from disaggregated graphs; and all neighboring equipment is absent from disaggregated line-node local graphs. Candidate-action tests additionally verify that a boundary line remains addressable through its line node without adding far-end equipment. Controlled-readout tests verify both that the pooled set is fixed and that gradients can still reach visible context through message passing.

# Appendix C

## Reproducibility Summary

This appendix records the protocol choices needed to interpret and reproduce the main experimental blocks. It replaces the previous configuration archive and checkpoint-step registry with a compact description of what was held fixed, what changed between campaigns, and how checkpoints were selected. Exact machine-readable configurations and evaluation JSON files remain in the project repository.

### C.1 Campaign protocols

Table C.1: Training, selection, and evaluation protocols used by the main experimental blocks.

| Campaign                     | Environment and budget                                                                      | Checkpoint selection and evaluation                                                                                             | Statistical scope                                                       |
|------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Stable MLP baseline          | <code>bus14</code> ; 15M steps; 20 parallel environments                                    | Periodic deterministic 20-chronic training-split rollouts; retain the highest rollout mean                                      | 80/20 chronic split; multi-seed core comparisons                        |
| Preliminary GINE viability   | <code>bus14</code> ; 15M steps; 2,000 rollout steps                                         | Deterministic evaluation every 80,000 steps under the same split as the MLP baseline                                            | Exploratory architecture study                                          |
| Screens B–F                  | <code>bus14</code> ; 15M steps; 72 environments and 576 rollout steps                       | Evaluation every 82,944 steps on a changing 20-chronic training rollout; selected checkpoint evaluated on 201 held-out chronics | One seed per factorial cell (seed 0)                                    |
| Input-distribution study     | <code>bus14</code> and WCCI; 6,000 do-nothing steps; 300-step cap per chronic               | No learned checkpoint; every visible local-graph node is recorded                                                               | 426,000 and 1,236,000 node observations, respectively                   |
| WCCI transfer and adaptation | <code>bus36_wcci_nomaint</code> ; zero-shot, 15M-step scratch, or about 5M-step fine-tuning | Zero-shot uses the selected source checkpoint; WCCI-trained actors use their final checkpoint                                   | Seed 0; the same 50 chronics, split into 26 easy and 24 difficult cases |

### C.2 Shared MAPPO settings

The graph-design screens use the same optimization recipe so that each factorial changes only the named architectural choice:

Table C.2: MAPPO settings shared by Screens B–F.

| Component        | Fixed setting                                                                                                              |
|------------------|----------------------------------------------------------------------------------------------------------------------------|
| Optimization     | 10 update epochs, 16 minibatches, learning rate $10^{-4}$ with linear decay, maximum gradient norm 1.0                     |
| Returns and PPO  | $\gamma = 0.9$ , GAE $\lambda = 0.95$ , clip ratio 0.2, target KL 0.02, entropy coefficient 0.01                           |
| Actor and critic | Shared graph encoder with two 128-unit action-head layers; centralized three-layer, 256-unit MLP critic                    |
| Preprocessing    | Normalized flat observations and rewards; graph physical scaling and sparse-control mechanisms disabled during the screens |
| Exploration      | No initial do-nothing prior and no action-0 logit bonus                                                                    |

### C.3 Exploratory GINE variants

The preliminary viability study varied several choices simultaneously before the controlled Screens B–F. Table C.3 retains the principal configurations needed to interpret that exploratory comparison; isolated development variants remain in the machine-readable run configurations.

Table C.3: Principal configurations in the preliminary GINE viability study.

| Run                             | GINE size      | Critic | Actor/critic heads            | Main difference from the heavy baseline                                                              |
|---------------------------------|----------------|--------|-------------------------------|------------------------------------------------------------------------------------------------------|
| Heavy GINE Baseline             | $2 \times 128$ | GNN    | $3 \times 256 / 3 \times 256$ | Flat concatenation, edge pre-encoder, entropy 0.02, action-0 prior 0.7, legacy critic updates        |
| Light GINE (Concat, MLP Critic) | $1 \times 64$  | MLP    | $2 \times 128 / 2 \times 128$ | Smaller actor, no edge pre-encoder, optimized critic updates                                         |
| Heavy GINE (No Bias)            | $2 \times 128$ | GNN    | $3 \times 256 / 3 \times 256$ | Heavy baseline with the initial action-0 prior removed                                               |
| No Bias, Opt Critic             | $2 \times 128$ | GNN    | $3 \times 256 / 3 \times 256$ | No action-0 prior and optimized critic updates                                                       |
| Optimized Heavy GINE            | $2 \times 128$ | GNN    | $3 \times 256 / 3 \times 256$ | No action-0 prior, entropy 0.01, optimized critic updates                                            |
| Optimized Light GINE            | $1 \times 64$  | MLP    | $2 \times 128 / 2 \times 128$ | No flat concatenation or edge pre-encoder; entropy 0.01, no action-0 prior, optimized critic updates |

Table C.4 records the principal outcomes retained from this exploratory campaign. Figure C.2 gives the broader training-curve comparison over isolated development variants.

Table C.4: Survival summary for the principal preliminary GINE variants.

| Run                             | Intended change                      | Final survival (%) | Peak survival (%) |
|---------------------------------|--------------------------------------|--------------------|-------------------|
| Heavy GINE Baseline             | Historical GINE baseline             | 59.2               | 93.9              |
| Light GINE (Concat, MLP Critic) | Light GINE with MLP critic           | 78.9               | 90.4              |
| No Bias, Opt Critic             | Optimized critic and bias 0.0        | 82.5               | 85.7              |
| Optimized Light GINE            | Light optimized GINE with MLP critic | 95.7               | 97.4              |

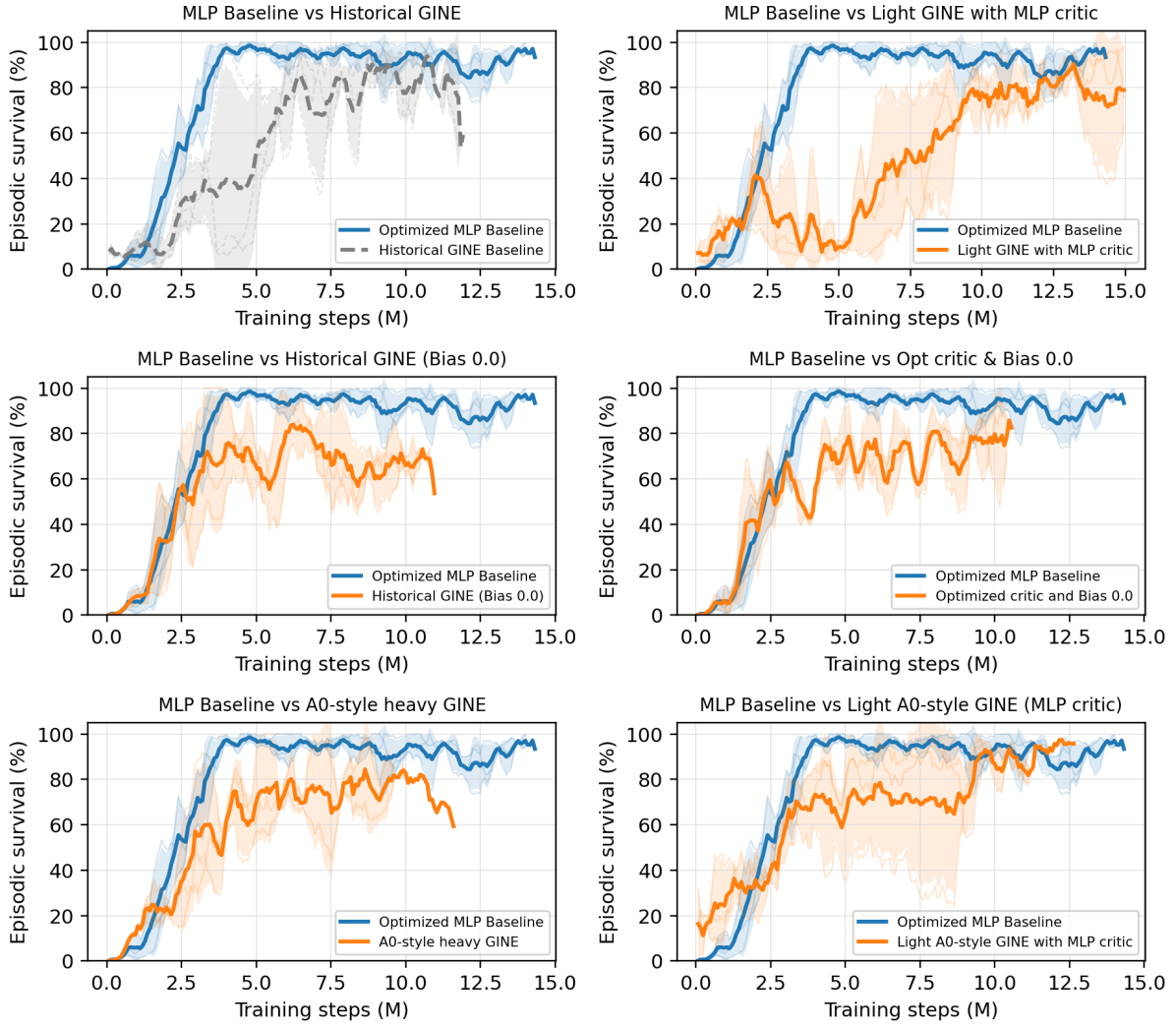


Figure C.1: Training comparison between the optimized MLP baseline and the strongest shared GINE variants in the preliminary viability study.

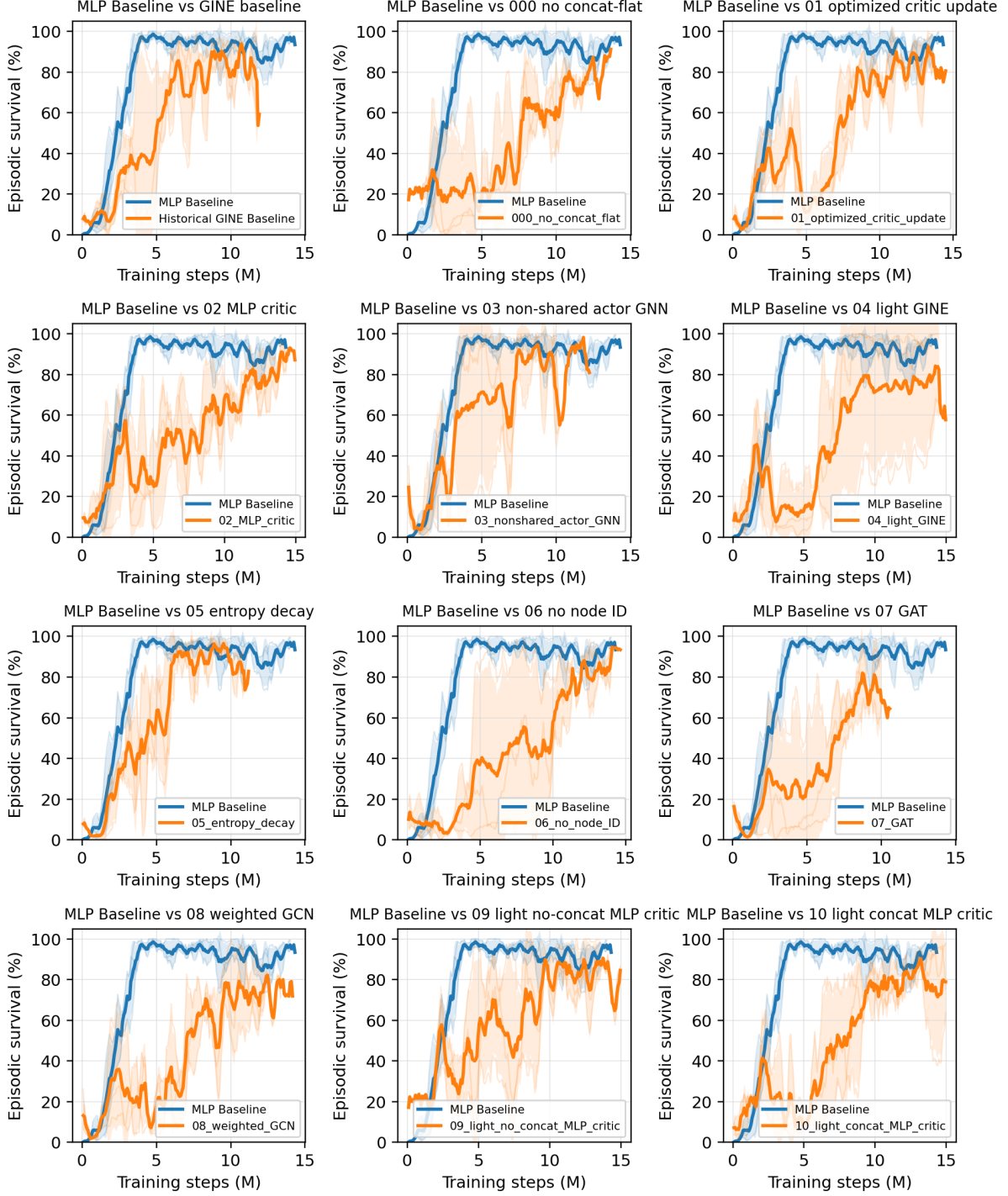


Figure C.2: Broader preliminary GINE comparison. Each subplot compares the MLP baseline with one exploratory variant. The campaign mixes several architectural and optimization changes and is therefore used as supporting evidence rather than as a controlled factorial.

## C.4 Checkpoint and artifact provenance

For bus14, checkpoint selection uses only the training split. At every evaluation, a new set of 20 training chronics is rolled out, and the checkpoint score is their mean survival. The selected checkpoint is then evaluated on the held-out chronics used by the corresponding experiment. The graph-design screens use all 201 held-out chronics for this comparison.

Zero-shot WCCI evaluation reuses the selected `bus14` checkpoint without target updates. Actors trained or fine-tuned on WCCI are reported at their final checkpoint, and the transfer comparisons use the same 50 chronics and seed 0. Exact checkpoint identifiers, per-seed global steps, action summaries, and evaluation outputs remain in the project artifacts rather than being duplicated as a thesis table. This also avoids mixing MAPPO environment steps with behavioral-cloning optimizer steps, which are not directly comparable.

# Bibliography

- [1] E. B. Fisher, R. P. O'Neill, and M. C. Ferris, "Optimal transmission switching," *IEEE Transactions on Power Systems*, vol. 23, no. 3, pp. 1346–1355, 2008.
- [2] J. Viebahn, M. Naglic, A. Marot, B. Donnot, and S. H. Tindemans, "Potential and challenges of AI-powered decision support for short-term system operations," in *CIGRE Paris Session*, 2022.
- [3] B. Donnot, "Grid2op: A testbed platform to model sequential decision making in power systems," *arXiv preprint arXiv:2011.07929*, 2020.
- [4] A. Marot et al., "Learning to run a power network challenge for training topology controllers," *Electric Power Systems Research*, 2021.
- [5] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *International Conference on Machine Learning*, 1993.
- [6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, 2017.
- [7] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, 2022.
- [8] M. Hassouna, C. Holzhüter, P. Lytaev, J. Thomas, B. Sick, and C. Scholz, *Graph reinforcement learning for power grids: A comprehensive survey*, 2024. arXiv: [2407.04522](https://arxiv.org/abs/2407.04522). [Online]. Available: <https://arxiv.org/abs/2407.04522>
- [9] M. Subramanian, J. Viebahn, S. H. Tindemans, B. Donnot, and A. Marot, "Exploring grid topology reconfiguration using a simple deep reinforcement learning approach," in *IEEE Madrid PowerTech*, 2021.
- [10] D. Yoon, S. Hong, B.-J. Lee, and K.-E. Kim, "Winning the L2RPN challenge: Power grid management via semi-Markov afterstate actor-critic," in *International Conference on Learning Representations*, 2021.
- [11] A. Chauhan, M. Baranwal, and A. Basumatary, "PowRL: A reinforcement learning framework for robust management of power networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 14 757–14 764.
- [12] M. Lehna, J. Viebahn, A. Marot, S. Tomforde, and C. Scholz, "Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents," *Energy and AI*, vol. 14, p. 100 276, 2023.
- [13] B. Manczak, J. Viebahn, and H. van Hoof, *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: [2311.02129](https://arxiv.org/abs/2311.02129). [Online]. Available: <https://arxiv.org/abs/2311.02129>
- [14] E. van der Sar, A. Zocca, and S. Bhulai, *Multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: [2310.02605](https://arxiv.org/abs/2310.02605). [Online]. Available: <https://arxiv.org/abs/2310.02605>
- [15] B. de Mol, D. Barbieri, J. Viebahn, and D. Grossi, "Centrally coordinated multi-agent reinforcement learning for power grid topology control," in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems (e-Energy)*, 2025.
- [16] E. Marchesini et al., "MARL2Grid-TR: A multi-agent RL benchmark in power grid operations," in *International Conference on Learning Representations*, 2026.
- [17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.

- [18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [19] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *International Conference on Learning Representations*, 2019.
- [20] M. de Jong, J. Viebahn, and Y. Shapovalova, *Generalizable graph neural networks for robust power grid topology control*, 2025. arXiv: [2501.07186](https://arxiv.org/abs/2501.07186). [Online]. Available: <https://arxiv.org/abs/2501.07186>
- [21] E. Marchesini et al., *RL2Grid: Benchmarking reinforcement learning in power grid operations*, 2025. arXiv: [2503.23101](https://arxiv.org/abs/2503.23101). [Online]. Available: <https://arxiv.org/abs/2503.23101>
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [23] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” *International Conference on Machine Learning*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.10528>
- [24] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID lagrangian methods,” *International Conference on Machine Learning*, 2020. [Online]. Available: <https://arxiv.org/abs/2007.03964>
- [25] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, “Budgeted reinforcement learning in continuous state space,” *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.01004>
- [26] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: [2502.00040](https://arxiv.org/abs/2502.00040). [Online]. Available: <https://arxiv.org/abs/2502.00040>
- [27] W. Hu et al., “Strategies for pre-training graph neural networks,” in *International Conference on Learning Representations*, 2020.